Claude Code'u iyi kullanmanın kontrol listesi

19 bölüm, 113 madde. Başlangıç için on madde, her maddede neden, nasıl, kopyalanabilir örnek ve doğrulama.

29 Ekim 2026 · Kaynak kontrolü; yerel CLI 2.1.284
Claude Code belgelerine dayanan teknik notlar ve editoryal iş akışı önerileri.

Nasıl okunur

Bu liste tekrar işi azaltmak, kapsamı belirlemek ve sonucu kanıtla teslim almak için hazırlanmış bir çalışma rehberidir. Her madde her projeye uygun değildir; kullanımına uygun olanları seç.

Neden gerekçeyi, **nasıl** uygulama yolunu, **doğrula** beklenen kanıtı gösterir. Başlangıç onlusu ve temel/ileri etiketleri editoryal öneridir; ölçülmüş getiri sıralaması değildir.

Temel 25, **ileri** 88 madde vardır. Teknik davranışların yanında resmî kaynak bağlantısı bulunur; bağlantısız maddeler iş akışı önerisidir. Örnekler kendi projenin araçlarına ve yetkisine uyarlanmalıdır.

Kaynaklar 21 Ekim 2026'da kontrol edildi, yeni altı madde ve ekler 29 Ekim 2026'da. Yerel CLI 2.1.284 için yalnız sürüm ve yardım çıktısı alındı; listelenen bütün Claude özellikleri canlı çalıştırılmadı. CLI, Desktop, bulut ve sağlayıcı farklarını ilgili belgede kontrol et.

Bir kutuyu işaretlemek kişisel ilerleme kaydıdır; otomatik doğrulama veya güvenlik garantisi değildir. Tarayıcı verisi temizlenebilir; ilerlemeni JSON olarak dışa aktarabilirsin.

Önce bu on madde

Bütün listeyi okumaya vaktin yoksa buradan başla. Bunlar başlangıç önerileri; işine uymayanı atlayabilirsin.

1 **Bitiş ölçütünü baştan yaz** madde 3.1
Ölçüt yoksa iş parça parça ilerler ve her parçada seni bekler.

2 **Tutarlı oturum ve devir kaydı** madde 2.1
Birbirinden kopuk işleri aynı bağlama yığmak kararları karıştırabilir; her küçük adımda yeni oturum açmak da tekrar yaratır.

3 **Kısa ve doğrulanmış proje talimatı** madde 1.2
Derleme komutu ve tuzaklar her oturumda yeniden keşfedilmesin.

4 **Projeye izin listesi** madde 6.1
Projeye ait dar liste tekrarlanan izinleri azaltabilir; test betikleri de kod çalıştırdığı için önce incelenmelidir.

5 **Gizli dosyaları okuma dışı bırak** madde 7.1
Gizli verinin bağlama veya çıktıya girmesini azaltmak gerekir; tek bir araç kuralı tam yalıtım sağlamaz.

6 **Alt ajan modelini ölçerek seç** madde 5.1
Daha küçük model her doğrulama işinde yeterli olmayabilir. Aynı temsilî işte kalite, süre ve kullanımı karşılaştır.

7 **"Geçti" yerine sayıyı oku** madde 12.1
Eşleşmeyen bir test filtresi sıfır test koşup başarılı diyebilir.

8 **Kaynak kodu Git'te, commit kapsamı belli** madde 15.1
Küçük, anlamlı commit geri almayı kolaylaştırır. Git dışı veriler ve sırlar için ayrıca uygun yedek gerekir.

9 **Düşünme bütçesini işe göre ayarla** madde 5.3
Mimari karar ile dosya yeniden adlandırma aynı bütçeyi istemez. "Think hard" gibi ifadeler sıradan metin olarak geçer, hiçbir ayarı değiştirmez. Yalnız `ultrathink` o tur için ek bir talimat ekler, effort ayarı yine aynı kalır.

10 **Eleştiriyi yazandan ayrı bağlama ver** madde 9.1
Ayrı bağlam farklı varsayımlarla inceleme yapabilir; bağımsızlık kalite garantisi değildir.

1 Talimat katmanları

Yeni oturumun konuşma bağlamı yenidir; talimat dosyaları ve otomatik bellek oturumlar arasında bilgi taşıyabilir. Kalıcı kuralları kısa tut, ayrıntılı prosedürleri ilgili işe taşınacak dosyalara ayır.

- [ ] **1.1 Global CLAUDE.md'ye yalnız varsayılan davranışı değiştiren kuralları yaz.** TEMEL

  NEDEN Talimatlar bağlamda yer kaplar. Gereksiz tekrar, ilgili kuralları bulmayı zorlaştırır.

  NASIL `~/.claude/CLAUDE.md` . Genel tavsiye değil, davranış yaz.

  ```
  # Davranışı tarif et
  - Yetkilendirilmiş değişiklikleri anlamlı commit'lere böl.
  - Push ve yayını kullanıcının belirttiği hedef ve kapsam içinde yap.
  - Karar sorarken önerini ve gerekçeni ekle.
  ```

  DOĞRULA Her kural için sor: silsem davranış değişir mi? Değişmiyorsa sil.

  Belge: Talimat ve bellek

- [ ] **1.2 Her aktif projede kısa ve doğrulanmış bir CLAUDE.md tut.** TEMEL

  NEDEN Derleme komutu ve tuzaklar her oturumda yeniden keşfedilmesin.

  NASIL `/init` taslağını gözden geçir; gerçek test ve derleme komutlarını yaz. 100 satır bir ürün sınırı değildir. Uzun prosedürü skill veya kapsamlı kurala ayır.

  ```
  # Proje: fiyat-izleyici
  ## Komutlar
  - Test: npm test -- --run
  - Tek dosya: npm test -- src/parser.test.ts
  ## Tuzaklar
  - data/ altındaki JSON elle düzenlenmez, scripts/topla.mjs üretir.
  - Saat dilimi her yerde Europe/Istanbul.
  ```

  DOĞRULA Yeni oturumda dosya okutmadan "test komutu ne?" diye sor. Doğru cevap gelmeli.

  Belge: Talimat ve bellek

- [ ] **1.3 Makineye özgü bilgiyi `CLAUDE.local.md` 'ye yaz ve gitignore'la.** İLERİ

  NEDEN Repo başka makinede ya da bulut oturumunda açılınca yanlış yol talimatı kalmaz.

  NASIL Proje kökünde `CLAUDE.local.md` : venv yeri, yerel port, cihaz adı.

  DOĞRULA `git check-ignore CLAUDE.local.md` eşleşme döndürmeli.

  Belge: Talimat ve bellek

- [ ] **1.4 Yalnız belli dosyalarda geçerli kuralı `.claude/rules/` altına koy.** İLERİ

  NEDEN Kural yalnız eşleşen dosyaya dokunulunca yüklenir, geri kalan zamanda bağlamı doldurmaz.

  NASIL Frontmatter'da `paths` . Kullanıcı düzeyi için `~/.claude/rules/` .

  ```
  ---
  paths: ["**/*.swift"]
  ---
  - Görünüm dosyalarında iş mantığı yok, ViewModel'e taşı.
  - Yeni ekran eklenince Previews da eklenir.
  ```

  DOĞRULA Eşleşen bir dosyayı okut, ardından `/context` ile kuralın yüklendiğini gör.

  Belge: Talimat ve bellek

- [ ] **1.5 Dış dosya import'unun gerçekten yüklendiğini doğrula.** İLERİ

  NEDEN Proje dışı import onayı ve çalışma modu, hangi bilginin yüklendiğini etkileyebilir.

  NASIL Gerekli dosyayı `@path` ile belirt, dış import onayını yönet ve `/memory` listesinden kontrol et. SessionStart yalnız gerekli dinamik bilgiyi eklesin; onay atlatma yolu olarak kullanılmasın.

  DOĞRULA Etkileşimli ve headless kullanımını ayrı dene; yüklenmeyen bilgiyi varsayma.

  Belge: Talimat ve bellek

- [ ] **1.6 Başka bir ajanla paylaşılan repoda kuralları tek dosyada tut.** İLERİ

  NEDEN İki dosya ayrı ayrı güncellenince birbirinden kopar.

  NASIL Güncel sürüm AGENTS.md'yi doğrudan okuyabilir; aynı dizinde veya üstünde CLAUDE.md/CLAUDE.local.md bulunması varsayılan seçimi değiştirir. Ortak dosyayı uyumlu biçimde paylaşmak için aşağıdaki import kullanılabilir. `/memory` ile yüklenen dosyayı doğrula.

  ```
  # CLAUDE.md
  @AGENTS.md
  ```

  DOĞRULA Her iki araçta yüklenen dosya ve kritik kural aynı; benzer cevap tek başına yüklenme kanıtı değildir.

  Belge: Talimat ve bellek

- [ ] **1.7 Otomatik belleği düzenli olarak gözden geçir.** İLERİ

  NEDEN Talimat dosyası ile modelin biriktirdiği notlar farklıdır; eski çıkarım sonraki işi etkileyebilir.

  NASIL `/memory` ile kayıtları ve etkin ayarı incele; yanlış veya artık geçerli olmayan bilgiyi düzelt.

  DOĞRULA Yeni oturum kritik notu doğru kullanıyor; otomatik belleğe güvenilerek açık iş kaydı atlanmıyor.

  Belge: Talimat ve bellek

- [ ] **1.8 Ayarın hangi katmandan geldiğini doğrula.** İLERİ

  NEDEN Aynı anahtar kullanıcı, proje, yerel ve yönetilen katmanda farklı olabilir. Beklemediğin davranışın kaynağı çoğu zaman yanlış katmandaki bir ayardır.

  NASIL `/status` içindeki `Setting sources` satırı o oturumda yüklenen ayar dosyalarını listeler, tek tek anahtarın kaynağını göstermez. Öncelik yüksekten düşüğe: yönetilen, komut satırı, `settings.local.json`, `.claude/settings.json`, kullanıcı. İzin listeleri birleşir, ezilmez. Ortam değişkeni ayrı işler: kabuktaki `ANTHROPIC_MODEL` dosyadaki `model` anahtarını geçer.

  ```
  grep -n '"model"' ~/.claude/settings.json .claude/settings.json
  .claude/settings.local.json 2>/dev/null
  env | grep ^ANTHROPIC_
  ```

  DOĞRULA `Setting sources` beklediğin dosyaları listeliyor. Şüpheli anahtar her katmanda aranmış, etkisi küçük bir denemeyle görülmüş. Tek dosyayı okuyup etkin değeri varsayma.

  Belge: Ayarlar

2 Bağlam yönetimi

Gereksiz araç çıktısı ve konu değişimleri bağlamı büyütür. Sıkıştırma ayrıntı kaybettirebilir; yeni oturum da bağlamı tekrar kurmayı gerektirir. Geçiş kararını işin sınırına ve açık kanıtlara göre ver.

- [ ] **2.1 Oturumu tutarlı bir iş etrafında tut; geçişte devir dosyası bırak.** TEMEL

  NEDEN Birbirinden kopuk işleri aynı bağlama yığmak kararları karıştırabilir; her küçük adımda yeni oturum açmak da tekrar yaratır.

  NASIL Oturum sonunda Claude'a proje kökünde bir devir dosyası yazdır, yeni oturum oradan başlasın.

  ```
  Oturumu kapatıyoruz. BEKLEYENLER.md'ye yaz:
  1. Sıradaki adım (komut, dosya, beklenen sonuç)
  2. Açık sorular
  3. Alınan kararlar ve gerekçeleri, denenip olmayanlar
  4. Bitenler (tarihli, kısa)
  ```

  DOĞRULA Yeni oturum açık işi, ilgili dosyayı ve son doğrulama sonucunu devirden bulabiliyor.

  Belge: İyi uygulamalar

- [ ] **2.2 Bağlamı tahminle değil sayıyla izle.** TEMEL

  NEDEN Doluluğu hissedemezsin. Sayı görünce devri zamanında yaparsın.

  NASIL `/context` ile anlık dağılım. `/statusline` ile alt satıra doluluk yüzdesi.

  ```
  /statusline model adını ve context_window.used_percentage değerini göster
  ```

  DOĞRULA Durum satırında yüzde görünüyor.

  Belge: Durum satırı

- [ ] **2.3 Konu değişince `/clear` de.** TEMEL

  NEDEN Eski konunun ayrıntısı yeni işte gürültüdür, yanlış varsayıma da yol açar.

  NASIL Alakasız yeni iş için yeni oturum ya da `/clear`.

  DOĞRULA Bir oturumda iki farklı projeye ait iş kalmamalı.

  Belge: Komutlar

- [ ] **2.4 Çok dosya okuyan araştırmayı alt ajana ver.** İLERİ

  NEDEN Ham metin alt ajanın bağlamında kalır, ana oturuma yalnız sonuç döner.

  NASIL Dönüş biçimini baştan söyle.

  ```
  Bir alt ajanla src/ altındaki bütün ödeme akışını tara.
  Bana yalnız şunları dön: giriş noktaları (dosya:satır),
  harici servis çağrıları, hata yönetimi olmayan yerler.
  ```

  DOĞRULA Ana oturumda dosya dökümü yerine kısa rapor görürsün.

  Belge: Alt ajanlar

- [ ] **2.5 Büyük araç çıktısını dosyaya yazdır, bağlama yalnız özetini al.** İLERİ

  NEDEN Büyük araç çıktısı gerekli kanıtı görünmez kılabilir.

  NASIL Tam çıktıyı göreve özel loga yaz; özeti ve komutun gerçek çıkış kodunu birlikte raporla.

  ```
  mkdir -p work
  npm test > work/test.log 2>&1
  sonuc=$?
  tail -n 30 work/test.log
  exit "$sonuc"
  ```

  DOĞRULA Sonda çalışan tail komutunun başarısı testin başarısı sanılmıyor.

  Belge: Headless kullanım

2.6 `/compact` yaparken neyin korunacağını söyle. İLERİ

NEDEN Talimatsız sıkıştırma neyin önemli olduğunu tahmin eder.

NASIL Komutun arkasına korunacakları yaz.

```
/compact veri modeli kararlarını, açık hataları ve değişen dosya yollarını koru
```

DOĞRULA Sıkıştırmadan sonra kritik bir kararı sor, doğru gelmeli.

Belge: Komutlar

2.7 Devam, dallandırma ve temiz başlangıcı ayır. İLERİ

NEDEN Kaldığın yerden sürmek, aynı noktadan başka bir yol denemek ve sıfırdan başlamak farklı ihtiyaçlardır. Yanlışını seçmek ya bağlamı kirletir ya da işi yeniden kurdurur.

NASIL Aynı işe dönmek için `claude --continue` ya da `/resume`. Başka bir yaklaşımı orijinali bozmadan denemek için `/branch` ya da `claude --resume` ile `--fork-session`. `/fork` farklıdır: konuşmayı arka planda koşan yeni bir oturuma kopyalar, sen burada sürersin. Kodu ve konuşmayı geri sarmak için `/rewind` (boş girdide Esc Esc). İlgisiz iş için `/clear`.

```
# Aynı iş, yeni gün
claude --continue
# Aynı noktadan ikinci yaklaşım, orijinal korunur
claude --resume odeme-refactor --fork-session
# Oturum içinde: /branch alternatif-cozum
```

DOĞRULA Dalı açınca orijinal oturum `/resume` listesinde duruyor. Rewind sonrası yalnız Bash ile değişmiş dosyalar Git ile ayrıca kontrol edilmiş.

Belge: Oturumlar · Checkpoint

3 Görev verme

Bir isteğin kalitesi, en çok neyin "bitti" sayılacağının açıklığına bağlı. Bitiş tanımı yoksa model ya erken durur ya da senden "devam" bekler.

3.1 Bitiş ölçütünü baştan yaz. TEMEL

NEDEN Ölçüt yoksa iş parça parça ilerler ve her parçada seni bekler.

NASIL Bitiş koşulunu ve durma sınırını yaz. `/goal` değerlendirmesi konuşmadaki kanıta bakar; bağımsız dosya veya test denetimi yapmaz. Metne yazılan tur sınırı da model tarafından yorumlanır, süreç düzeyinde katı bütçe değildir.

```
Ödeme doğrulamasını uygula. Kabul: geçerli kart kabul, bozuk kart ret,
servis hatasında tekrar deneme; ilgili testlerin sayısını ve çıktısını göster.
/goal bu kabul senaryoları ve ilgili testler geçiyor; ya da 20 turda eksikleri
raporlayıp dur
```

DOĞRULA İstenen davranış ve test çıktısı görünür; sınırda duran iş tamamlandı diye raporlanmıyor.

Belge: Hedefler

3.2 Kabul ölçütünü doğrulanabilir yaz. TEMEL

NEDEN "Çalışsın" denetlenemez, "şu test 12/12 geçsin" denetlenir.

NASIL Sayı, ekran ya da komut çıktısıyla tarif et.

```
Kötü: Arama düzgün çalışsın.
İyi:  "ist" yazınca 300 ms içinde İstanbul ilk sırada gelir;
      boş sorguda liste temizlenir; search.test.ts 8/8 geçer.
```

DOĞRULA Ölçütü başka biri okuyup evet/hayır diyebiliyor mu?

İş akışı önerisi

- [ ] **3.3 Büyük ve belirsiz işte önce plan iste, küçük işte doğrudan yaptır.** TEMEL

  NEDEN Plan, yanlış yönü kod yazılmadan yakalar. Küçük işte plan zaman kaybıdır.

  NASIL Plan modu: CLI'da Shift+Tab ile modlar arasında geç ya da tek bir isteğin başına `/plan` yaz. Uygulamada mod seçiciyi kullan.

  DOĞRULA Plan aşamasında hedef uygulama dosyaları değişmemiş; izin isteyen bir araç plan modunda diye otomatik güvenli sayılmamış.

  Belge: Komutlar

- [ ] **3.4 Kısıtları ve yapılmayacakları açıkça söyle.** TEMEL

  NEDEN Model boşluğu en olası çözümle doldurur. O çözüm çoğu zaman senin istemediğindir.

  NASIL İsteğe ayrı bir "Kısıtlar" bloğu ekle.

  ```
  Kısıtlar:
  - Yeni bağımlılık ekleme.
  - Sunucu tarafı yok, veri cihazda kalır.
  - Mevcut renk ve boşluk değerlerini kullan.
  ```

  DOĞRULA Çıktıda kısıtı ihlal eden bir öğe var mı?

  İş akışı önerisi

- [ ] **3.5 İstediğin çıktının bir örneğini, mümkünse bir de karşı örneğini ver.** İLERİ

  NEDEN Bir örnek, bir paragraf tarifin anlatamadığı tonu ve biçimi taşır.

  NASIL "Şuna benzesin / şuna benzemesin" diye iki kısa örnek.

  ```
  Bildirim metni şuna benzesin: "Bugün 3 dakikan var mı? Bir sayfa kaldı."
  Şuna benzemesin: "Hedeflerinize ulaşmak için şimdi harekete geçin!"
  ```

  DOĞRULA İlk çıktı örneğe yakın mı?

  İş akışı önerisi

- [ ] **3.6 Claude'dan soru sorarken önerdiği cevabı da yazmasını iste.** İLERİ

  NEDEN Boş soru işi sana geri yükler. Önerili soruya "önerinle devam" demek yeter.

  NASIL Global CLAUDE.md'ye tek kural.

  ```
  - Sorduğun her soruya önerdiğin cevabı ve gerekçesini ekle.
  - Seçenekleri harflendir, tek mesajda tek karar iste.
  ```

  DOĞRULA Gelen sorulara tek kelimeyle cevap verebiliyor musun?

  İş akışı önerisi

- [ ] **3.7 Uzun bir brief'i yapıştırmadan önce eksiklerini buldur.** İLERİ

  NEDEN Brief'teki boşluk sonradan "beğenmedim" ve yeniden iş olarak döner.

  NASIL Kararı sonucu değiştiren boşlukları ve çelişkileri ayır; geri alınabilir uygulama ayrıntılarını öneriyle tamamla.

  ```
  Bu brief'te sonucu değiştiren belirsizlikleri ve çelişkileri listele.
  Her biri için öneri, gerekçe ve karar sahibini yaz.
  Geri alınabilir teknik ayrıntıları varsayım olarak belirtip ilerle.
  ```

  DOĞRULA Ürün davranışı veya dış taahhüt değiştiren kararlar çözülmüş; bağımsız işler gereksiz beklemiyor.

  İş akışı önerisi

4 Prompt yazımı: önce ve sonra

İyi bir istek uzun olmak zorunda değil. Dört şey taşıması yeter: amaç, "iyi"nin tanımı, bağlam ve çıktının biçimi. Aşağıdaki örnekler aynı isteğin zayıf ve güçlü hâlleri.

- [ ] **4.1 "En iyi" ya da "kaliteli" diyorsan ölçütünü yaz.** TEMEL

NEDEN Tanımsız sıfatı model kendi ortalamasıyla doldurur.

NASIL Sıfatın yerine iki-üç gözlemlenebilir özellik koy.

```
Önce: Bana en iyi onboarding ekranını tasarla.
Sonra: İlk açılışta 30 saniyede bitecek, en fazla 3 ekranlık,
       izin isteğini ilk değer görüldükten sonra soran bir onboarding.
       Üç farklı yön çiz, her biri tam ekran.
```

DOĞRULA Ölçütü okuyan biri çıktıyı puanlayabilir mi?

İş akışı önerisi

- [ ] **4.2 Okuru ve kullanım yerini söyle.** TEMEL

NEDEN Aynı içerik yöneticiye, geliştiriciye ve müşteriye farklı yazılır.

NASIL "Kim okuyacak, nerede, ne karar verecek" cümlesi.

```
Önce: Bu sürüm için not yaz.
Sonra: Bu sürüm notunu App Store'daki "Yenilikler" alanı için yaz.
       Okur teknik olmayan kullanıcı. En fazla 4 madde, her biri
       kullanıcının fark edeceği bir değişiklik.
```

DOĞRULA Çıktının tonu ve ayrıntı düzeyi okura uyuyor mu?

İş akışı önerisi

- [ ] **4.3 Uzunluk istiyorsan dolguyu da yasakla.** İLERİ

NEDEN "En az 10 sayfa" tek başına doldurma davetidir.

NASIL Uzunluk hedefinin yanına "madde uydurma" kısıtı.

```
Yeterli açıklama ve somut örnek ver. Hedef uzunluk yaklaşık 10 sayfa;
kaynak veya gerekçe yetmiyorsa kısa tut ve boşluğu açıkla.
Sayfa hedefi için tekrar, uydurma veri veya gereksiz bölüm ekleme.
```

DOĞRULA Silinse kimsenin aramayacağı bir paragraf var mı?

İş akışı önerisi

- [ ] **4.4 Çıktının biçimini şablonla ver.** İLERİ

NEDEN Biçim belirsizse her bölüm farklı yapıda gelir, karşılaştırılamaz.

NASIL Her öğe için alan listesi.

```
Her madde şu biçimde:
- tek cümlelik eylem (fiille başlar)
- neden önemli (bir cümle)
- nasıl yapılır (komut, ayar ya da dosya)
- nasıl doğrularım
```

DOĞRULA Bütün öğeler aynı alanları taşıyor mu?

İş akışı önerisi

5 Model, effort ve maliyet

Her işe en güçlü modeli ve en yüksek düşünme bütçesini vermek limiti erken bitirir. Doğru eşleştirme, aynı limitle daha çok iş demek.

5.1 Alt ajan modelini işin zorluğu ve hata maliyetiyle seç. TEMEL

NEDEN Daha küçük model her doğrulama işinde yeterli olmayabilir. Aynı temsilî işte kalite, süre ve kullanımı karşılaştır.

NASIL Ajan tanımında `model` seç; genel varsayılan için aşağıdaki ayar kullanılabilir. Çağrı ve ajan tanımı bu varsayılanı geçersiz kılabilir; Explore/Plan da aynı varsayımı izlemez.

```
{
  "env": { "CLAUDE_CODE_SUBAGENT_MODEL": "sonnet" }
}
```

DOĞRULA Çalışan ajanın gerçek modelini `/tasks` ile gör; sonuç kalitesini ayrı değerlendir.

Belge: Alt ajanlar

5.2 Model takma adı ile sürüme sabitleme arasında bilinçli seçim yap. TEMEL

NEDEN Takma ad güncellemeleri izler; tam model kimliği tekrar üretilebilirlik sağlar. Takma adın hedefi sağlayıcıya göre de değişir.

NASIL Gündelik işte hesabının desteklediği takma adı, karşılaştırmalı ölçümde tam kimliği kaydet.

```
{ "model": "opus" }
```

DOĞRULA `/model` ve oturum bilgisi beklediğin modeli gösteriyor; güncel takma ad en ucuz veya en iyi sonuç garantisi sayılmıyor.

Belge: Model ve effort

5.3 Düşünme bütçesini işe göre ayarla. TEMEL

NEDEN Mimari karar ile dosya yeniden adlandırma aynı bütçeyi istemez. "Think hard" gibi ifadeler sıradan metin olarak geçer, hiçbir ayarı değiştirmez. Yalnız `ultrathink` o tur için ek bir talimat ekler, effort ayarı yine aynı kalır.

NASIL Oturumda `/effort`, başlatırken `--effort` kullan. `max` oturum içindir; kalıcı `effortLevel` için desteklenen seviyeyi seç. Seçenekler modele ve sürüme bağlıdır.

```
/effort high
/effort status
claude --effort max
```

DOĞRULA `/effort status` ile seçimi gör; aynı kabul senaryosunda kalite ve süreyi karşılaştır.

Belge: Model ve effort

5.4 Kullanımı haftada bir kez sayıyla gör. İLERİ

NEDEN Limit bittiğinde değil, bitmeden önce neyin harcadığını bilmek gerekir.

NASIL `/usage` içinde oturum tüketimini ve varsa plan limitlerini ayrı oku. Tahmini dolar tutarı abonelik faturası değildir; API faturası için Console kayıtlarına bak.

DOĞRULA Notta dönem, cihaz kapsamı ve API/abonelik ayrımı var; yaklaşık yerel analiz tüm hesap kullanımı sayılmıyor.

Belge: Kullanım ve maliyet

- [ ] **5.5 Advisor'u zor kararlar için kullan.** İLERİ

  NEDEN İkinci modelin danışması da kullanım tüketir; API faturalaması ve abonelik limiti ayrı değerlendirilir.

  NASIL `/advisor opus` ile aç, `/advisor off` ile kapat. Kullanılabilirlik model, sağlayıcı ve özellik erişimine bağlıdır; Anthropic API altyapısı kullanır, bu yalnız API anahtarı kullanıcıları demek değildir.

  DOĞRULA `/advisor` ile bağlı modeli kontrol et.

  Belge: Advisor

- [ ] **5.6 Model değişikliğini aynı görevle karşılaştır.** İLERİ

  NEDEN Genel izlenim ve tek bir başarılı cevap model seçimini kanıtlamaz. Hata maliyeti, süre ve kullanım ancak aynı işte yan yana görünür.

  NASIL Kendi işinden temsilî iki üç görev seç. Her birini iki modelle aynı prompt ve aynı başlangıç durumuyla headless koştur. JSON çıktısındaki sonucu, tur sayısını ve maliyeti kaydet, sonucu önceden belirlediğin ölçütle değerlendir. Belgede hazır bir karşılaştırma tarifi yok, bu parçalarla kurulur.

  ```
  for m in sonnet opus; do
    claude -p "$(cat gorev.md)" --model $m --output-format json \
      --no-session-persistence > sonuc-$m.json
  done
  ```

  DOĞRULA Karar tablosunda her görev için iki modelin ölçüt sonucu, `num_turns` ve `total_cost_usd` var. Tek koşumluk farkı kesin sonuç sayma.

  Belge: Model ve effort · Headless kullanım

6 İzinler

İzin listesi tekrarlanan yetkili işlemleri kolaylaştırır. Etkin mod, yönetilen politika, hook ve sandbox da sonucu etkiler. Bir allow kuralı komutun her girdide güvenli olduğunu kanıtlamaz.

- [ ] **6.1 Her aktif projede `.claude/settings.json` izin listesi tut.** TEMEL

  NEDEN Projeye ait dar liste tekrarlanan izinleri azaltabilir; test betikleri de kod çalıştırdığı için önce incelenmelidir.

  NASIL Yalnız incelenmiş test ve derleme komutlarını ekle; yazma veya yayın komutunu işin mevcut yetkisine göre ayrı değerlendir.

  ```
  {
    "permissions": {
      "allow": ["Bash(npm test)", "Bash(npm run build)"]
    }
  }
  ```

  DOĞRULA İzin isteme davranışını zararsız bir denemeyle ve etkin modla kontrol et; bekleyen adım varsa nedeni kayıtlı.

  Belge: İzinler

6.2 Yıkıcı komutları deny listesine koy. TEMEL

NEDEN Normal izin değerlendirmesinde deny kuralları allow'dan önce gelir. Komut kalıpları tüm yıkıcı eşdeğerleri yakalamaz; mod/eklenti ve OS sınırlarını da incele.

NASIL Global ya da proje ayarında `deny` dizisi.

```
"deny": [
  "Bash(rm -rf:*)",
  "Bash(git push --force:*)",
  "Bash(git reset --hard:*)"
]
```

DOĞRULA Gerçek veride yıkıcı komut çalıştırma. Geçici, verisiz bir test ortamında engellenen ve izinli karşı örneği kontrol et.

Belge: İzinler

6.3 Allow girdisini dar tut. İLERİ

NEDEN Geniş bir joker, istemediğin alt komuta da izin verir.

NASIL Araç adı yerine alt komut düzeyinde yaz.

```
Geniş: "Bash(git:*)"           # push, reset, clean da dahil
Dar:   "Bash(git status:*)", "Bash(git diff:*)", "Bash(git commit:*)"
```

DOĞRULA Her joker için kapsadığı komutları ve seçenekleri gözden geçir; izinli betiğin içeriği değişince yeniden incele.

Belge: İzinler

6.4 Komutu izin motorunun eşleştirebileceği biçimde yaz. İLERİ

NEDEN Kabuk genişletmeleri ve birleşik komutlar ek izin değerlendirmesine girebilir; doğrudan yorumlayıcı yolu davranışı sadeleştirir.

NASIL venv'i aktive etme, yorumlayıcıyı doğrudan çağır.

```
Karmaşık: source .venv/bin/activate && python -m pytest
Daha kolay denetlenir: .venv/bin/python -m pytest
```

DOĞRULA Allow'da olan bir komut yine soruyorsa içinde genişletme ara.

Belge: İzinler

6.5 Tekrarlanan analizleri betiğe çevir. İLERİ

NEDEN Sürüm kontrollü betik, tekrar eden anlık koddan daha kolay incelenir. Betiği izin listesine almak içindeki her işlemi güvenli yapmaz.

NASIL Analizi projede `scripts/` altına koy, betiği izin listesine ekle.

DOĞRULA Betik değişiklikleri incelenmiş; çalıştırılan dosya ve sürüm raporda belli.

Belge: İzinler

6.6 Geçmişten izin listesi önerisi al. İLERİ

NEDEN Hangi komutun sık sorulduğunu tahmin etmek yerine kayıttan çıkar.

NASIL `/fewer-permission-prompts`.

DOĞRULA Önerilen her girdiyi tek tek gözden geçir, yıkıcı olanı alma.

Belge: İzinler

- [ ] **6.7 Etkin izin modunu başlangıçta kontrol et.** İLERİ

  NEDEN Plan, manuel, otomatik onay ve bypass farklı davranır.

  NASIL `/permissions` ve oturumun mod göstergesini oku; geniş bypass yerine görevine uygun izin/sandbox bileşimini seç.

  DOĞRULA Zararsız örnekte beklenen izin davranışı görülmüş; mod adı tek başına güvenlik kanıtı değil.

  Belge: İzinler

7 Güvenlik ve gizlilik

Araç izinleri, dış içerik ve veri paylaşımı farklı riskler taşır. Talimat, izin kuralı ve sandbox farklı katmanlardır; biri diğerinin yerine geçmez.

- [ ] **7.1 Gizli dosyaları okuma dışı bırak.** TEMEL

  NEDEN Gizli verinin bağlama veya çıktıya girmesini azaltmak gerekir; tek bir araç kuralı tam yalıtım sağlamaz.

  NASIL Read deny kuralları dosya araçlarını ve tanınan bazı kabuk erişimlerini kapsar; keyfî Python/Node alt süreçlerini bütünüyle engellemez. Güçlü ayrım için sırları ortamdan çıkar veya sandbox dosya erişim sınırlarını uygula.

  ```
  "deny": [
    "Read(./.env)",
    "Read(./.env.*)",
    "Read(./**/*.pem)",
    "Read(./secrets/**)"
  ]
  ```

  DOĞRULA Gerçek sır yerine sahte işaret içeren dosyayla dosya aracı ve alt süreç sınırlarını ayrı test et.

  Belge: İzinler

- [ ] **7.2 Kurumsal veriyi kişisel hesaba ve depoya taşıma.** TEMEL

  NEDEN Şirket içi adresler, müşteri verisi ve iç doküman, özel bir depoda bile şirketin dışına çıkmış olur.

  NASIL Şirket işini şirketin onayladığı hesapla yap. Paylaşmadan önce iç alan adı ve iç kod deposu adresleri için tarama yaptır.

  ```
  Bu klasörü paylaşmadan önce tara: şirket alan adları, iç git adresleri,
  IP adresleri, e-posta adresleri, token'a benzeyen diziler.
  Bulduklarını dosya:satır olarak listele, hiçbirini silme.
  ```

  DOĞRULA Push öncesi tarama boş dönüyor.

  Belge: Güvenlik

- [ ] **7.3 Web sayfası, e-posta ve belgedeki talimata uyma refleksini kapat.** TEMEL

  NEDEN Okunan içerikteki "şunu yap" cümlesi senin talimatın değildir. Prompt injection böyle çalışır.

  NASIL Dış içerikle çalışırken izin modunu sıkı tut, yan etkili adımları (gönder, sil, öde) onaya bağla.

  DOĞRULA Dış metindeki talimat yetki sayılmıyor; gönderme, silme veya ödeme yalnız kullanıcının geçerli yetkisi kapsamındaysa uygulanıyor.

  Belge: Güvenlik

7.4 Ağ erişimini içerik güveniyle karıştırma. İLERİ

NEDEN Bir alan adına veya curl komutuna izin vermek indirilen içeriği güvenilir talimata dönüştürmez.

NASIL Gereken adres ve araçla sınırla, indirilen betiği çalıştırmadan incele. WebFetch, Bash ve MCP erişimlerini ayrı değerlendir.

DOĞRULA Dış içerikteki talimat uygulama yetkisi olarak kabul edilmiyor; erişim kapsamı kayıtlı.

Belge: Güvenlik

7.5 Güvenmediğin kodda sandbox'ı aç. İLERİ

NEDEN İndirilen bir repodaki betik dosya sistemine ve ağa serbestçe erişmesin.

NASIL `/sandbox` ile destek ve etkin sınırları incele. Varsayılan okuma erişimi geniş olabilir; gerekli denyRead sınırlarını, ağ alanlarını ve sandbox dışı istisnaları kontrol et. Yerleşik Read/WebFetch araçları ayrıca izin kurallarına tabidir.

```
{ "sandbox": { "enabled": true } }
```

DOĞRULA İzinli ve yasaklı geçici dosya yollarında zararsız deneme yap; yalnız enabled=true görmeyi yalıtım kanıtı sayma.

Belge: Sandbox

7.6 Tehlikeli komutu hook'la içeriğinden yakala. İLERİ

NEDEN Hook politika kontrolünü araç çağrısından önce çalıştırabilir; basit metin araması tüm komut eşdeğerlerini yakalamaz.

NASIL PreToolUse engellemesi için çıkış kodu 2 kullan. Girdiyi doğrula, hatada engelleme davranışını ve timeout'u test et. Aşağıdaki örnek yalnız öğretici bir kalıp kontrolüdür.

```
#!/bin/sh
# Öğretici örnek; kapsamlı bir komut güvenlik motoru değildir.
komut=$(jq -er ' .tool_input.command | select(type == "string") ' ) || {
  echo "Geçersiz hook girdisi" >&2; exit 2;
}
case "$komut" in
  *"DROP TABLE"*|*"--force"*) echo "Politika denetimi engelledi" >&2; exit 2;;
esac
exit 0
```

DOĞRULA Sahte girdide izinli, engelli ve bozuk JSON vakaları test edilmiş; gerçek komutu çalıştırmadan sonuç görülüyor.

Belge: Hook referansı

7.7 Yeni depodaki hook ve MCP yapılandırmasını güvenmeden önce incele. İLERİ

NEDEN Proje ayarları yerel komut veya dış araç başlatabilir.

NASIL Settings, hook betikleri ve MCP sunucu komutlarını oku; paket kaynağını ve istenen yetkileri kontrol et.

DOĞRULA Bilinmeyen komut otomatik çalıştırılmamış; güven kararı incelenen sürüme ait.

Belge: Ayarlar

8 Hook'lar

Hook belirli olaylarda çalışan bir mekanizmadır; güvenilirliği betiğin kapsamı, izinleri, hata davranışı ve testleriyle sınırlıdır.

- [ ] **8.1 Her oturuma girmesi gereken bilgiyi SessionStart hook'una koy.** İLERİ

  NEDEN Talimatı model unutabilir. Hook her oturumda çalışır.

  NASIL Betiğin stdout'u oturumun bağlamına girer.

  ```
  "hooks": {
    "SessionStart": [{ "hooks": [{
      "type": "command",
      "command": "head -80 BEKLEYENLER.md 2>/dev/null || true"
    }]}]
  }
  ```

  DOĞRULA Okuma araçları kapalı bir `claude -p` ile içerikten soru sor.

  Belge: Hook referansı

- [ ] **8.2 Biçimlendirmeyi düzenleme sonrası hook'a bağla.** İLERİ

  NEDEN Biçimi her seferinde istemek yerine her düzenlemeden sonra kendiliğinden düzelir.

  NASIL PostToolUse içinde yalnız değişen, desteklenen dosyayı biçimlendiren bir betik çağır. Betik JSON girdisindeki yolu doğrulasın ve proje dışına çıkmasın; tüm depoyu her düzenlemede taramasın.

  ```
  "PostToolUse": [{
    "matcher": "Edit|Write",
    "hooks": [{ "type": "command",
      "command": "python3 .claude/hooks/format_changed.py" }]
  }]
  # format_changed.py projene göre yazılmalı ve test edilmelidir.
  ```

  DOĞRULA Bozuk girintili bir dosya düzenlet, sonuç biçimli gelmeli.

  Belge: Hook referansı

- [ ] **8.3 Bağlam göstergesini belgelenmiş statusline alanından üret.** İLERİ

  NEDEN Transcript iç yapısını token sayacı gibi ayrıştırmak sürüm değişiminde bozulabilir.

  NASIL Statusline JSON girdisindeki `context_window.used_percentage` alanını kullan; ilk yanıt öncesi eksik/null değerini işle.

  DOĞRULA Sıfır, null ve yüksek doluluk örneklerinde gösterge doğru; bilinmeyen değer yüzde sıfır diye sunulmuyor.

  Belge: Durum satırı

- [ ] **8.4 Hook'u test et ve bozarak testin düştüğünü gör.** İLERİ

  NEDEN Hiçbir şeye bakmayan hook da sessizce "çalışır".

  NASIL Hook'a örnek bir JSON girdisi verip çıktısını kontrol eden küçük bir test.

  ```
  printf '%s\n' '{"cwd":"/tmp/ornek-proje"}' | sh .claude/hooks/oturum-basi.sh
  # Test: izinli, engelli ve bozuk JSON girdilerini ayrı doğrula.
  ```

  DOĞRULA Hook'u bilerek boz, test düşmeli.

  Belge: Hook referansı

- [ ] **8.5 Hook hata politikasını görevine göre seç.** İLERİ

  NEDEN Bilgi gösteren hook ile güvenlik kapısı aynı hatada-devam kuralını kullanmamalıdır.

  NASIL Bilgilendirme hatasını görünür kaydet; politika hook'unda hatayı sessizce başarıya çevirme. `onFailure: "block"` komut/HTTP hook'ları için v2.1.295+ gerektirir; eski sürümde yokmuş gibi davran.

  DOĞRULA Betiğin bulunamaması, bozuk JSON, hata çıkışı ve timeout ayrı denenmiş; engel gerektiğinde çağrı ilerlemiyor.

  Belge: Hook referansı

- [ ] **8.6 Engellemesi gereken hook'u arka plana atma.** İLERİ

  NEDEN Arka plan hook'u Claude'u bekletmez ama onu durduramaz da. Güvenlik kontrolünü async yapmak kontrolü sessizce kaldırmaktır.

  NASIL `async: true` yalnız bildirim, log, yavaş biçimlendirme gibi engellemesi gerekmeyen işlerde. Async hook'un `decision` ve `permissionDecision` alanları etkisizdir. `asyncRewake` exit 2 ile Claude'u uyarır, engellemez. Headless koşum bitince hâlâ koşan async hook iptal edilir.

  DOĞRULA Yasak bir komutu dene: senkron hook onu reddediyor, aynı hook async iken reddetmiyor. Bu farkı gördükten sonra güvenlik hook'unu senkron bırak.

  Belge: Hook referansı

- [ ] **8.7 Birden çok hook arasında çalışma sırası varsayma.** İLERİ

  NEDEN Aynı olaya eşleşen hook'lar paralel çalışır. Birinin diğerinin çıktısına dayandığı düzen bazen tutar, bazen tutmaz.

  NASIL Sıraya bağlı adımları tek bir betikte birleştir. Paralel hook'lar aynı dosyaya yazıyorsa çakışmayı göze al ya da ayrı dosyalara yazdır.

  DOĞRULA Bağımlı iki adım tek betikte. Hook'ları farklı sırada tanımlayınca sonuç değişmiyor.

  Belge: Hook referansı

9 Skill, komut, ajan, MCP

Aynı tarifi üçüncü kez yazıyorsan, o tarif bir dosyaya aittir. Skill gerektiğinde yüklenir, ajan kendi bağlamında çalışır, MCP dış sisteme API ile bağlanır.

- [ ] **9.1 Eleştiriyi yazandan ayrı bir bağlama ver.** TEMEL

  NEDEN Ayrı bağlam farklı varsayımlarla inceleme yapabilir; bağımsızlık kalite garantisi değildir.

  NASIL Statik inceleme için yazma ve Bash aracı olmayan doğrulayıcı ajan kullan. Test çalıştırmak gerekiyorsa ayrı, sınırlı ortam ve izin ver; Bash olan ajan salt okunur sayılmaz.

  ```
  ---
  name: dogrulayici
  description: Tamamlanan değişiklikte kanıtlı doğruluk hatalarını statik olarak
  inceler.
  tools: Read, Grep, Glob
  model: sonnet
  ---
  İddiaları diff ve kaynakla karşılaştır. Bulgu uydurma.
  Her bulguda dosya, tetikleyici ve etkisini yaz. Test çalıştırmadıysan belirt.
  ```

  DOĞRULA Bulgu varsa dosya ve kanıtı var; bulgu yoksa kapsam ve çalıştırılmayan kontroller açık.

  Belge: Alt ajanlar

9.2 Üçüncü kez anlattığın işi skill'e çevir. İLERİ

NEDEN Tarif her seferinde biraz farklı anlatılır, sonuç da farklı çıkar.

NASIL `~/.claude/skills/<ad>/SKILL.md` .

```
---
name: surum-notu
description: Son etiketten bu yana değişikliklerden kullanıcıya dönük sürüm notu
yazar. "Sürüm notu", "release notes" denince kullan. Teknik changelog için kullanma.
---
1. git log ile son etiketten bu yanaki commit'leri al.
2. Kullanıcının fark edeceği değişiklikleri seç, en fazla 5.
3. Her maddeyi kullanıcının diliyle bir cümlede yaz.
```

DOĞRULA Açık /surum-notu çağrısı ve doğal dille eşleşen istek ayrı denenmiş; otomatik seçilmeme de kaydedilmiş.

Belge: Skills

9.3 Skill açıklamasına "kullanma" sınırı koy. İLERİ

NEDEN Model skill'i açıklamadan seçer. Sınır yoksa komşu işlerde de yüklenir.

NASIL Açıklamaya hem tetikleyici ifadeleri hem de kullanılmayacağı durumu yaz (yukarıdaki örnekteki son cümle).

DOĞRULA Benzer ama farklı bir istekte skill yüklenmemeli.

Belge: Skills

9.4 Yan etkili skill'i kendiliğinden çalışmaya kapat. İLERİ

NEDEN Deploy ya da mesaj gönderen bir skill yanlış eşleşmede çalışmasın.

NASIL Frontmatter'da `disable-model-invocation: true` otomatik çağrıyı kapatır; araç izinleri veya kullanıcı yetkisi yerine geçmez. Etkin skill override ayarını da kontrol et.

```
disable-model-invocation: true
```

DOĞRULA İlgili cümleyi yaz, skill kendiliğinden çalışmamalı.

Belge: Skills

9.5 Bir uygulamaya bağlanırken önce o uygulamanın MCP'sini dene. İLERİ

NEDEN API tabanlı araç hızlı ve kesin. Ekranda tıklayarak sürmek yavaş ve kırılgan.

NASIL Sıra: uygulamanın MCP'si, sonra tarayıcı, en son bilgisayar kontrolü.

DOĞRULA Aynı iş için kaç araç çağrısı gittiğine bak.

Belge: MCP

9.6 Kendi setini plugin olarak paketle. İLERİ

NEDEN Başka makineye ya da ekip arkadaşına tek komutla taşınır.

NASIL `.claude-plugin/plugin.json` ve skill dizinleri.

DOĞRULA Temiz bir makinede kurulumdan sonra skill listede görünmeli.

Belge: Plugins

9.7 MCP bağlantısına yalnız gereken kapsamı ver. İLERİ

NEDEN Bağlantının çalışması doğru hesap ve en az yetki anlamına gelmez.

NASIL `/mcp` ile sunucu ve hesap durumunu incele; okuma/yazma yetkisini ayır, gereksiz bağlantıyı kaldır.

DOĞRULA Zararsız okuma testi doğru hedefte; yazma ayrıca kullanıcının yetki kapsamıyla eşleşiyor.

Belge: MCP

- [ ] **9.8 Skill'i olumlu ve olumsuz örneklerle değerlendir.** İLERİ

  NEDEN Tek başarılı çağrı yanlış eşleşmeleri ve eksik çıktıları yakalamaz.

  NASIL Tetiklenmesi gereken, gerekmeyen ve hata durumunu içeren küçük vaka seti tut; beklenen sonucu baştan yaz.

  DOĞRULA Gerekli çağrıda doğru çıktı, komşu istekte gereksiz çağrı yok; başarısız vaka saklanmış.

  Belge: Skills

10 Ekiple kullanım

Bir kişinin iyi düzeni, ekip için de düzen olabilir. Ama ortak dosyaya giren her şey herkesin oturumunu etkiler.

- [ ] **10.1 Ortak ayarları `.claude/settings.json`'da, kişisel olanları `settings.local.json`'da tut.** İLERİ

  NEDEN Ekibin izin listesi ve hook'ları git ile paylaşılır, kişisel tercihin başkasına geçmez.

  NASIL `settings.json` commit'lenir, `settings.local.json` git dışında kalır. Öncelik sırası yukarıdan aşağı: yönetilen ayarlar, komut satırı, `settings.local.json`, `settings.json`, kullanıcı ayarları.

  DOĞRULA Yeni ekip üyesi dosyaları ve etkin politikayı doğruluyor; yönetilen ayar veya kişisel mod farkı aynı izin garantisi sayılmıyor.

  Belge: Ayarlar

- [ ] **10.2 Proje CLAUDE.md'sini kod gibi gözden geçir.** İLERİ

  NEDEN Bir kural bütün ekibin oturumunu değiştirir.

  NASIL CLAUDE.md değişikliği de pull request'ten geçsin.

  DOĞRULA Son ay eklenen kuralların gerekçesi commit mesajında yazıyor mu?

  Belge: Talimat ve bellek

- [ ] **10.3 Ekipte tekrar eden işi ortak skill ya da plugin yap.** İLERİ

  NEDEN Herkes aynı işi kendi tarifinle yaptırınca sonuçlar tutarsız olur.

  NASIL Repoda `.claude/skills/` ya da ekip plugin'i.

  DOĞRULA İki kişinin aynı skill ile ürettiği çıktı aynı biçimde.

  Belge: Skills

11 Paralel çalışma

Aynı anda çok oturum çalıştırmak güçlü bir kaldıraç. Ama her paylaşılan kaynak bir çakışma noktası ve makine yükü her şeyi yavaşlatır.

- [ ] **11.1 Aynı dosyalara iki yazar sokma. Her yazar ayrı worktree'de.** İLERİ

  NEDEN İki oturum aynı dosyayı düzenlerse biri diğerinin işini ezer.

  NASIL Worktree ile başlat.

  ```
  claude --worktree odeme-ekrani
  ```

  DOĞRULA `git worktree list`'te her yazar için ayrı satır.

  Belge: İş akışları

- [ ] **11.2 Git index'ini ortak yazarlardan ayır ve commit kapsamını incele.** İLERİ

  NEDEN Aynı checkout index'i ortaktır; ayrı worktree index'leri ayrıdır. İki komutu && ile birleştirmek atomik kilit sağlamaz.

  NASIL Ayrı worktree kullan, yalnız kendi değişikliklerini stage et ve commit öncesi staged diff'i incele.

  ```
  git status --short
  git add -- src/odeme.ts
  git diff --cached -- src/odeme.ts
  git commit -m "Ödeme: kart doğrulama"
  # Önceden stage edilmiş başka dosya varsa önce kapsamı ayır.
  ```

  DOĞRULA Commit diff'inde başka işin değişikliği yok.

  Belge: Git worktree

- [ ] **11.3 Port ve tarayıcı oturumunu projeye özel yap.** İLERİ

  NEDEN Aynı porttaki iki sunucu birbirini düşürür, yanlış sayfa test edilir.

  NASIL Her projeye sabit ve farklı bir port.

  DOĞRULA Açılan sayfa gerçekten bu projenin sayfası.

  İş akışı önerisi

- [ ] **11.4 Ağır oturum sayısını makine yüküne göre sınırla.** İLERİ

  NEDEN Kaynak baskısı yavaşlama ve kararsız testlerle birlikte görülebilir; tek yük ölçümü nedeni kanıtlamaz.

  NASIL CPU, bellek baskısı, disk ve aktif işleri birlikte izle; bağımsız ağır işi kaynaklara göre sırala.

  DOĞRULA Bekleme süresi ve başarısızlıklar ölçülmüş; sabit bir load-average eşiği her makineye uygulanmıyor.

  İş akışı önerisi

- [ ] **11.5 Uzun gözetimsiz işi bulut oturumuna gönder.** İLERİ

  NEDEN Makineyi yormaz, uyku onu öldürmez.

  NASIL Uygulamada bulut hedefli oturum. Yerel cihaz isteyen iş yerelde kalır.

  DOĞRULA Bilgisayar kapalıyken iş ilerlemiş olmalı.

  İş akışı önerisi

- [ ] **11.6 Çok ajanlı iş akışını yalnız gerçekten paralel işe kullan.** İLERİ

  NEDEN Onlarca ajan çok token harcar. Sıralı iş paralelden kazanmaz.

  NASIL Bağımsız parçalar için, örneğin on dosyanın ayrı ayrı incelenmesi.

  DOĞRULA İş akışı sonrası kullanım farkına `/usage` 'dan bak.

  Belge: Alt ajanlar

12 Doğrulama

"Geçti" demek bir iddia, kanıt değil. En tehlikeli yeşil, hiçbir şeye bakmadığı için yeşil olandır.

- [ ] **12.1 Sonuçta durumu değil sayıyı oku.** TEMEL

  NEDEN Eşleşmeyen bir test filtresi sıfır test koşup başarılı diyebilir.

  NASIL Raporda test sayısını iste.

  ```
  Yeşil görünen ama boş:  Test run with 0 tests ... SUCCEEDED
  Gerçek:                 Executed 42 tests, with 0 failures
  ```

  DOĞRULA Her "geçti"nin yanında bir sayı.

  İş akışı önerisi

- [ ] **12.2 Yeni kontrolü hatayı geri koyarak doğrula.** TEMEL

  NEDEN Kontrol gerçekten bakıyorsa bozuk kodda düşmeli.

  NASIL İzole fixture veya geçici branch'te kusuru geri getir; testin düştüğünü ve düzeltilmiş eşinin geçtiğini göster. Kullanıcının çalışma dosyasını koru.

  ```
  Yazdığın testin gerçekten çalıştığını kanıtla: fiyat hesabındaki
  indirimi geçici olarak kaldır, testi koş, düştüğünü göster, sonra geri al.
  ```

  DOĞRULA Bozuk sürümde kırmızı.

  İş akışı önerisi

- [ ] **12.3 Alt ajanın raporunu kanıtla karşılaştır.** TEMEL

  NEDEN Ajan raporu bir iddiadır.

  NASIL Diff'e, dosyaya, çıktıya bak. Ondan sonra "bitti" de.

  DOĞRULA Rapordaki bir iddiayı rastgele seç ve doğrula.

  İş akışı önerisi

- [ ] **12.4 İş sonunda üç satır istet: kaçırılan, en zayıf iddia, gerekçesiz karar.** İLERİ

  NEDEN Övgü değil kırma denemesi, sorunun kendisini sana getirir.

  NASIL CLAUDE.md'ye tek kural ya da bir slash komutu.

  ```
  İş bitince sormadan üç satır yaz:
  1. Neyi kaçırmış olabilirim (bakmadığım dosya, koşmadığım yol)
  2. Hangi iddiam en zayıf
  3. Hangi kararım gerekçesiz
  ```

  DOĞRULA Üç satırın her biri somut bir dosya ya da karar adı içeriyor mu?

  İş akışı önerisi

- [ ] **12.5 Görsel değişikliği önce/sonra ekran görüntüsüyle gör.** İLERİ

  NEDEN "Ne değişti" sorusu metinle cevaplanmaz.

  NASIL İki ekran görüntüsü yan yana.

  DOĞRULA Farkı ilk bakışta görebiliyor musun?

  İş akışı önerisi

- [ ] **12.6 Önemli değişikliği teslimden önce kod incelemesinden geçir.** İLERİ

  NEDEN Yazan bağlam kendi hatasını kaçırır.

  NASIL `/code-review` ile hedef diff'i incele. `--fix` değişiklik yapar, `--comment` dışarı yorum gönderir; bunları amaç ve yetki olmadan ekleme.

  DOĞRULA Bulgu ve kanıtları veya kapsamı belirtilmiş "bulgu yok" sonucu var; test yapılmadıysa ayrı yazıyor.

  Belge: Komutlar

- [ ] **12.7 Teslim aşamasını kanıtıyla adlandır.** İLERİ

  NEDEN Yerel test, push ve herkese açık yayın farklı sonuçlardır.

  NASIL PASS, FAIL, NOT_RUN ve BLOCKED durumlarını ilgili komut, tarih ve sürümle kaydet; dış hedefi tekrar oku.

  DOĞRULA Build geçti diye cihaz testi, upload oldu diye canlı yayın geçti denmiyor.

  İş akışı önerisi

13 Gözetimsiz koşum

"Ben yatıyorum, sabaha bitir" dediğin iş, sabah çalışan bir sonuç bırakmalı. Gece boyu izin ekranında beklemek ya da sessizce ölmek en pahalı başarısızlık.

- [ ] **13.1 Gözetimsiz koşum için talimat setini önceden yaz.** İLERİ

  NEDEN "Sormadan ilerle" demek, hangi durumda ne yapılacağını söylemez.

  NASIL İşe özel yetki, kapsam, bütçe ve engel davranışını belirt; her proje için sınırsız global yetki yazma.

  ```
  ## Bu görevde ben yokken
  - Yetkili kapsam içindeki geri alınabilir kararları ver, varsayımı yaz.
  - Eksik yetki veya sonucu değiştiren karar varsa ilgili adımı engelli işaretle.
  - Bağımsız işleri tamamla; başarısız testi gizleme.
  - Sonuç: yapılan, yapılmayan, kanıt, kalan karar ve kaynak temizliği.
  ```

  DOĞRULA Bir koşumu dene, sabah rapor eksiksiz mi?

  İş akışı önerisi

- [ ] **13.2 Ayrılmadan önce izin, disk, yük ve uyku koşullarını doğrula.** İLERİ

  NEDEN Bu dördünden biri geceyi boşa çıkarır.

  NASIL İzinleri etkin ayardan, kaynakları işletim sistemi araçlarından kontrol et. macOS örneği aşağıda; diğer sistemlerde eşdeğerini kullan.

  ```
  df -h .
  uptime
  pmset -g
  # Claude oturumunda /permissions ile etkin izinleri incele.
  ```

  DOĞRULA Dört kontrolün sonuçları ve sınırları yazılmış; ilk 20 ayar satırını okumak bütün izin denetimi sayılmıyor.

  İş akışı önerisi

- [ ] **13.3 Uzun işi kaldığı yerden devam edecek biçimde kur.** İLERİ

  NEDEN Uyku ya da çökme işi baştan başlatmasın.

  NASIL Ara çıktıyı diske yaz, her adım öncesi "bu adım bitti mi" kontrolü.

  DOĞRULA İşi yarıda öldür, yeniden başlat, tamamlanan adımlar atlanmalı.

  İş akışı önerisi

- [ ] **13.4 Zamanlanmış oturumda uzun işi arka plana atıp turu kapatma.** İLERİ

  NEDEN Arka plan süreci oturumla birlikte ölebilir ve geriye log bile kalmayabilir.

  NASIL Turu açık tut, log'u oturumdan bağımsız bir yola yaz.

  DOĞRULA Sabah log'un son satırı işin bitişini gösteriyor mu?

  Belge: Masaüstü

- [ ] **13.5 İş bitince açık kaynakları kapat.** İLERİ

  NEDEN Açık kalan simülatör, sunucu ya da tarayıcı belleği ve diski tüketir.

  NASIL Koşum sonunda kendi açtığın süreçleri kapat, başka oturumunkine dokunma.

  DOĞRULA Sabah yalnız beklediğin süreçler açık.

  İş akışı önerisi

- [ ] **13.6 Geçici derleme ve bağımlılıkları görev sonunda hedefli temizle.** İLERİ

  NEDEN Göreve özel ürün ve cache diskte birikir; isim benzerliği sahiplik kanıtı değildir.

  NASIL Başta kesin yolları kaydet. Kullanım bitince logları ve teslim dosyalarını koruyup yalnız göreve ait yeniden üretilebilir artıkları kaldır.

  DOĞRULA Silinen yol yok; kaynak, release arşivi, sonuç ve paylaşılan cache korunmuş.

  İş akışı önerisi

14 Rutinler ve zamanlama

Tekrarlanan işi elle tetiklemek unutulur. Ama bilgisayar kapanınca ölen bir rutin de güvenilir değildir.

- [ ] **14.1 Zamanlamanın nerede çalıştığını açıkça seç.** İLERİ

  NEDEN Bulut görevleri bilgisayarından bağımsızdır; yerel Desktop işi cihaz ve uygulama koşullarına bağlıdır.

  NASIL Bulut görevleri için `/schedule` ve desteklenen claude.ai hesabını kontrol et. Araştırma önizlemesinin koşulları değişebilir; Desktop ve oturum içi döngüyü aynı mekanizma sanma.

  DOĞRULA Doğru ortamda bir deneme koşumu, sonraki çalışma zamanı ve sonuç bağlantısı görülmüş.

  Belge: Bulut zamanlama

- [ ] **14.2 Headless rutinde çıktıyı ve tur sayısını sınırla.** İLERİ

  NEDEN Sınırsız koşum takılı kalınca saatlerce harcar.

  NASIL JSON çıktı, tur sınırı, kendi log yolu.

  ```
  mkdir -p work/runs
  claude -p "$(cat gorev.md)" --output-format json --max-turns 30 \
    > work/runs/sonuc.json 2> work/runs/hata.log
  sonuc=$?
  printf '%s\n' "$sonuc" > work/runs/exit-code.txt
  exit "$sonuc"
  ```

  DOĞRULA JSON parse ediliyor, hata/başarı alanı ve süreç çıkış kodu birlikte kaydediliyor; limit dolması başarı değil.

  Belge: Headless kullanım

- [ ] **14.3 Rutinin sessiz ölümünü ayrı bir nöbetle yakala.** İLERİ

  NEDEN Hiç koşmayan rutin hata da vermez.

  NASIL Son başarılı koşumun zamanını kontrol eden ikinci küçük bir rutin.

  DOĞRULA Bir rutini bilerek durdur, nöbet uyarmalı.

  İş akışı önerisi

- [ ] **14.4 Dış servis kotasına uy ve yeniden denemeyi sınırla.** İLERİ

  NEDEN Eşzamanlı istekler kotayı tüketebilir; zamanları rastgele dağıtmak erişim engelini aşma yöntemi değildir.

  NASIL Servis koşullarına ve Retry-After yanıtına uy; sınırlı geri çekilme ve jitter uygula.

  DOĞRULA 429/503 denemesinde bekleme ve azami deneme sayısı doğru; engelde görev durumu açık.

  İş akışı önerisi

- [ ] **14.5 Oturum içi /loop ile kalıcı zamanlamayı ayır.** İLERİ

  NEDEN /loop çalışan oturumun yaşam döngüsüne bağlıdır; cihazdan bağımsız bulut görevi değildir.

  NASIL `/loop 5m` gibi sabit aralıklı döngünün görev kaydını ve durdurma biçimini kontrol et; bilgisayardan bağımsız işte bulut zamanlamasını seç.

  ```
  /loop 5m Yayının tamamlanıp tamamlanmadığını kontrol et;
  anlamlı değişiklikte bildir, tamamlanınca döngüyü iptal et.
  ```

  DOĞRULA Bir deneme tetiklenmiş, iptal sonrası yeni koşum yok; oturum kapalıyken çalışma varsayılmıyor.

  Belge: Oturum içi zamanlama

- [ ] **14.6 Headless başarısını süreç ve sonuç alanından birlikte doğrula.** İLERİ

  NEDEN Geçerli JSON üretmek işin tamamlandığını göstermez.

  NASIL Çıkış kodu, `is_error`, sonuç türü ve kabul ölçütünü değerlendir; yarım işi tekrar koşarken yan etkileri çoğaltma.

  DOĞRULA İzin reddi ve tur sınırı vakaları hata/eksik olarak görünür; başarılı eşinde kabul ölçütü sağlanır.

  Belge: Headless kullanım

- [ ] **14.7 Makineye aktarılacak sonucu şemayla tanımla.** İLERİ

  NEDEN Başka bir betiğin okuyacağı sonucu serbest metinden ayıklamak kırılgandır. Biçim bir gün değişir, betik sessizce yanlış veri alır.

  NASIL Headless koşumda `--output-format json` ile birlikte `--json-schema` ver. Sonuç `structured_output` alanında gelir. Claude geçerli çıktı üretemezse alan hiç gelmez, betik önce varlığına baksın. Başarıyı yalnız `subtype` ile ölçme, `is_error` alanına da bak.

  ```
  claude -p "Bu haftanın hata kayıtlarını özetle" --output-format json \
    --json-schema '{"type":"object","properties":{"kritik":
  {"type":"integer"}},"required":["kritik"]}' \
    > sonuc.json
  jq -e '.is_error == false and has("structured_output")' sonuc.json
  ```

  DOĞRULA Şemaya uymayan bir durumu dene, ör. boş girdi: betik alanın yokluğunu fark edip hata veriyor, boş veriyle devam etmiyor.

  Belge: Headless kullanım

15 Git ve yayın

Git, Claude ile çalışırken en güvenilir geri alma düğmesi. Sık commit, Claude'a cesur değişiklik yaptırmanın güvencesi.

- [ ] **15.1 Kaynak kodunu Git'te tut ve commit kapsamını tanımla.** TEMEL

  NEDEN Küçük, anlamlı commit geri almayı kolaylaştırır. Git dışı veriler ve sırlar için ayrıca uygun yedek gerekir.

  NASIL `git init`, sonra CLAUDE.md'ye commit kuralı.

  ```
  - Sormadan commit at, mantıklı parçalara böl.
  - Push ve deploy yalnız açıkça istendiğinde.
  ```

  DOĞRULA `git log --oneline`'da her satır tek bir değişikliği anlatıyor.

  Belge: Git worktree

- [ ] **15.2 Checkpoint'in sınırını bil.** TEMEL

  NEDEN `/rewind` yalnız dosya araçlarıyla yapılan düzenlemeleri geri alır. Bash'le silinen ya da taşınanı almaz.

  NASIL Bash'le toplu değişiklikten önce commit.

  DOĞRULA Commit öncesi diff incelenmiş; Bash, dış servis ve veritabanı etkileri için ayrı geri alma yolu var.

  Belge: Checkpoint

- [ ] **15.3 Push ve deploy yetkisini hedefe ve kapsama bağla.** İLERİ

  NEDEN Uzak depo ve yayın hedefi yerel değişiklikten farklı bir teslim aşamasıdır.

  NASIL Kullanıcının mevcut yayın talimatını koru; belirsiz hedef veya yeni dış taahhütte karar iste. Geniş, kalıcı push/deploy iznini varsayılan yapma.

  DOĞRULA Doğru hesap, repo, dal ve görünürlük doğrulanmış; yayından sonra canlı sonuç okunmuş.

  Belge: İzinler

- [ ] **15.4 Commit atfını talimatla değil ayarla yönet.** İLERİ

  NEDEN Talimat unutulabilir, ayar her commit'te uygulanır.

  NASIL `attribution` ayarı (`includeCoAuthoredBy` eskidi). Boş metin o atfı gizler, `false` hepsini gizler (v2.1.281 ve sonrası).

  ```
  { "attribution": { "commit": "", "pr": "" } }
  ```

  DOĞRULA Ayardan sonraki ilk commit'te atıf satırı beklediğin gibi.

  Belge: Ayar referansı

16 Tasarım ve arayüz

Tasarımda en pahalı döngü "beğenmedim, tekrar dene". Onu kısaltan şey araştırma, birden çok yön ve görsel karşılaştırma.

- [ ] **16.1 Tasarımdan önce gerçek örneklerden ilham topla.** İLERİ

  NEDEN İlhamsız ilk taslak şablona yakın çıkar.

  NASIL Kaynaklı bir ilham raporu iste.

  ```
  Alışkanlık uygulamalarından 8 örnek bul, ana ekranlarını karşılaştır.
  Her biri için: neyi iyi yapıyor, bizim uygulamaya ne alınabilir, kaynak.
  ```

  DOĞRULA Her öneri bir gerçek örneğe dayanıyor mu?

  İş akışı önerisi

- [ ] **16.2 İki-üç farklı yönü tam ekran çizdir.** İLERİ

  NEDEN Anlatılan tasarım kafada farklı canlanır, alt alanlar unutulur.

  NASIL Her yönü alt alanlar dahil tam ekran iste.

  DOĞRULA Seçimini görsele bakarak yapabiliyor musun?

  İş akışı önerisi

- [ ] **16.3 Beğenmediğinde aynı yönü değil, farklı bir yönü iste.** İLERİ

  NEDEN Rötuş aynı fikrin varyasyonunu üretir.

  NASIL Neyin çalışmadığını söyle ve yön değişikliği iste.

  ```
  Bu yönden çık. Kart dizisi yerine tek bir odak alanı dene,
  renk değil boşluk ve tipografiyle hiyerarşi kur.
  ```

  DOĞRULA Yeni öneri ilkinden yapısal olarak farklı mı?

  İş akışı önerisi

- [ ] **16.4 Tasarım dilini bir kez yaz, her turda yeniden anlatma.** İLERİ

  NEDEN Renk, tipografi ve boşluk kuralları her seferinde sohbetle aktarılınca kayar.

  NASIL Projede tasarım ilkeleri dosyası ya da tasarım skill'i.

  DOĞRULA Yeni oturumda tasarım dili sorulunca dosyadan cevap gelmeli.

  İş akışı önerisi

17 Mobil ve simülatör

iOS ya da Android uygulaması geliştiriyorsan simülatör en çok kullanılan araç olur. Paylaşılan cihaz, elle tıklama ve açık kalan simülatör en sık zaman kaybı.

- [ ] **17.1 Paralel testlerde cihaz sahipliğini ve UDID'yi kaydet.** İLERİ

  NEDEN Paylaşılan simülatörde başka oturumun uygulaması öne gelir, senin uygulaman bozuk sanılır.

  NASIL Önce mevcut cihaz ve runtime listesini oku; gerekli ise göreve özel cihaz oluştur, dönen UDID'yi kaydet.

  ```
  xcrun simctl list devices available
  xcrun simctl list devicetypes
  xcrun simctl list runtimes
  # create için bu listelerdeki geçerli tip ve runtime kimliğini kullan.
  ```

  DOĞRULA Testin cihazı, runtime'ı ve uygulama build'i kayıtlı; başka görevin cihazı değiştirilmemiş.

  İş akışı önerisi

- [ ] **17.2 Tekrarlanan arayüz akışını otomatik teste çevir.** İLERİ

  NEDEN Elle tıklayarak doğrulama her seferinde aynı maliyeti öder.

  NASIL Maestro, XCUITest ya da Espresso akışı, tek komutla koşan.

  ```
  appId: com.ornek.fiyat
  ---
  - launchApp
  - tapOn: "Ürün ekle"
  - inputText: "Kahve"
  - assertVisible: "Kahve"
  ```

  DOĞRULA Akış bozuk bir ekranda kırmızı dönmeli.

  İş akışı önerisi

- [ ] **17.3 Türkçe metin girişini kullanılan araçta gerçekten dene.** İLERİ

  NEDEN Bazı otomasyon giriş yöntemleri Unicode veya klavye davranışını eksik taklit edebilir.

  NASIL Desteklenen giriş yolunu seç; İstanbul, ışık ve çığ gibi metinleri ekranda ve kaydedilen veride karşılaştır. ASCII yardımcı fixture gerçek dil testinin yerine geçmesin.

  DOĞRULA Girilen, gösterilen ve kaydedilen metin eşit; denenemeyen giriş yolu NOT_RUN.

  İş akışı önerisi

- [ ] **17.4 Farklı ekran boyutlarında kontrol et.** İLERİ

  NEDEN Küçük ekranda taşan metin büyük ekranda görünmez.

  NASIL En küçük ve en büyük desteklenen cihazda ekran görüntüsü.

  DOĞRULA İki görüntüde de taşma yok.

  İş akışı önerisi

- [ ] **17.5 Göreve ait geçici simülatörü kanıtları koruyup sil.** İLERİ

  NEDEN Shutdown cihazın veri alanını kaldırmaz; erase de cihaz kaydını silmez.

  NASIL Önce log ve ekran görüntülerini dışarı al, aktif kullanıcı olmadığını doğrula. Yalnız bu görevde oluşturduğun kesin UDID'yi kapat ve sil.

  ```
  xcrun simctl shutdown <GOREVE_AIT_UDID>
  xcrun simctl delete <GOREVE_AIT_UDID>
  xcrun simctl list devices --json
  ```

  DOĞRULA UDID cihaz listesinden kalkmış; önceden var olan veya sahibi belirsiz cihaz korunmuş.

  İş akışı önerisi

18 Bilgi birikimi

Öğrenilen ders yazılmazsa bir sonraki oturumda yeniden öğrenilir. Yanlış yere yazılırsa da kaybolur.

- [ ] **18.1 Dersi doğru katmana yaz.** İLERİ

  NEDEN Makine gerçeği bir proje dosyasında kalırsa diğer projelere görünmez.

  NASIL "Bu cümle nerede doğru olmaktan çıkıyor" diye sor.

  ```
  Makine ve araç gerçeği   -> ~/.claude/CLAUDE.md
  Koda ait gerçek          -> proje CLAUDE.md
  Repo + bu makine         -> CLAUDE.local.md
  Tarihli ölçüm, kanıt     -> ayrı kanıt dosyası
  ```

  DOĞRULA Her yeni kural doğru dosyada mı?

  Belge: Talimat ve bellek

- [ ] **18.2 "Sonra" ve "unutma" denen her şeyi hemen dosyaya yazdır.** İLERİ

  NEDEN Oturum içi hafıza devirde kaybolur.

  NASIL Proje kökünde bekleyenler dosyası, CLAUDE.md'de buna dair tek kural.

  DOĞRULA "Neler kaldı" sorusu dosyadan cevaplanıyor.

  Belge: Talimat ve bellek

- [ ] **18.3 Aynı düzeltme üçüncü kez geldiyse kural yap.** İLERİ

  NEDEN Üçüncü tekrar artık tesadüf değil.

  NASIL Düzeltmeyi say, üçte kurala çevir.

  DOĞRULA Son ayın düzeltmelerinden kaçı kurala dönüştü?

  Belge: Talimat ve bellek

- [ ] **18.4 Kuralı kısa, kanıtını ayrı tut.** İLERİ

  NEDEN Kısa kural bağlamda taşınır, uzun kanıt yalnız gerektiğinde açılır.

  NASIL Kuralın sonunda kanıt dosyasına bir atıf.

  DOĞRULA Her global kuralın bir kanıt atfı var mı?

  Belge: Talimat ve bellek

19 Bakım ritmi

Ayarlar, kurallar ve izin listeleri zamanla eskir. Küçük ve düzenli bakım, büyük temizlikten ucuzdur.

- [ ] **19.1 Haftalık: kullanımı ve bekleyenleri gözden geçir.** İLERİ

  NEDEN Limiti ve yarım işleri hafta bitmeden görürsün.

  NASIL `/usage` ve aktif projelerin bekleyenler dosyası.

  DOĞRULA Haftalık not: en çok harcayan proje, en eski bekleyen madde.

  Belge: Kullanım ve maliyet

- [ ] **19.2 Eski worktree ve disk kullanımını sahiplik bilgisiyle gözden geçir.** İLERİ

  NEDEN Disk doluluğu derleme ve testleri aksatabilir; tek bir yüzde bütün projeler için yeterli sınır değildir.

  NASIL Aktif süreç, yerel değişiklik ve kurtarma ihtiyacını kontrol et. `prune` yalnız eski yönetim kayıtlarını temizler; kullanılan checkout'u kaldırmaz.

  ```
  git worktree list
  git worktree prune --dry-run
  df -h .
  ```

  DOĞRULA Kaldırılan kesin yol ve korunmuş kanıt belli; boş alan işin beklenen derleme ihtiyacına yeterli.

  Belge: Git worktree

- [ ] **19.3 Aylık: CLAUDE.md'leri buda.** İLERİ

  NEDEN Kurallar birikir, bazıları artık hiç tetiklenmez.

  NASIL Geçerliliğini kaybeden kuralı düzelt; seyrek ama kritik olanı sırf geçen ay tetiklenmedi diye silme.

  DOĞRULA Çelişen veya eski kurallar düzeltilmiş; dosyanın küçük olması tek başına başarı ölçütü değil.

  Belge: Talimat ve bellek

- [ ] **19.4 Aylık: izin listelerini ve hook testlerini gözden geçir.** İLERİ

  NEDEN Yeni komutlar eklenir, Claude Code güncellemesi hook girdisini değiştirebilir.

  NASIL `/fewer-permission-prompts`, ardından hook testleri.

  DOĞRULA Gözetimsiz koşumlarda izin bekleyen adım yok, testler yeşil.

  Belge: Hook referansı

- [ ] **19.5 Çeyrekte bir: kendi kullanım analizini çıkar.** İLERİ

  NEDEN Alışkanlığın değişip değişmediğini sayı söyler.

  NASIL Claude'dan oturum kayıtlarını taramasını iste.

  ```
  ~/.claude/projects altındaki oturum kayıtlarını tara. Çıkar:
  mesaj sayısı, en sık kısa mesajlar ("devam" gibi), bağlam dolma sayısı,
  alt ajan çağrılarının model dağılımı. Önceki çeyrekle karşılaştır.
  ```

  DOĞRULA Sonuçları önceki çeyrekle karşılaştır.

  Belge: Kullanım ve maliyet

- [ ] **19.6 Sürüm, platform ve sağlayıcı farkını kaydet.** İLERİ

  NEDEN Belgede olan özellik eski CLI'da, başka sağlayıcıda veya masaüstü arayüzünde aynı olmayabilir.

  NASIL `claude --version`, `claude --help` ve ilgili komut menüsünü kontrol et; sürüm yükseltmesini kurulum yöntemine göre yap.

  DOĞRULA Rapor ortamı ve erişilemeyen özelliği belirtir; belge kontrolü canlı çalışma testi sayılmaz.

  Belge: Genel bakış

Rehber 1 · Zihinsel model: döngü, bağlam ve harness

Temel · Claude Code'la yaşanan her sürprizin açıklaması üç parçadan birindedir: model, araç ya da harness. Bağlam penceresi modelin gördüğü tek şeydir.

Zihinsel model: bir döngü, üç parça

Claude Code'a bir iş verdiğinde arka planda bir döngü döner. Belge bu döngüyü üç evreyle anlatır: bağlam toplamak, harekete geçmek ve sonucu doğrulamak. Evreler iç içe geçer. Claude bir dosyayı okur, bir şey dener, sonucu görür, yeniden okur. Sen de döngünün herhangi bir anında araya girebilirsin.

Döngüyü üç parça yürütür:

- **Model** karar verir: hangi dosyayı okuyacağına, hangi komutu deneyeceğine, ne zaman duracağına.
- **Araçlar** iş yapar: dosya okur ve düzenler, arama yapar, kabuk komutu çalıştırır, web'e çıkar, alt ajan başlatır.
- **Harness** modeli saran düzendir. Hangi aracın izinli olduğunu, bağlama neyin gireceğini, hangi hook'un çalışacağını o belirler. Claude Code'un kendisi bir harness'tır.

Rehberde sık geçen üç terim: **hook**, bir olay olunca kendiliğinden çalışan küçük bir betiktir. **Skill**, Claude'a bir işin nasıl yapılacağını anlatan, yeniden kullanılabilir bir tarif dosyasıdır. **MCP**, Claude'u dış bir sisteme araç olarak bağlayan sunucudur.

Bu ayrım ilk bakışta akademik görünür. Ama Claude Code'la yaşadığın hemen her sürprizin açıklaması bu üç parçadan birindedir. "Claude kuralımı unuttu" bir bağlam sorunudur. "Claude bu komutu çalıştırmadı" bir izin sorunudur. "Claude yanlış dosyayı düzenledi" bir model kararıdır. Uzman, sonucu gördüğünde önce hangi parçadan geldiğini sorar.

Bağlam penceresi: modelin gördüğü tek şey

Model yalnız bağlam penceresinde olanı görür. Diskteki dosyaları, dünkü konuşmayı, senin aklındaki niyeti görmez. Pencerede konuşma geçmişi, okunan dosyaların içeriği, komut çıktıları, CLAUDE.md, otomatik bellek, yüklenmiş skill'ler ve sistem talimatları durur.

Bir kısmı oturum başında yüklenir, bir kısmı gerektiğinde:

NE ZAMAN	NE YÜKLENİR
Oturum başında	CLAUDE.md dosyaları, otomatik belleğin dizin dosyası, skill açıklamaları, MCP araçlarının adları
Gerektiğinde	Okunan dosyalar, araç çıktıları, çağrılan skill'in gövdesi, eşleşen dosyaya dokununca yol kuralları, alt dizin CLAUDE.md'leri

Her yeni oturum boş bir pencereyle başlar. Oturumdan oturuma taşınan yalnız diskte duran şeylerdir: CLAUDE.md her seferinde yeniden okunur, otomatik bellek yeniden yüklenir, dosyalar yerinde durur. Konuşmanın kendisi taşınmaz. Aynı konuşmaya dönmenin tek yolu `--continue` ya da `--resume` ile o oturumu açmaktır.

Pencere dolunca Claude Code konuşmayı özetler. Pencerenin boyutu modele ve plana göre değişir. Sıkıştırma, yerel 1 milyon token'lık pencereli modellerde varsayılan olarak 967 bin civarında tetiklenir, 200 bin token'lık pencerede sınıra gelince. Belge sıkıştırmanın bedelini açıkça yazar: konuşmanın başındaki ayrıntılı talimatlar kaybolabilir. Proje kökündeki CLAUDE.md ve otomatik bellek sıkıştırmadan sonra diskten yeniden eklenir. Sohbette verdiğin bir talimat ise özetin insafına kalır.

Neyi kim yapar

SORU	CEVABI VEREN	NEREDEN GÖRÜRSÜN
Claude bunu biliyor mu?	Bağlam penceresi	`/context`, `/memory`

SORU	CEVABI VEREN	NEREDEN GÖRÜRSÜN
Bunu yapmaya karar verecek mi?	Model	Konuşmanın kendisi, `Ctrl+O` transkript
Yapmasına izin var mı?	Harness: izin kuralları, mod, hook	`/permissions`, `/hooks`, `/status`
Yaptığı şey doğru mu?	Kimse kendiliğinden söylemez	Test, ekran, diff: senin kanıtın

Son satır bilerek boş bırakıldı. Döngünün "doğrula" evresi modelin kendi kontrolüdür. Senin kontrolünün yerini tutmaz. Bunu Bölüm 4: Kanıtla teslim: iddiayı sayıya çevir ve kontrol listesinin doğrulama bölümü ayrıntılandırıyor.

Alt ajan: ayrı bir pencere

Alt ajan kendi bağlam penceresinde çalışır ve ana oturuma yalnız bir özet döndürür. Konuşma geçmişini, okunmuş dosyaları ve çağrılmış skill'leri görmez. Ana oturum ona görevi anlatan bir devir mesajı yazar, alt ajan oradan başlar. Bu iki sonuç doğurur. Geniş bir aramayı alt ajana vermek ana pencereyi temiz tutar. Ama alt ajan, senin oturumda söylediğin bir şeyi ancak devir mesajına girdiyse bilir. Bir istisna var: fork, ana konuşmanın kopyasıyla başlar.

Karar kuralları

- **Claude bir şeyi "bilmiyorsa" önce bağlama bak.** Bilgi pencerede yoksa model için yoktur. `/context` neyin yüklendiğini, `/memory` hangi talimat dosyalarının okunduğunu gösterir. 2.2
- **Oturum boyunca geçerli olması gereken talimatı sohbete değil dosyaya yaz.** Sıkıştırmadan sonra dosya geri gelir, sohbet gelmez. 2.6
- **Konu değişince yeni oturum aç.** Eski işin dosyaları ve kararları penceredeyse yeni işi de etkiler. 2.3
- **Çok dosya okuyacak bir araştırmayı alt ajana ver.** Ana pencereye yalnız sonuç girer. 2.4
- **Büyük çıktıyı pencereye değil dosyaya aldır.** Uzun bir log'un yüzde birlik kısmı gerekiyorsa geri kalanı yer kaplamasın. 2.5
- **Bir sürprizi açıklarken üç soruyu sırayla sor:** Bağlamda var mıydı? Model ne karar verdi? Harness izin verdi mi?

Hata biçimleri

Gerçek vaka: kural vardı, model görmedi. Ortak kurallar bir dosyaya taşınıp dört projenin CLAUDE.md'sine import edildi. `claude -p` ile koşan, ekransız ve soru sormayan headless oturumda model yalnız import satırını gördü, dosyanın içeriğini görmedi, çünkü proje dışı import onayı o koşumda hiç sorulmamıştı. Model kuralı "çiğnemedi", kural hiç bağlama girmedi. Ayrıntısı Gerçek deneyimlerden on ders sayfasında D07.

Gerçek vaka: bağlamdaki eski bilgi, model için gerçektir. On gün önceki bir rapor "uygulama yalnız bir ülkede satışta" diyordu ve bu cümle oturuma yapıştırıldı. Model buna dayanarak öneri yaptı. Gerçekte uygulama 175 ülkede satıştaydı. Pencereye giren her cümle, kaynağına bakılmadıkça doğru kabul edilir. Ayrıntısı D10.

Kurgu senaryo: "Dün söylediğimi unuttu." Elif bir gün boyunca Claude'la bir rapor betiği üzerinde çalışır ve tarih biçiminin gün/ay/yıl olacağını sohbette söyler. Ertesi gün yeni bir oturum açar, betik ay/gün/yıl üretir. Model unutmadı, o bilgi yeni pencerede hiç yoktu. Kural CLAUDE.md'ye yazılınca sorun bir daha çıkmaz.

Alıştırma: bağlamı ölç

Herhangi bir git deposunda yap.

1. Yeni bir oturum aç. Hiçbir şey sormadan `/context` yaz, çıktıyı bir dosyaya kaydet. 2. Depodaki en büyük kaynak dosyalardan birini Claude'a okut ve ne yaptığını özetlemesini iste. `/context` çıktısını yeniden kaydet. 3. `/memory` ile hangi talimat dosyalarının yüklendiğine bak. Listede beklemediğin ya da eksik bulduğun bir dosya var mı? 4. CLAUDE.md'ye geçici bir satır ekle, ör. "Bu projede tarih biçimi gün/ay/yıl." Yeni bir oturum aç ve Claude'a dosya okutmadan tarih biçimini sor. Sonra satırı sil.

Bitti sayılması için: iki `/context` çıktısı ve üç cümle. Hangi kategori ne kadar büyüdü, neden büyüdü, 4. adımda Claude bilgiyi nereden aldı.

Bağlantılar

Kontrol listesi: 2.1 2.2 2.3 2.4 2.5 2.6 1.5 1.7. Ekler: Komutlar ve kısayollar, Dosya konumları.

Rehber 2 · İş vermek: bitiş çizgisini çizmek

Temel · İyi bir istek hedefin yanında bitiş ölçütünü, kısıtı, örneği ve okuru da söyler. Ölçütü Claude'un kendi koşturabileceği bir kanıta çevirirsen tek doğrulayıcı sen kalmazsın.

Zihinsel model: iş vermek, bitiş çizgisini çizmektir

Claude Code'a verdiğin istek bir talimat olduğu kadar bir sözleşmedir. İsteğin "ne yapılacak" kısmı genellikle belli olur. Eksik kalan kısım "ne zaman bitti sayılacak" ve "neye dokunulmayacak" sorularıdır. Bu iki boşluğu sen doldurmazsan Claude kendi tahminiyle doldurur ve tahmin çoğu zaman makul görünür ama senin aklındakiyle aynı çıkmaz.

Belgenin en çok vurguladığı alışkanlık da budur: Claude'a işin sonucunu kendi kendine kontrol edebileceği bir yol ver. Testler, beklenen çıktı, ekran görüntüsü ya da bir komutun çıkış kodu bu yolu sağlar. Böyle bir ölçüt yoksa tek doğrulayıcı sen kalırsın ve her parçada Claude seni bekler. Ölçüt varsa Claude denemeyi, ölçmeyi ve düzeltmeyi kendi döngüsünde yapar.

İyi bir iş isteği beş parçadan oluşur:

PARÇA	CEVAPLADIĞI SORU	EKSİKSE NE OLUR
Hedef	Ne değişecek?	Claude başka bir şeyi çözer
Bitiş ölçütü	Hangi kanıtla biter?	İş hep "bitti gibi" görünür
Kısıt	Neye dokunma?	Çalışan kısmı bozarsın
Örnek	Çıktı nasıl görünür?	Biçim tutmaz
Okur ve yer	Kim, nerede kullanacak?	Ton ve ayrıntı seviyesi şaşar

Bitiş ölçütü ve doğrulanabilir kabul ölçütü

"Giriş formundaki hatayı düzelt" bir hedeftir, bitiş ölçütü değildir. "Boş e-postayla gönderince form kırmızı uyarı göstersin, `npm test` içinde bu durumu sınayan yeni bir test olsun ve tüm testler geçsin" ise bitiş ölçütüdür. İkinci cümlede Claude'un kendi başına koşturabileceği iki şey var: bir test ve bir komut. 3.1

Kabul ölçütünün sınaması basit: bir yabancı sadece bu cümleye bakıp "oldu" ya da "olmadı" diyebilir mi? "Hızlı açılsın" diyemez. "Ana sayfa soğuk açılışta iki saniyenin altında yüklensin, ölçümü şu komut versin" diyebilir. Ölçülemeyen bir ölçüt yazamıyorsan işi ikiye böl: önce ölçmeyi kuran bir iş, sonra ölçüte göre çalışan bir iş. 3.2

Kısıt ve korunacak davranış

Claude bir hatayı düzeltirken yakındaki "çirkin" kodu da toparlayabilir. Niyeti iyidir ama diff'in okunmasını zorlaştırır ve çalışan bir davranışı bozabilir. Bunu kısıtla önlersin: hangi dosyaya dokunulacağını, hangi davranışın aynı kalacağını ve neyin yapılmayacağını söyle.

Güçlü kısıt cümlesi üç şey içerir: korunacak davranış, sınır ve kanıt. Örnek: "Ödeme akışı aynı çalışsın, yalnız `fiyat.py` içindeki yuvarlamayı değiştir, mevcut ödeme testleri değişmeden geçsin." Yasak listesi uzun olmak zorunda değil. Sadece Claude'un makul bulup yapacağı ama senin istemediğin şeyleri yaz. 3.4

Örnek ve karşı örnek

Çıktının biçimini anlatmaktansa göstermek daha kısadır ve yanlış anlaşılmaz. Bir örnek "buna benzesin" der. Karşı örnek "buna benzemesin" der ve çoğu zaman daha çok bilgi taşır, çünkü Claude'un kolay yolunu keser.

Kurgu senaryo: Sibel haftalık durum özeti istiyor. Bir önceki haftanın özetini örnek olarak yapıştırıyor ve altına bir karşı örnek ekliyor: "Şu özet fazla uzundu, her satırın başında 'bu hafta' yazıyordu, bunu tekrarlama." İlk

denemede format tutuyor. Karşı örnek olmasaydı Claude örneğin uzunluğunu ve sözcüklerini kopyalayacaktı. 3.5

Plan modu ne zaman

Plan modu Claude'a dosyaları okutur ama düzenletmez. Sen planı onaylayınca uygulama başlar. Belgeye göre `Shift+Tab` ile durum çubuğunda `plan mode on` yazana kadar geçilir ya da `claude --permission-mode plan` ile başlanır. Bu iş akışı önerisi değil, belgelenmiş bir mod. Ne zaman kullanacağın ise senin kararın:

- Hedef belirsizse, birden çok dosyaya dokunuyorsa ya da yaklaşımı sen de bilmiyorsan önce plan iste.
- Değişiklik bir cümleyle diff olarak anlatılabiliyorsa, ör. bir yazım hatası ya da tek satırlık düzeltme, plan modu zaman kaybıdır.
- Gereksinimleri kendin de net bilmiyorsan belgenin önerisi: Claude'dan seni röportaj yapıp bir `SPEC.md` yazmasını iste, sonra temiz bir oturumda o dosyadan inşa et. 3.3

"İyi"yi ölçüte çevirmek

"En iyi", "kaliteli", "profesyonel" gibi sözcükler Claude'a yön vermez, çünkü iyinin ne olduğunu senin kafanda kalan kıstaslar belirler. Sözcüğü bir ölçüte çevir. "Kaliteli bir README" yerine "kurulum üç komutta bitsin, ilk örnek çalışsın, bir ekran görüntüsü olsun" yaz. "İyi performans" yerine bir süre, bir bellek sınırı ya da bir kıyas komutu yaz. 4.1

Okur ve kullanım yeri

Aynı bilgi bir mühendise yazılan commit mesajında, bir yöneticiye giden özette ve bir müşteri e-postasında farklı görünür. Claude okuru bilmezse ortalama bir ton seçer. "Bu metni bütçeyi onaylayacak yönetici okuyacak, teknik terim kullanma, tek ekranı geçmesin" demek sonucu belirgin biçimde değiştirir. Kullanım yeri de aynı işi görür: bir e-posta, bir slayt, bir kod yorumu, bir issue. 4.2

Karar kuralları

- **İsteği göndermeden önce şunu sor:** Claude bitirdiğinde "oldu" demek için hangi kanıta bakacağım? Cevap yoksa ölçütü ekle.
- **Ölçütü Claude'un koşturabileceği biçimde yaz.** Bir test, bir komut, bir dosyada beklenen satır. Senin gözüne kalan kontrol seni darboğaz yapar.
- **Kısıtı dört parça yap:** neye dokunabilir, neye dokunamaz, hangi davranış aynı kalır, neyin kanıtı var.
- **Biçim önemliyse bir örnek yapıştır.** Çoğu zaman bir de karşı örnek ekle.
- **Küçük ve net işi doğrudan yaptır.** Büyük ve belirsiz işte önce plan iste.
- **Sıfatları ölçüte çevir.** Okuru ve yeri her yazı işinde söyle.

Hata biçimleri

Gerçek vaka: ölçüt yoktu, yeşil yanıltıcıydı. Sıfır test koşan bir komut "başarılı" dedi (D02, Gerçek deneyimlerden on ders). İstekte yalnız "testler geçsin" yazıyorsa bu yeşil kabul edilir. "En az şu kadar test koşsun ve geçsin" yazıyorsa sıfır hemen göze çarpar. Vakanın ayrıntısı Bölüm 4: Kanıtla teslim: iddiayı sayıya çevir bölümünde.

Gerçek vaka: olgu, eski rapordan geldi. İsteğin içine yapıştırdığın her olgu ölçütün bir parçası olur, yanlış olgu yanlış ölçüt doğurur. Eski bir raporun cümlesinin oturuma girip öneriyi nasıl bozduğunu Bölüm 1: Zihinsel model: döngü, bağlam ve harness bölümünde gördün (D10).

Kurgu senaryo: kısıtsız düzeltme. Kaan bir sipariş toplamı hatasını düzelttirir. Claude hatayı düzeltir, ayrıca yakındaki üç fonksiyonun adını "daha okunur" olacak şekilde değiştirir. Diff 40 satıra çıkar, Kaan dört fonksiyonun hangisinin gerçekten gerektiğini ayıklayamaz ve hepsini geri alır. "Yalnız `toplam.py`, başka dosya ve ad değişikliği yok" kısıtı bunu önlerdi.

Alıştırma: aynı iş, iki istek

Herhangi bir git deposunda yap. Küçük bir iş seç, ör. bir fonksiyona giriş doğrulaması eklemek ya da bir betiğe `--yardim` seçeneği koymak.

1. İlk isteği zayıf yaz, tek cümle: "X'e doğrulama ekle". İş bitince çıktıyı ve `git diff --stat` satırını kaydet. Sonra `git restore .` ile değişikliği geri al. 2. Aynı işi bu sefer güçlü isteyle ver. İsteğe şunlar girsin: bitiş ölçütü (hangi komut, hangi çıktı), iki kabul ölçütü, bir kısıt (dokunulacak dosya, korunacak davranış), biçim için bir örnek ve bir karşı örnek. 3. İkisini karşılaştır: diff kaç dosyaya ve kaç satıra değdi? Claude kaç kez durup sana soru sordu? Test ya da komut kanıtı sundu mu? 4. Büyük bir iş seçtiysen ikinci denemeyi plan modunda başlat ve planı onaylamadan önce kısıtlarına uyup uymadığına bak.

Bitti sayılması için: iki satırlık bir karşılaştırma tablosu (dosya sayısı, satır sayısı, soru sayısı, kanıt var mı) ve bir cümle: hangi ölçüt farkı yarattı.

Bağlantılar

Kontrol listesi: 3.1 3.2 3.3 3.4 3.5 4.1 4.2. Önceki bölüm: Bölüm 1: Zihinsel model: döngü, bağlam ve harness. Sonraki: Bölüm 3: Oturum: masayı doğru yönetmek, Bölüm 4: Kanıtla teslim: iddiayı sayıya çevir. Ekler: Komutlar ve kısayollar, İpuçları ve püf noktaları.

Rehber 3 · Oturum: masayı doğru yönetmek

Temel · Her oturum komutu bağlam penceresinin içeriğini değiştirir. Konu başına oturum, devir dosyası, sıkıştırma, dallandırma ve geri sarma arasında hangisinin ne zaman kullanılacağı ve sınırları.

Zihinsel model: oturum, bir çalışma masasıdır

Bir oturumun bağlam penceresi küçük bir çalışma masasına benzer. Üzerinde konuşma, okunan dosyalar ve komut çıktıları durur. Masa doldukça önemli kâğıtlar altta kalır. Yeni bir iş için masayı boşaltmak, aynı masada üç işi birden yürütmekten hem ucuz hem güvenlidir. Belge bunu doğrudan söyler: `/clear` ilgisiz görevler arasında maliyetsizdir ve sonraki görev bir öncekinin geçmişine ödeme yapmaz.

Oturumları yönetmek beş soruya iner. Bu iş için yeni masa mı açayım? Masayı bir yere not alıp mı bırakayım? Aynı masaya mı döneyim? Masayı kopyalayıp başka bir yol mu deneyim? Yaptığım son şeyi geri mi alayım? Bölüm 1: Zihinsel model: döngü, bağlam ve harness bölümündeki zihinsel modeli hatırla: Claude'un bildiği, pencerede olandır. Bu bölümdeki her komut pencerenin içeriğini değiştirir.

Konu başına oturum

Bir oturum tek bir tutarlı işe ait olsun. Önceki işin dosyaları ve kararları pencerede kalırsa yeni işi de etkiler. Yeni bir konuya geçince `/clear` yaz. Belge ayrıca `/clear oauth-gecisi` gibi bir adla önceki konuşmayı etiketleyebileceğini söyler, böylece sonra `/resume` listesinde bulursun. 2.1 2.3

Belgenin bir başka uyarısı: Claude'u iki kez düzelttin ve hâlâ yanlış yapıyorsa üçüncü düzeltmeyi yazma. `/clear` yap ve öğrendiklerinle daha iyi bir prompt yaz. Kirli bağlamda yapılan üçüncü düzeltme genellikle ilk ikisinin üstüne eklenen bir karmaşa olur.

Devir dosyası

Konuşma taşınmaz, dosya taşınır. İş yarım kalacaksa oturumu kapatmadan Claude'dan bir devir dosyası yazmasını iste: neyi bitirdin, neyi bitirmedin, hangi kararları verdin, sıradaki adım ne. Dosya projede durur, yeni oturum onu okuyarak kaldığı yerden başlar. Bu belgenin zorunlu kıldığı bir özellik değil, iş akışı önerisidir. Ama CLAUDE.md'nin ve dosyaların oturumlar arası tek taşıyıcı olduğu gerçeğine dayanır.

İyi bir devir dosyası üç şey içerir: bitenin kanıtı, bekleyen işin adımları ve "denedim, olmadı" listesi. Üçüncüsü en çok unutulandır ve yeni oturumun aynı çıkmaz sokağa girmesini önler. 2.1

/compact: ne kaybolur

`/compact` konuşmayı özetleyip sürdürür. Belgeye göre sıkıştırmadan sonra şu şeyler diskten geri gelir: proje kökündeki CLAUDE.md, kapsamsız kurallar, otomatik bellek ve plan modunda yazılan plan. Çağırdığın skill'lerin gövdesi de geri gelir (skill başına 5.000, toplam 25.000 token, eskiler düşer). Son okunan ya da düzenlenen dosyalardan en çok beşi yeniden okunur ve 5.000 token'dan büyük dosya yalnız yol olarak döner.

Şunlar özetin insafına kalır: sohbette verdiğin talimatlar, `paths` alanıyla yüklenen kurallar ve alt dizin CLAUDE.md dosyaları (bunlar ilgili dosyaya yeniden dokunulunca geri yüklenir), hook'ların eklediği bağlam ve araç çıktılarının ayrıntısı. Bu yüzden `/compact` kullanırken neyin korunacağını yaz: `/compact API değişikliklerini ve test komutlarını koru`. Önemli bir talimat sohbetteyse önce dosyaya taşı. 2.6

/resume ve --continue

Aynı konuşmaya dönmenin iki yolu var. `claude --continue` bu dizindeki son oturumu açar. Orada oturum yoksa belge bunu açıkça yazar: "No conversation found to continue" der ve çıkar. `/resume` (ya da `claude --resume`) bir seçici açar ve `/rename` ile ad verdiysen adla bulursun. Geçmişe dönmek yeni bir masa değil, eski masaya dönmektir: o gün pencerede ne varsa geri gelir, düne ait eski bilgi de dahil. 2.7

/branch ile /fork farkı

İkisi de konuşmayı kopyalar ama kopyanın nerede yaşadığı farklıdır.

KOMUT	NE YAPAR	SEN NEREDE KALIRSIN
`/branch [ad]`	Konuşmayı bu noktadan dallar	Dalın içine geçersin, orijinale `/resume` ile dönersin
`/fork [prompt]`	Kopyayı arka plan oturumunda sürdürür	Bulunduğun oturumda kalırsın

Belgenin dediği: `/fork` kopyayı ayrı bir arka plan oturumu olarak çalıştırır. Agent view kapalıysa `/fork` bir fork edilmiş alt ajan başlatır. İki yolu yan yana denemek istiyorsan `/branch` yeterli. Birini arka planda koştururken kendi işine devam etmek istiyorsan `/fork` kullan. Yerel sürümün `/fork`'u desteklemeyebilir, belge daha yeni olabilir: `claude --version` ile bak. 2.7

/rewind ve sınırı

`/rewind` (ya da boş istemde `Esc Esc`) kodu, konuşmayı ya da ikisini bir önceki noktaya sarar. Menüde "Summarize from here" ve "Summarize up to here" seçenekleri de var, yani rewind aynı zamanda seçmeli bir `/compact` gibi de çalışır. Istemde yazı varsa çift `Esc` yazıyı siler, rewind menüsünü açmaz. Silinen yazıyı yukarı okla geri çağırırsın.

Sınırı bil. Belgeye göre rewind yalnız Claude'un dosya araçlarıyla düzenlediği dosyaları izler. Yalnız Bash komutuyla değişen bir dosya geri alınmaz. Oturum dışında elle yaptığın değişiklikler ve başka bir paralel oturumun düzenlemeleri de normalde yakalanmaz. Asıl güvence Git'tir. Riskli bir adımdan önce commit at, rewind'a yedek olarak güvenme. 15.2

Esc ile araya girmek ve /btw

`Esc` Claude'u işin ortasında durdurur ve o ana kadarki her şeyi korur. Belge ilk dakikayı izlemeyi önerir: Claude yanlış yöne giderse hemen `Esc` bas ve yönlendir. Geç kalırsan yanlış yönde yüz satır üretilir.

`/btw` ise ana işi bölmeden kısa bir soru sormaya yarar. Soru ve cevap konuşma geçmişine girmez, bu yüzden bağlamı şişirmez. Claude cevabı yalnız o anda konuşmada olan bilgiden verir, yeni dosya okumaz. Yani "az önce hangi yapılandırma dosyasından söz etmiştin?" iyi bir `/btw` sorusudur. "Şu dosyada ne var?" değildir.

Karar kuralları

- **Konu değişti mi? `/clear`.** Konuyu etiketleyerek çık, sonra bulursun.
- **Yarım kalan iş mi? Önce devir dosyası, sonra çık.**
- **Sıkıştırmadan önce korunacakları yaz.** Sohbette kalan talimatı dosyaya taşı.
- **Dün üzerinde çalıştığın işe mi dönüyorsun? `--continue` ya da `/resume`.** Yeni bir yaklaşım için `/clear`.
- **İki yol mu deneyeceksin? `/branch`.** Biri arka planda koşsun istiyorsan `/fork`.
- **Son adım yanlış mıydı? `/rewind`, ama Bash'le değişen dosyayı Git'ten geri al.**
- **Claude yanlış yöne gidiyorsa `Esc`.** Kısa bir yan soru mu? `/btw`.

Hata biçimleri

Gerçek vaka: oturumlar arası ortak olan şey sandığından fazlaydı. Birden çok oturum aynı depoda çalışırken stage edilen dosyalar, başka bir oturumun commit'iyle başka bir mesajın altına girdi. Oturum yönetimi yalnız konuşmayla ilgili değil, ortak kaynaklarla da ilgilidir. Ayrıntısı Gerçek deneyimlerden on ders sayfasında D06. Stage ile commit'i tek adımda yapmak bu sorunu önler.

Gerçek vaka: geçici dizin devir dosyası için kötü yerdi. Birden çok gün sürecek bir işin kaynakları oturumun geçici çalışma dizinindeydi ve dizin toptan silindi. Devir dosyasını proje dizinine yaz. Ayrıntısı D09.

Kurgu senaryo: rewind'ın görmediği dosya. Defne, Claude'a bir betik çalıştırtır. Betik `rapor.csv` dosyasını yeniden üretir ve eski hâlini ezer. Sonuç yanlıştır, Defne `/rewind` yapar. Kod düzenlemeleri geri döner, `rapor.csv` yerinde kalır, çünkü yalnız Bash değiştirmiştir. Dosya Git'te olsaydı `git restore rapor.csv` bir saniyede çözerdi.

Alıştırma: iki yolu dallayıp birini seç

Herhangi bir git deposunda yap. Kendi çalışma dalında kal.

1. Küçük bir iş seç ve iki makul yolu olduğundan emin ol, ör. bir listeyi önce sıralamak mı yoksa önce filtrelemek mi. Claude'a işi anlat ve "henüz kod yazma, yaklaşımı konuşalım" de. Konuşma bu noktaya gelince iki yol da açık kalır. 2. `/branch yol-a` yaz. Bu dalda birinci yolu uygulat, sonucu `git diff --stat` ile kaydet, sonra `git restore .` ile dosyaları temizle. 3. `/resume` ile ilk konuşmaya, yani dallanma noktasına dön. `/branch yol-b` yaz ve ikinci yolu uygulat. Sonucu yine `git diff --stat` ile kaydet. 4. İki diff'i yan yana oku. Hangisi daha küçük, hangisi daha okunur, hangisinin testi var? Birini seç. Seçmediğin yolun diff'i artık bir kayıttan ibaret, onu zaten geri almıştın. 5. Seçtiğin yolu seçtiğin dalda yeniden uygulat, commit'le ve bir devir dosyası yaz: ne seçtin, neden, öteki yolda ne gördün.

Bitti sayılması için: iki yolun diff özeti, seçimin gerekçesi bir cümle ve `/resume` listesinde iki dalın adı.

Bağlantılar

Kontrol listesi: 2.1 2.3 2.6 2.7 15.2. Önceki bölüm: Bölüm 2: İş vermek: bitiş çizgisini çizmek. Sonraki: Bölüm 4: Kanıtla teslim: iddiayı sayıya çevir. Ekler: Komutlar ve kısayollar, Sorun giderme, Dosya konumları.

Rehber 4 · Kanıtla teslim: iddiayı sayıya çevir

Temel · "Geçti" bir iddiadır. Sayı, kasten kırılan kontrol, önce ve sonra görüntü ve diff ile eşleşen rapor ise kanıttır. Teslim yerel, push ve yayın olarak üç ayrı aşamadır.

Zihinsel model: kanıt, iddianın yerini tutmaz

Claude Code bir işi bitirdiğinde "tamamlandı, testler geçiyor" diyebilir. Bu cümle bir kanıt değil, bir iddiadır. Bölüm 1: Zihinsel model: döngü, bağlam ve harness bölümündeki döngünün son evresi doğrulamadır ve belge açık söyler: yazdığı kodun çalıştığını gösterebilecek bir kanıt vermezsen tek doğrulayıcı sen kalırsın. Gözetmediğin bir oturumda ise elindeki tek kanıt budur.

Bu bölümün tek ilkesi şu: teslim almadan önce "geçti" sözcüğünü bir sayıya, bir çıktıya ya da bir görüntüye çevir. Çevirebildiğin şey kanıttır. Çeviremediğin şey Claude'un anlatısıdır ve anlatı çoğu zaman doğrudur ama doğru olmadığı gün pahalıdır.

"Geçti" yerine sayı

Test aracı "başarılı" der. Bu tek sözcük iki çok farklı durumu örter: 200 test koştu ve geçti, ya da sıfır test koştu ve hiçbir şey bulmadığı için geçti. İkincisi yeşildir ama hiçbir şey ölçmemiştir. Bu yüzden her sonuçta durum satırının yanındaki sayıyı oku: kaç test koştu, kaç dosya tarandı, kaç satır değişti. Beklediğin sayıyla eşleşmiyorsa yeşil bir şey söylemiyor. 12.1

Sıfır test koşan filtre

Test araçları tek bir testi çalıştırmak için bir filtre kabul eder. Filtre hiçbir testle eşleşmezse bazı araçlar hata vermez ve boş koşuyu başarılı sayar. Süitin okunur adını verirsen eşleşme olmayabilir. Tip ya da fonksiyon adıyla filtrelersen eşleşir ve çıktıda sayı sıfırın üstüne çıkar. Sözdizimi araca göre değişir, ilke değişmez: filtreli koşunun sonunda test sayısı sıfırdan büyük olmalı.

Gerçek vaka D02. Test komutuna süitin okunur adı filtre olarak verildi, eşleşen test olmadı ve çıktı "başarılı" dedi. Rapora "test geçiyor" diye girmek üzereydi. Aynı gün iki benzer yeşil daha çıktı: hiçbir dosya bulamadığı için geçen bir lint ve penceresi dar olduğu için körleşen bir lint (lint, kod stilini ve basit hataları denetleyen araçtır). Üçünün ortak yanı, durum satırının doğru, kapsamın boş olmasıdır. Ayrıntısı Gerçek deneyimlerden on ders sayfasında D02. 12.1

Yeni kontrolü hatayı geri koyarak sınamak

Claude yeni bir test, lint kuralı ya da doğrulama betiği yazdıysa onun çalıştığını göstermek için bir tek yol var: kontrolün yakalaması gereken hatayı kasten geri koy ve kontrolün düştüğünü gör. Düşmüyorsa kontrol hiçbir şeye bakmıyor demektir. Sonra hatayı çıkar ve kontrolün yeşile döndüğünü gör. Bu üç adımlık döngü (yeşil, kasten kır, kırmızı, düzelt, yeşil) bir kontrolün gerçekten çalıştığının tek kanıtıdır. 12.2

Gerçek vaka D08 (yan örnek). Ağır komutları reddeden bir hook (araç çalışmadan önce devreye giren betik, ayrıntısı Bölüm 8: Hook ile garanti: olay, çıkış kodu, sınama) 16 vakalık bir testle sınandı ve sıfır yanlış verdi. Test gerçekten bakıyor mu diye hafif komut istisnası kasten bozuldu ve test iki yanlış verdi. Düşen test, testin işe yaradığını gösterdi. Ayrıntısı D08.

Görsel değişikliği önce/sonra görmek

Bir arayüz, bir slayt ya da bir grafik değiştiyse metin raporu yetmez. Claude "düğmeyi hizaladım" diyebilir ve düğme hâlâ kaymış olabilir. Değişiklikten önce bir ekran görüntüsü al, sonra bir tane daha al ve yan yana bak. Belge `/verify` komutunu önerir: Claude uygulamayı derler, çalıştırır ve sonuca bakar, testlerin ve tip kontrolünün kaçırdığı sorunları yakalar. Görüntü yoksa "ne değişmiş ki" sorusunun cevabı da yoktur. 12.5

Alt ajan raporunu kanıtla eşleştirmek

Alt ajan kendi penceresinde çalışır ve ana oturuma bir özet döner. Özet, ajanın yaptığı şeyin anlatısıdır. "Dört dosyayı güncelledim ve testler geçti" cümlesini üç şeyle eşleştir: `git diff --stat` gerçekten dört dosya mı gösteriyor, test çıktısında sayı var mı, alt ajanın dokunduğunu söylediği her dosyada değişiklik görünüyor mu? Eşleşmeyen her parça bir soru işaretidir. Bunu kendin yapamıyorsan ayrı bir doğrulayıcı ajana "bu rapordaki her iddiayı kanıtla kontrol et" diye ver, ama yine ondan da çıktı iste. 12.3

Gerçek vaka D10. Yapıştırılan bir denetim notundaki "uygulama yalnız bir ülkede satışta" cümlesi kaynağında kontrol edilmedi (Bölüm 1: Zihinsel model: döngü, bağlam ve harness). Aynı oturumda bir sürüm notundaki "hepsinin görseli var" iddiası da 11 eksik görselle çürüdü. Rapor ne kadar özenli yazılmış olursa olsun, değişebilen bir durumu kaynağında kontrol etmeden aktarma. Ayrıntısı D10. 12.3

Bağımsız inceleme ve üç satır

Kendi yazdığın işi kendin incelemek zayıf bir kontroldür, çünkü aynı varsayımlarla bakarsın. Önemli bir değişikliği teslimden önce `/code-review` ile geçir. Komut güncel dalı ya da bir PR'ı, dalı veya yolu inceler. `--fix` bayrağı bulguları çalışma ağacına uygulatır, `--comment` ise GitHub'da satır içi yorum bırakır. Belgenin bir uyarısı da var: kendi yazdığın inceleme prompt'unda "stili atla" kısmını silme, yoksa inceleyici her zaman bir şey bulur ve bulgu listesi gürültüyle dolar. 12.6

İş bitince ayrıca Claude'dan üç satır iste: neyi kaçırmış olabilir, hangi iddiası en zayıf, hangi kararı gerekçesiz. Bu satırlar övgüyü değil kırma denemesini tetikler. 12.4

Commit kapsamı ve teslim aşamaları

Bir commit tek bir mantıklı parça olsun. Kaynak kodu Git'te dursun ve commit'e neyin gireceğini sen tanımla. Başka bir oturum ya da başka bir iş aynı depoya yazıyorsa stage ve commit'i tek komutta yap. **Gerçek vaka D06:** değişiklikler stage edildi, üç dakikalık bir derleme sürerken başka bir oturum kendi raporunu commit'ledi ve commit index'in tamamını aldı. Dört dosya yanlış bir mesajın altına girdi. Kanıt `git log --stat` çıktısındaydı. 15.1

Teslimin üç ayrı aşaması var ve her birini kendi kanıtıyla adlandır:

AŞAMA	ANLAMI	KANIT
Yerel	Değişiklik bu makinede çalışıyor	Test sayısı, çıktı, görüntü
Push	Değişiklik uzak depoda	Dal adı, commit kimliği, CI (her push'ta testleri otomatik koşturan sistem) sonucu
Yayın	Kullanıcı artık görüyor	Yayındaki sürümün kendisine bakılmış olması

"Bitti" demek bu üçünden hangisini kastettiğini söylemeden anlamsızdır. Push ve deploy yetkisini hedefe ve kapsama bağla: bir dala push izni vermek başka bir dala ya da yayına izin vermek demek değildir. 12.7 15.3

Karar kuralları

- **"Geçti" gördüğünde sayıyı ara.** Sayı yoksa ya da sıfırsa kontrol boştur.
- **Yeni kontrolü kasten kırarak sına.** Düşmeyen kontrolü bitmiş sayma.
- **Görsel işte iki görüntü iste.** Önce, sonra, yan yana.
- **Alt ajan raporunu diff ve çıktıyla eşleştir.** Eşleşmeyen parçayı geri sor.
- **Önemli değişikliği ayrı bir gözle incele.** `/code-review`, gerekirse `--fix`.
- **Teslim aşamasını adlandır.** Yerel, push ya da yayın. Her birinin kendi kanıtı var.

Hata biçimleri

Kurgu senaryo: yeşil ama boş. Ayşe, Claude'a bir hata düzelttirir ve yeni bir test yazdırır. Claude "yeni test geçiyor" der. Ayşe testi çalıştırır, yeşil görür, commit'ler. Haftalar sonra aynı hata geri gelir. Test dosyasının adı

yanlış desenle başladığı için hiç keşfedilmemiş, hiç koşmamıştır. Hatayı geri koyup testin düştüğünü görmüş olsaydı sorun o gün çıkardı.

Kurgu senaryo: raporun fazlası. Emre bir alt ajana "tüm modülleri güncelle" der. Rapor altı modülü sayar. `git diff --stat` dördünü gösterir. İki modül hiç değişmemiştir ve rapor iki satırı niyetle değil beklentiyle yazmıştır.

Alıştırma: kasten bozulan testin gerçekten düştüğünü göster

Bunu bir deneme deposunda ya da kendi çalışma dalında yap, işin sonunda değişiklikleri geri al.

1. Küçük bir fonksiyon seç ve Claude'dan onun için bir test yazmasını iste. 2. Testi kendi başına koştur. Durum satırının yanındaki test sayısını yaz. Sayı 1 ya da üstü olmalı, sıfırsa filtreni düzelt. 3. Fonksiyonun içine kasten bir hata koy, ör. bir `+` işaretini `-` yap. Testi yeniden koştur ve kırmızı olduğunu gör. 4. Test kırmızı olmadıysa testin yanlış şeye baktığını biliyorsun. Claude'a testin neden yakalamadığını sor ve testi düzelt. 5. Hatayı geri al, testi yeniden koştur ve yeşile döndüğünü gör. Bir sorun çıktıysa hepsini `git restore .` ile temizle.

Bitti sayılması için: üç çıktı: yeşil koşu (test sayısıyla), kasten bozulmuş koşu (kırmızı) ve düzeltilmiş koşu (yeşil), her birinin yanında test sayısı. Bir cümle: bu test gerçekten neye bakıyor.

Bağlantılar

Kontrol listesi: 12.1 12.2 12.3 12.4 12.5 12.6 12.7 15.1 15.3. Önceki bölüm: Bölüm 3: Oturum: masayı doğru yönetmek. Ekler: Gerçek deneyimlerden on ders, Komutlar ve kısayollar.

Rehber 5 · Talimat mimarisi: rica ile garanti

Orta · Bir kural ya Claude'a söylenir ya da Claude Code'a uygulatılır. Hangisinin nereye gideceğini bilmek, kuralın tutup tutmayacağını belirler.

Zihinsel model: rica ile garanti

Claude Code'a bir kuralı iki türlü verebilirsin. Ya Claude'a söylersin, ya da Claude Code'a uygulatırsın. Belge bu ikisini ayrı adlarla anar. Birinde **Claude karar verir**: prompt, CLAUDE.md, kural dosyası ve Claude'un kendi seçtiği skill'ler. Ötekinde **Claude Code uygular**: hook, deny kuralı ve yönetilen ayar. İlki çoğu zaman tutar. İkincisi her zaman tutar.

Bu ayrım CLAUDE.md'nin nasıl iletildiğinden gelir. CLAUDE.md sistem prompt'unun parçası değildir. Sistem prompt'undan sonra bir kullanıcı mesajı olarak modele verilir. Belge bunu açıkça yazar: Claude bu dosyaları bağlam olarak görür, zorlanan yapılandırma olarak değil, ve sıkı uyumun garantisi yoktur. Hook ise modelin kararına hiç girmez. Olayı gelince çalışır, Claude ne düşünürse düşünsün.

KATMAN	KİM KARAR VERİR	NE ZAMAN YÜKLENİR	NE İÇİN
CLAUDE.md	Claude	Her oturumda	Her konuşmada geçerli bağlam: komutlar, tuzaklar, tercihler
`.claude/rules/`	Claude	Eşleşen dosyaya dokununca	Yalnız belli dosyalarda geçerli kural
Skill	Claude seçer, sen `/ad` ile zorlayabilirsin	Açıklama her zaman, gövde çağrılınca	Tekrarlanan prosedür
Alt ajan	Claude	Çağrılınca, kendi penceresinde	Yalıtılmış iş, sonuç özet döner
Hook (`command`, `http`)	Claude Code	Olayı gelince	Her seferinde olması gereken şey
Deny kuralı, yönetilen ayar	Claude Code	Her araç çağrısından önce	Hiçbir zaman olmaması gereken şey
Sandbox	İşletim sistemi	Her kabuk komutunda	Komut adından fazlasını yapsa bile tutan sınır

Belgenin hook ile skill karşılaştırmasındaki iki cümle bütün bölümü özetler: skill "bir rica, garanti değil". Korkulukları hook'a koy.

Karar kuralları

İstisnası olmayan şey kesin katmana gider. Belge bunu tek cümleyle söyler: bir şey her seferinde olmak zorundaysa onu hook, deny kuralı ya da yönetilen ayar yap. `.env` okunmayacaksa bu bir CLAUDE.md satırı değil, deny kuralıdır. Commit'ten önce testler koşacaksa bu bir hook'tur. 7.1 7.6

Geri kalan her şeyi Claude'a bırak. Belge burada da açık: çoğu zaman Claude'un karar vermesini istersin. Kırk hook'a ulaştıysan çoğu kural değil tercihtir. Kesin katman bakım ister, her hook test ister ve yanlış yazılmış bir hook işi tamamen durdurur. 8.4

Vurguyla uyum satın alınmaz. Bir kural çiğneniyorsa başına IMPORTANT eklemek belgenin deyişiyle büyük olasılıkla düzeltmez. Eski modeller vurguya ihtiyaç duyuyordu. Güncel modeller düz yazılmış bir satırı izler, vurgu yığmak kuralı fazla uygulatır. Vurgu eklemeye devam ediyorsan o satır hook'a aittir. 1.1

CLAUDE.md'yi kısa tut. Belge dosya başına 200 satırın altını hedefler. Uzun dosya hem bağlam harcar hem uyumu düşürür. Çelişen iki talimattan Claude birini rastgele seçebilir. Uzun prosedür skill'e, belli dosyalara özgü

kural `.claude/rules/` altına gider. 1.2 1.4

Yol kuralının bir bedeli var. `paths` deseniyle yüklenen kural konuşma geçmişine girer. Sıkıştırmada özetlenip kaybolabilir, ancak eşleşen dosyaya yeniden dokununca geri gelir. Oturum boyunca kesin geçerli olması gerekiyorsa `paths` alanını kaldır ya da kökteki CLAUDE.md'ye taşı.

Skill'in açıklaması tetiğidir. Claude işini skill açıklamalarıyla karşılaştırıp hangisinin ilgili olduğuna kendisi karar verir. Açıklama belirsizse ya da başka bir skill'le örtüşüyorsa yanlış skill yüklenir ya da hiç yüklenmez. Mutlaka çalışması gerekiyorsa `/ad` ile kendin çağır. Yan etkili bir skill'i otomatik çağrıya `disable-model-invocation: true` ile kapat. 9.3 9.4

Her hook kesin değildir. `command` ve `http` tipindeki hook kesindir. `prompt` ve `agent` tipindeki hook kararı başka bir modele bırakır, belgeye göre bunlar da kesin değildir. Hook'un kendisi çökerse eylemi de durdurmak istiyorsan hata politikasını buna göre seç. 8.5

Deny kuralı bir güvenlik sınırı değildir. Bash kuralları komutun metnini eşler, niyetini değil: `Bash(rm *)` kuralı `/bin/rm` biçimini durdurmaz. Gerçek sınır işletim sistemi düzeyindeki sandbox'tır ve o da yalnız kabuk komutlarını sarar. İkisinin ayrıntısı Bölüm 6: İzin ve güvenlik: mod, kural, sandbox bölümünde. 6.2 7.5

Bir kuralın yerini bulmak

Belgenin önerdiği tetikler kısa bir karar listesi verir:

GÖZLEM	YERİ
Claude aynı şeyi iki kez yanlış yaptı	CLAUDE.md
Aynı uzun prompt'u tekrar tekrar yazıyorsun	Skill
Bir yan iş konuşmayı çıktıyla boğuyor	Alt ajan
Bir şeyin sormadan her seferinde olması gerekiyor	Hook
Bir şeyin hiçbir zaman olmaması gerekiyor	Deny kuralı, gerekirse sandbox
Aynı kurulum ikinci bir depoda da lazım	Plugin

Hata biçimleri

Gerçek vaka: metin kuralı yeni oturuma ulaşmadı. Dört paralel oturum aynı anda derleme koştu, makinenin yükü 720'ye çıktı. Oturumlara "ağır komutu kilitle sar" diye talimat verildi. Bir oturum kilitsiz bir derleme başlattı ve sonradan durdurmak zorunda kaldı. Metin kuralı o anki oturumlara ulaştı, yenilerine ve başka projelere ulaşmadı. Çalışan çözüm kesin katmandaydı: ağır komutları kilitle koşturan bir sarmalayıcı ve sırasız ağır komutu reddeden bir PreToolUse hook'u. Hook 16 vakalık bir testle sınandı. Ayrıntısı Gerçek deneyimlerden on ders sayfasında D08.

Gerçek vaka: kesin sandığın katman hiç yüklenmedi. Proje dışı bir dosyayı import eden CLAUDE.md headless koşumda içeriği yüklemedi (D07, Bölüm 1: Zihinsel model: döngü, bağlam ve harness ve Bölüm 8: Hook ile garanti: olay, çıkış kodu, sınama). Çözüm olan SessionStart hook'u kuralı uygulatmadı, yalnız bağlama girmesini garanti etti. Kuralın kendisi yine Claude'un kararındaydı.

Kurgu senaryo: çoğu zaman tutan kural. Mert'in ekibi CLAUDE.md'ye "commit'ten önce tip kontrolünü çalıştır" yazar. Haftalarca sorun çıkmaz. Bir gün uzun bir oturumun sonunda Claude tip kontrolünü atlayıp commit atar, kırık kod ana dala girer. Belge tam bu satırı örnek verir: çoğu zaman çalışır, her zaman değil. Ekip kuralı bir hook'a taşır. Artık tip kontrolü geçmeyen commit hiç oluşmaz.

Alıştırma: aynı kuralı üç katmanda dene

Bunu gerçek bir iş deposunda değil, bu iş için açtığın bir deneme deposunda yap.

1. Depoya içinde sahte bir değer olan bir `.env` dosyası koy, ör. `DENEME_ANAHTARI=12345`. 2. **Rica katmanı.** CLAUDE.md'ye "`.env` dosyasını okuma" yaz. Yeni oturumda Claude'dan `.env` içindeki değeri söylemesini iste.

Ne olduğunu kaydet. 3. **Kesin katman.** CLAUDE.md satırını sil. `.claude/settings.json` içine `Read(./.env)` için deny kuralı ekle. Yeni oturumda aynı isteği yap. Sonra Claude'dan değeri `cat .env` ile okumasını iste. İkisinin sonucunu kaydet. 4. **Kuralın sınırı.** Claude'dan `.env` dosyasını okuyup değeri yazdıran küçük bir betik yazmasını ve çalıştırmasını iste. Deny kuralı bunu durdurdu mu? 5. Platformun destekliyorsa `/sandbox` ile sandbox'ı aç ve 4. adımı tekrarla.

Bitti sayılması için: dört satırlık bir tablo. Her satırda katman, deneme ve sonuç olsun. Altına bir cümle yaz: bu depoda `.env`'i gerçekten koruyan katman hangisi. Sonra deneme deposunu sil.

Bağlantılar

Kontrol listesi: 1.1 1.2 1.4 6.2 7.1 7.5 7.6 8.4 8.5 9.3 9.4. Önceki bölüm: Bölüm 4: Kanıtla teslim: iddiayı sayıya çevir. Ekler: İpuçları ve püf noktaları, Dosya konumları.

Rehber 6 · İzin ve güvenlik: mod, kural, sandbox

Orta · İzin modu, izin kuralı ve sandbox üç ayrı soruya cevap verir. Hangisinin neyi koruduğu, kuralın dar ve eşleşebilir yazılması ve gözetimsiz koşumda etkin modun doğrulanması.

Zihinsel model: üç ayrı soru, üç ayrı katman

İzin ve güvenlik tek bir konu gibi görünür ama üç ayrı soruya cevap verir. Her sorunun cevabı farklı bir katmandan gelir. Katmanları karıştırırsan yanlış yere kural yazarsın.

SORU	KATMAN	KİM UYGULAR
Claude bu aracı sormadan kullanabilir mi?	İzin modu ve izin kuralları	Claude Code
Çalışan komut makinede neye erişebilir?	Sandbox	İşletim sistemi
Okuduğu metin Claude'u yönlendirebilir mi?	Dış içeriğe güvenmeme alışkanlığı, onay noktaları	Sen ve auto modundaki sınıflandırıcı

İlk katmanı Bölüm 5: Talimat mimarisi: rica ile garanti içinde gördün: izin kuralını Claude Code uygular, model uygulamaz. Belge bunu açık yazar: prompt ve CLAUDE.md Claude'un ne denediğini şekillendirir, Claude Code'un neye izin verdiğini değiştirmez. Bu bölüm o katmanı derinleştirir ve diğer ikisini yanına koyar.

Kuralların nasıl bir araya geldiğini tek cümleyle hatırla: deny önce gelir, sonra ask, sonra allow. İlk eşleşen kazanır ve kuralın ne kadar özel olduğu sırayı değiştirmez. Geniş bir deny kuralı, dar bir allow kuralının açtığı kapıyı kapatır. Allow kuralıyla deny kuralından istisna çıkaramazsın. Bir PreToolUse hook'u "allow" dese bile deny ve ask kuralları yine değerlendirilir.

İzin modları: hangisi ne zaman

Mod, kural yazılmamış durumda Claude'un neyi sormadan yapabileceğini belirler. Kurallar modun üstüne biner. Belgedeki tabloyu kendi cümlelerimle özetliyorum:

MOD	SORMADAN NE YAPAR	NE ZAMAN SEÇ
`default` (arayüzde Manual)	Yalnız okuma	Hassas iş, tanımadığın kod
`acceptEdits`	Okuma, çalışma dizinindeki dosya düzenlemeleri, `mkdir`, `mv`, `cp` gibi basit dosya komutları	Sonucu `git diff` ile sonradan inceleyeceğin iş
`plan`	Okuma ve araştırma, düzenleme yok	Değiştirmeden önce depoyu anlamak
`auto`	Hemen her şey, ama arka planda bir sınıflandırıcı model her işlemi gözden geçirir	Yönünü güvendiğin uzun işler
`dontAsk`	Yalnız önceden izin verilenler, sorulacak her şey reddedilir	CI ve kilitli betikler
`bypassPermissions`	Hemen her şey, güvenlik kontrolleri kapalı	Yalnız yalıtılmış konteyner ya da sanal makine

Üç ayrıntı kararını değiştirir.

- **`auto` güvenlik garantisi değildir.** Belge uyarıyı kendisi yazar: istem sayısını azaltır, hassas işlemde incelemenin yerini tutmaz. Belge, `auto`'nun v2.1.283 ve sonrasında terminal ve VS Code için yerleşik başlangıç modu olduğunu söylüyor. Yerel sürüm 2.1.284 bu eşiğin üstünde. Yani oturum senin seçmediğin bir modda açılmış olabilir. Başlangıçta durum çubuğuna bak.

- **`dontAsk` kuralsız çalışmaz.** Hiçbir şey sormadığı için eksik allow girdisi sessiz ret demektir. CI'da bunu istersin, etkileşimli işte istemezsin.
- **`bypassPermissions` her şeyi açmaz.** Allow kuralları bu modda etkisizdir ama deny kuralları yine çalışır. `rm -rf /` gibi tanınan kritik silmeler ve açık ask kuralları yine sorar. Belge bu modu internetsiz konteyner ve sanal makine için ayırır.

Başlangıç modu şu sırayla belirlenir: komut satırı bayrağı, sonra ayar dosyasındaki `defaultMode`, sonra yerleşik varsayılan. Bir tuzak var. Belgeye göre `.claude/settings.json` ya da `.claude/settings.local.json` içine `bypassPermissions` yazarsan değer etkisizdir ve oturum Manual başlar. `claude -p` ile açılan oturum ise yerleşik olarak `default` ile başlar.

Dar allow girdisi

Kuralın genişliği riskini belirler. Üç davranışı bil.

Yıldızı alt komuttan sonra koy. `Bash(git log *)` yalnız `git log` komutlarını açar. `Bash(git *)` push, reset ve clean dahil bütün git'i açar. Belge, alt komuttan önce yıldız içeren allow kuralları için başlangıçta uyarı verdiğini söyler. 6.3

Bileşik komutta her parça ayrı değerlendirilir. `Bash(npm test *)` kuralı `npm test && curl ...` komutunun tamamını açmaz. `&&`, `||`, `|`, `&`, noktalı virgül ve satır sonu ayırıcıdır. Her parça ayrı izinli olmalı. Bu yüzden komutu tek parçada yazmak sorudan kurtarır. 6.4

Sarmalayıcıların bir kısmı atlanır, bir kısmı atlanmaz. `timeout`, `time`, `nice`, `nohup`, `stdbuf` ve bilinen güvenli birkaç ortam değişkeni atılır, yani `NODE_ENV=test npm test` yine `npm test` kuralına uyar. `npx`, `docker exec` ya da `devbox run` gibi ortam çalıştırıcıları atılmaz. `Bash(devbox run *)` yazarsan `devbox run rm -rf .` da geçer. Çözüm, çalıştırıcıyı ve iç komutu birlikte yazmaktır: `Bash(devbox run npm test)`.

Komutu motorun eşleştirebileceği biçimde yaz

İzin motoru komut metnini statik eşleştirir. Metnin neye çözüleceğini bilemediği bir parça görünce kuralı uygulamaz ve sorar. `$(...)` ve `${...}` genişletmeleri, `source venv/bin/activate` ile aktive edilen ortamlar, bir `&&` zincirinin içindeki kuralsız tek halka bu gruptandır. Çözüm yeni bir allow girdisi değil, komutu o parça olmadan yazmaktır: venv'i aktive etmek yerine yorumlayıcıyı doğrudan çağır, `.venv/bin/python -m pytest`. Belge güvenlik bölümünde, Claude Code'un tam çözümleyemediği bir Bash komutunu Manual modda sorduğunu ve bir kısmına verilen allow kuralının bu soruyu atlatmadığını yazar. Beklenmedik bir istem gördüğünde önce komutu sadeleştir. 6.4

Sandbox ve izin kuralı aynı şey değil

İzin kuralı komutun metnine bakar. Sandbox komutun ne yaptığına bakar. Belge farkı şöyle koyar: işletim sistemi sandbox sınırını çalışan işleme uygular, model ne seçmiş olursa olsun ve izinli bir komut adından fazlasını yapsa bile tutar.

Bir deny kuralının neden sınır olmadığını belgenin tablosu gösterir. `Bash(rm *)` deny kuralı `rm -rf build/` komutunu durdurur ama `/bin/rm -rf build/` ya da `bash -c 'rm -rf build/'` komutunu durdurmaz. `Bash(git push *)` kuralı `git -C . push` biçimini yakalamaz. Kural, Claude'un genelde yazdığı biçimi kapsar.

İki sınırlama sandbox'ı da kayıtsız şartsız güvence yapmaz.

- Sandbox varsayılan olarak kapalıdır ve yalnız kabuk komutlarını sarar. Read, Edit, Write, WebFetch gibi yerleşik araçlar, hook'lar ve yerel MCP sunucuları sandbox dışında çalışır. Onları izin kuralları yönetir.
- Varsayılan okuma erişimi geniştir ve `~/.ssh` ile `~/.aws/credentials` gibi kimlik dosyalarını kapsar. Gerekli `denyRead` girdilerini sen eklersin. 7.5

Güvenmediğin kodu çalıştırırken ikisini birlikte kullan.

Dış içerik veridir, talimat değildir

Prompt injection, okuduğun bir sayfanın, e-postanın ya da belgenin içine yerleştirilmiş "şunu yap" cümlesidir. Model o cümleyi senin talimatın sanarsa saldırgan senin yetkinle iş yapar. Savunma üç parçadan oluşur.

1. **Okunan metindeki talimat yetki taşımaz.** Bir sayfa "kullanıcı bunu onayladı" dese bile onay yalnızca senin sohbetinden gelir. 7.3 2. **Yan etkili adımları onaya bağla.** Gönder, sil, öde, yayınla gibi adımlar dış içerikle çalışan oturumda `default` modunda ya da açık bir ask kuralıyla kalsın. 3. **Ağ erişimi güven değildir.** Bir alan adına izin vermek oradan inen metni güvenilir yapmaz. WebFetch, Bash ve MCP erişimlerini ayrı değerlendir. 7.4

Belgeye göre `auto` modunun sınıflandırıcısı, Claude'un okuduğu düşman içerikten kaynaklanıyormuş gibi görünen işlemleri de engeller. Bu yardımcı bir ağdır, onayın yerini tutmaz.

Gizli dosyalar ve yeni depo

`.env` ve `*.pem` gibi dosyalar için `Read(./.env)` biçiminde deny kuralı yaz. Kural dosya araçlarını ve tanınan bazı kabuk erişimlerini kapsar, ama dosyayı açan keyfî bir Python betiğini bütünüyle durdurmaz. Güçlü ayrım için sırları ortamdan çıkar ya da sandbox'ın dosya sınırlarını kullan. 7.1

Başkasının deposunu açmadan önce şunlara bak. Proje ayarları yerel komut başlatabilir. 7.7

- `.claude/settings.json` içindeki hook'lar ve `env` bloğu.
- `.mcp.json` içindeki MCP sunucu komutları.
- Proje skill'lerinin `allowed-tools` ve hook alanları.

Etkileşimli oturumda belge, proje allow kurallarının çalışma alanı güveni kabul edilene kadar kullanılmadığını söyler. `claude -p` ise güven penceresini hiç göstermez. Orada depodaki hook'lar çalışır, `.mcp.json` sunucuları sormadan bağlanır. Tanımadığın depoda `claude -p` koşmadan önce `--bare` ya da `--setting-sources user` kullan.

Gözetimsiz koşumda etkin modu doğrula

Kayıtlı vaka D01: zamanlanmış oturumların izinsiz ilerlemesi için `bypassPermissions` ayarlanmıştı. Bir projenin 43 oturum kaydındaki 345 mod değeri tarandı ve bu mod hiç görünmedi. Oturum hata vermeden varsayılan moda düşmüştü. Ders, ayarladığın modla oturumun açıldığı modun ayrı şeyler olması. Ayırıcı kanıt, oturum kaydındaki etkin mod değeridir, ayar dosyasındaki değer kanıt sayılmaz. Belge bu vakanın nedenini anlatmaz ama iki ilgili gerçeği yazar. Proje ayar dosyalarına yazılan `bypassPermissions` etkisizdir. `-p` oturumu güven penceresini hiç göstermediği için, klasör daha önce güvenilmediyse projenin allow kuralları da kullanılmaz. Hook girdisinde `permission_mode` alanı da bulunur, istersen bir hook modu log'a yazsın.

Gözetimsiz iş kurarken sırayla şunu yap: modu bayrakla ver, etkin modu kayıttan oku, allow listesini zararsız bir komutla dene. Çalışmayan adımı sessizce geçme, nedenini yaz. 6.7

Karar kuralları

- **Hassas işte `default`, kendi gözden geçireceğin işte `acceptEdits`, yönünü güvendiğin uzun işte `auto`.** `bypassPermissions` yalnız yalıtılmış kutuda.
- **Tekrar eden istemi azaltmak için önce kaydı oku.** `/fewer-permission-prompts` transcript'lerinden salt okunur komut önerir. Her girdiyi tek tek oku. Yazma ya da yayın komutu içerenleri alma. 6.6
- **Yıkıcı komutu deny'a koy ama sınır sanma.** Gerçek sınır gerekiyorsa sandbox ya da komut içeriğine bakan bir hook ekle. 6.2
- **Allow kuralını alt komut düzeyinde yaz, yıldızı sona koy.** 6.3
- **Bir komut allow'da olduğu halde soruyorsa içinde genişletme, aktivasyon ya da zincir ara.** 6.4
- **Mod adını güvenlik kanıtı sayma.** Zararsız bir deneme komutuyla gerçek davranışı gör. 6.7

Hata biçimleri

Gerçek vaka: ayarlanan mod, oturumun modu değildi. D01 yukarıda. Ayrıntısı Gerçek deneyimlerden on ders sayfasında.

Kurgu senaryo: geniş joker. Deniz'in ekibi tekrar eden istemden bıkıp `Bash(git *)` ekler. Birkaç hafta sonra Claude bir birleştirme çözerken `git clean -fd` komutunu sormadan çalıştırır ve izlenmemiş dosyalar silinir. Kuralın amacı `git status`, `git diff` ve `git commit` idi. Dar liste bunları tek tek sayardı ve yıkıcı komutlar sorulurdu.

Alıştırma: bir haftalık kullanımdan dar bir allow listesi

Herhangi bir git deposunda yap. Yazma ya da silme yok, yalnız ayar dosyası değişir.

1. Bir hafta boyunca normal çalış. `/fewer-permission-prompts` oturum kayıtlarını tarar, ayrıca not tutman gerekmez. 2. Bir oturumda `/fewer-permission-prompts` çalıştır. Önerdiği listeyi dosyaya yazdırmadan önce ekrana getirt. 3. Her girdi için üç sütunlu bir tablo yap: girdi, neden güvenli, neyi yanlışlıkla açar. Örneğin `Bash(git status *)` için "yalnız okur", `Bash(npm run *)` için "package.json'daki her betiği açar, betikler incelenmiş mi?" 4. Gerekçesini yazamadığın girdiyi at. Yazma ya da yayın yapan girdiyi (`git push`, `deploy`) at. 5. Kalan listeye bir de `deny` bölümü ekle: en az `Read(./.env)` ve `Bash(git push *)`. 6. Listeyi `.claude/settings.json` içine koy. Yeni oturumda biri listede biri listede olmayan iki zararsız komutla dene ve beklenen davranışı gör.

Bitti sayılması için: her girdinin gerekçesini taşıyan tablo, atılan girdilerin listesi ve tek bir cümle: bu liste neyi bilerek açmıyor. İki denemenin sonucunu da ekle.

Bağlantılar

Kontrol listesi: 6.1 6.2 6.3 6.4 6.5 6.6 6.7 7.1 7.3 7.4 7.5 7.7. Önceki bölüm: Bölüm 5: Talimat mimarisi: rica ile garanti. Gerçek vaka: D01, Gerçek deneyimlerden on ders. Ekler: Komutlar ve kısayollar, Sorun giderme.

Rehber 7 · Yeniden kullanım: skill, alt ajan, MCP, plugin

Orta · Skill, alt ajan, MCP ve plugin'in her biri farklı bir sorunu çözer. Hangisinin ne zaman seçileceği, skill açıklamasının neden tetik olduğu ve MCP'nin gerçek bir okumayla sınanması.

Zihinsel model: aynı işi üçüncü kez anlatma

Bir işi üçüncü kez aynı cümlelerle anlatıyorsan o tarif bir dosyaya aittir. Claude Code bu dosya için dört kap sunar ve her kap farklı bir sorunu çözer. Sorunu yanlış kaba koyarsan ya bağlamını şişirirsin ya da hiç tetiklenmeyen bir şey yazarsın.

KAP	ÇÖZDÜĞÜ SORUN	NEREDE ÇALIŞIR
Skill	Tekrar eden tarif ya da başvuru bilgisi	Ana konuşmada, bağlama eklenir
Alt ajan	Konuşmayı boğan yan iş, bağımsız göz	Kendi bağlam penceresinde, sana özet döner
MCP	Claude'un göremediği bir dış sistem	Dış sunucuda, Claude'a araç olarak görünür
Plugin	Aynı kurulum başka depoda ya da başka kişide lazım	Paketleme katmanı, içindekileri birlikte taşır

Belgenin "Extend Claude Code" sayfası seçimi tetiklere bağlar. Tetikler kısa:

GÖZLEM	EKLENECEK ŞEY
Aynı başlangıç prompt'unu tekrar tekrar yazıyorsun	Skill (sen `/ad` ile çağırırsın)
Aynı çok adımlı tarifi üçüncü kez sohbete yapıştırıyorsun	Skill
Bir tarayıcı sekmesinden ya da bir sistemden sürekli veri kopyalıyorsun	MCP
Yan iş konuşmayı çıktıyla dolduruyor	Alt ajan
Bir şeyin sormadan her seferinde olması gerekiyor	Hook (Bölüm 8: Hook ile garanti: olay, çıkış kodu, sınama)
İkinci bir depo aynı kuruluma ihtiyaç duyuyor	Plugin

Skill ile alt ajan arasındaki fark bağlama bakarak anlaşılır. Skill konuşma penceresine eklenir. Alt ajan kendi penceresinde çalışır ve yalnız özet döndürür. İkisi birleşebilir: bir alt ajan skill'leri önceden yükleyebilir, bir skill de `context: fork` ile kendi alt ajan bağlamında koşabilir. Bölüm 5: Talimat mimarisi: rica ile garanti rica ile garanti ayrımını anlattı. Bu bölüm kapların kendisini anlatıyor.

Skill: açıklama bir tetiktir

Skill dosyasının başındaki `description`, Claude'un skill'i ne zaman yükleyeceğine karar verdiği tek şeydir. Gövde ancak çağrılınca yüklenir. Belge ayrıntıyı şöyle koyar: açıklama ve `when_to_use` metni listede toplam 1.536 karakterde kesilir, bu yüzden ana kullanım durumunu başa yaz.

İyi bir açıklama üç şey taşır.

- **Ne yaptığı:** "Son etiketten bu yana değişikliklerden sürüm notu yazar."
- **Kullanıcının söyleyeceği ifadeler:** "Sürüm notu", "release notes" denince kullan.
- **Ne zaman kullanılmayacağı:** "Teknik changelog için kullanma." Bu cümle komşu işlerde yanlış yüklenmeyi keser. 9.3

Belge sorun giderme bölümünde iki yönü de ayrı ayrı ele alır. Skill tetiklenmiyorsa açıklamaya kullanıcının doğal olarak söyleyeceği sözcükleri koy ve `/ad` ile doğrudan çağır. Çok sık tetikleniyorsa açıklamayı daralt. Frontmatter

YAML'ı bozuksa skill gövdesi boş metadata ile yüklenir: `/ad` çalışır ama Claude açıklamayı eşleştiremez. Bu yüzden "elle çağırınca çalışıyor" tek başına tetik kanıtı değildir.

Yan etkili skill'i kapat

Mesaj gönderen, deploy eden ya da dosya silen bir skill'in modelin kararıyla çalışmasını istemezsin. Frontmatter'a `disable-model-invocation: true` yaz. Belgeye göre bu durumda skill'in açıklaması bağlama hiç girmez, Claude kendiliğinden çalıştıramaz ve denerse Claude Code çağrıyı reddeder. Ama skill'i sen `/ad` ile çalıştırabilirsin. Bu alan araç izinlerinin ya da senin yetkinin yerini tutmaz, yalnız kimin tetikleyebileceğini belirler. 9.4

Skill'in `allowed-tools` alanı da benzer bir karışıklık yaratır. Bu alan skill çalıştığı tur boyunca listedeki araçlara sormadan izin verir, kısıtlamaz. Bir skill'e araç kısıtı koymak istiyorsan deny kuralı ya da alt ajanın `tools` alanı gerekir.

Son bir tuzak: skill çağrıldığında içeriği konuşmaya girer ve orada kalır. Dosya sonraki turlarda yeniden okunmaz. Sıkıştırmadan sonra her skill'in yalnız başı geri eklenir (ilk 5.000 token) ve çok skill çağırdıysan eskileri tamamen düşebilir. Önemli kuralı skill dosyasının başına koy. Bir kural her seferinde geçerli olmak zorundaysa belgenin önerisi hook'tur.

Alt ajan: ayrı bağlam, ayrı model

Alt ajan kendi bağlam penceresinde, kendi sistem prompt'u ve kendi araç listesiyle çalışır. Konuşmanı görmez, ona görevi bir devir mesajı anlatır. Üç karar vereceksin.

Ne zaman alt ajan? Belgenin ölçütü: görev çok çıktı üretiyor ama o çıktıya sonra ihtiyacın yok, ya da araç kısıtı koymak istiyorsun, ya da iş kendi içinde bütün ve özetle dönebilir. Sık gidip gelen, aşamaları ortak bağlama dayanan ya da küçük bir değişiklik olan işte ana konuşma daha iyidir, çünkü alt ajan sıfırdan başlar ve bağlam toplamak zaman alır.

Hangi model? Alt ajanın `model` alanı `sonnet`, `opus`, `haiku`, `fable`, tam bir model kimliği ya da `inherit` alabilir. Belge çözümleme sırasını verir: çağrı başına verilen `model` parametresi, sonra tanımdaki `model`, sonra `CLAUDE_CODE_SUBAGENT_MODEL` ortam değişkeni, en son ana konuşmanın modeli. Hata maliyeti düşük bir arama işini küçük modele vermek tasarruf sağlar. Hata maliyeti yüksek bir doğrulamayı küçük modele vermek ise bedelini sonradan ödetir. Aynı temsilî işte kalite, süre ve kullanımı karşılaştırmadan karar verme. 5.1

Eleştiri kimde? Yazan bağlam kendi hatasına kör kalır, çünkü varsayımlarını kendisi koymuştur. Eleştiriyi ayrı bir alt ajana ver. İncelemeyi statik tutmak istiyorsan ajanın `tools` alanına yalnız `Read, Grep, Glob` yaz. `Bash` eklediğin ajan salt okunur sayılmaz, çünkü komut çalıştırabilir. Ayrı bağlam bağımsızlığı artırır, doğruluğu garanti etmez. Ajanın cevabı bulgu sayısı değil, dosya ve kanıt taşımalı. Bulgu yoksa neye baktığını ve neyi çalıştırmadığını yazmalı. 9.1

Alt ajan açıklamaları da bağlama girer. Belge, özel ajanların açıklamalarının toplamı 15.000 token'ı aşınca başlangıçta uyarı verir. Açıklamayı kısa tut, ayrıntıyı ajanın sistem prompt'una koy.

MCP: bağlı görünmek çalışmak demek değildir

MCP, Claude'u dış bir sisteme araç olarak bağlar. Bir uygulamaya erişeceksen önce o uygulamanın MCP'sini dene, çünkü API üzerinden giden çağrı ekranda tıklamaktan hızlı ve kesindir. Sıra: uygulamanın MCP'si, sonra tarayıcı, en son bilgisayar kontrolü. 9.5

Bağlantıyı iki yerde sına.

1. `/mcp` ile sunucu durumuna bak. `claude mcp list` komutu `Connected`, `Needs authentication` ya da `Failed to connect` gösterir. Bu yalnız bağlantıyı söyler. 2. Zararsız gerçek bir okuma işlemi yaptır: bir kaydı listele, bir kaydı oku. İşlem doğru hesapta ve doğru hedefte sonuç veriyorsa bağlantı çalışıyordur. Yazma işlemini ayrıca, kendi yetki kapsamınla eşleştirerek dene.

Kapsamı dar tut. Belge MCP sunucusunu üç kapsamda tanımlar: yerel (yalnız bu proje, yalnız sen), proje (`.mcp.json`, depoyla paylaşılır) ve kullanıcı (bütün projelerin, yalnız senin). Kişisel ve kimlik bilgisi taşıyan sunucuyu yerel kapsamda tut. Ekiple paylaşılan sunucuyu proje kapsamına koy ama içeriğini kod gibi gözden

geçir. Çalışan bir bağlantı doğru hesap ve en az yetki demek değildir. Okuma ve yazma yetkisini ayır, kullanmadığın bağlantıyı kaldır. 9.7

Proje kapsamlı sunucular için güvenlik tarafı: etkileşimli oturum onay ister. `claude -p` onay penceresini gösteremediği için `.mcp.json` sunucularını sormadan yükler. Başkasının deposunda `-p` koşmadan önce Bölüm 6: İzin ve güvenlik: mod, kural, sandbox bölümündeki önlemleri uygula.

Plugin: paketle, ama gerektiğinde

Plugin skill, ajan, hook ve MCP sunucusunu tek kurulum birimi yapar. Manifest `.claude-plugin/plugin.json` içindedir. Plugin skill'leri `/plugin-adi:skill` biçiminde ad alanı kazanır, böylece çakışmaz. Belgenin söylediği önemli bir ayrıntı: etkin plugin her oturumun parçasıdır, kullanmasan da bağlamına ve kullanımına yük bindirir. Tek bir depoda ve tek kişilik iş için dosyaları `.claude/skills/` altında tutmak yeter. İkinci bir depo ya da ekip arkadaşı gerektirince paketle. 9.6 10.3

Temiz bir makinede kurulumdan sonra skill'in listede göründüğünü doğrula. Ekip için tek kişinin ürettiği çıktıyla başkasının ürettiğini karşılaştır, biçim aynı olmalı.

Karar kuralları

- **Tarifi üçüncü kez yazıyorsan skill yap.** İkinci kez yazıyorsan yalnız not al.
- **Açıklamayı kullanıcının söyleyeceği sözcüklerle ve "kullanma" cümlesiyle yaz.**
- **Yan etkisi olan her skill'e `disable-model-invocation: true` koy.**
- **Çok çıktı üreten yan işi alt ajana, eleştiriyi yazandan ayrı bağlama ver.**
- **Alt ajana `Bash` verdiysen salt okunur saymayı bırak.**
- **MCP için `Connected` yeterli değil. Gerçek bir okuma yaptır.**
- **Paketlemeyi ikinci kullanıcı gelince yap, öncesinde değil.**

Hata biçimleri

Kurgu senaryo: tek örnekle sınanan skill. Zeynep `surum-notu` skill'ini yazar, bir kez çalıştırır, sonuç güzel çıkar. İki hafta sonra ekip arkadaşı "teknik changelog hazırla" der ve Claude yanlışlıkla bu skill'i yükler. Açıklamada "kullanma" sınırı yoktur. Zeynep bir olumsuz örnek (teknik changelog) ve bir hata örneği (hiç etiket olmayan depo) ekleyip yeniden sınar.

Kurgu senaryo: bağlı ama yanlış hesap. Hakan bir proje yönetimi MCP'si ekler, `/mcp` yeşil gösterir. İlk okuma işlemi başka bir çalışma alanının kayıtlarını döndürür, çünkü sunucu kişisel hesapla giriş yapmıştır. Durum ekranı yalnız bağlantıya bakar, hesabı sınamaz.

Alıştırma: bir skill yaz ve olumlu, olumsuz örnekle sına

Herhangi bir git deposunda yap. Skill'i `.claude/skills/` altına koy.

1. Tekrar eden küçük bir iş seç. Örnek: son etiketten bu yana değişenlerden en fazla beş maddelik kullanıcıya dönük sürüm notu. 2. `.claude/skills/surum-notu/SKILL.md` dosyasını yaz. Açıklamada ne yaptığını, tetikleyici ifadeleri ve "teknik changelog için kullanma" sınırını belirt. 3. Sınama listesini önce yaz, sonra çalıştır. En az altı satır olsun: üç istek tetiklemeli (örn. "sürüm notu yaz", "release notes hazırla", "son sürümde ne değişti, kullanıcıya anlat"), iki istek tetiklememeli (örn. "teknik changelog üret", "bu dosyayı özetle"), bir tanesi hata durumu olmalı (örn. etiketi olmayan depo). Her satıra beklediğin sonucu yaz. 4. Her satırı **yeni bir oturumda** çalıştır. Önceki sohbet kalıntısı eksik yönergeleri maskeler. Hem doğal dille hem `/surum-notu` ile dene. Hangi isteğin skill'i yüklediğini kaydet. 5. Tetiklenmemesi gerekirken tetiklenen ya da tetiklenmesi gerekirken tetiklenmeyen satırlar için açıklamayı düzelt ve listeyi baştan koş. İstersen skill'i `skillOverrides` ile kapatıp aynı istekleri skill'siz çalıştırarak bir karşılaştırma tabanı al.

Bitti sayılması için: beklenen ve gerçekleşen sonucun yan yana durduğu altı satırlık bir tablo. Başarısız olan her satır için açıklamada neyi değiştirdiğini yaz. Başarısız vakaları sakla, bir sonraki değişiklikte tekrar koşarsın.

Bağlantılar

Kontrol listesi: 9.1 9.2 9.3 9.4 9.5 9.6 9.7 9.8 10.3 5.1. Önceki bölümler: Bölüm 5: Talimat mimarisi: rica ile garanti, Bölüm 6: İzin ve güvenlik: mod, kural, sandbox. Sonraki bölüm: Bölüm 8: Hook ile garanti: olay, çıkış kodu, sınama. Ekler: Komutlar ve kısayollar, Dosya konumları.

Rehber 8 · Hook ile garanti: olay, çıkış kodu, sınama

Orta · Hook, modelin kararına girmeden olay gelince çalışan koddur. Çıkış koduna göre neyi engelleyip neyi engellemediği, hook tiplerinin farkı ve hook'u bozarak sınamak.

Zihinsel model: olay gelir, betik karar verir

Bölüm 5: Talimat mimarisi: rica ile garanti rica ile garanti ayrımını kurdu. Hook, garanti tarafının en esnek aracıdır. Claude Code yaşam döngüsünde belirli noktalara geldiğinde senin betiğini, HTTP isteğini ya da küçük bir model çağrısını çalıştırır. Betik olay hakkında JSON girdisi alır. Çıktısıyla işlemi durdurur, bağlama bilgi ekler ya da sessiz kalır. Model araya girmez, çünkü Claude ne düşünürse düşünsün olay geldiği anda hook çalışır ve kararını kendi başına verir.

Hook'u anlamak için üç şeyi bil: ne zaman çalıştığı, neye bakarak seçildiği ve çıkışının neyi etkilediği.

Ne zaman çalışır? Olaylar üç tempoda gelir. Oturum başına: `SessionStart`, `SessionEnd`. Tur başına: `UserPromptSubmit`, `Stop`. Araç çağrısı başına: `PreToolUse` (araç çalışmadan önce, durdurabilir) ve `PostToolUse` (araç çalıştıktan sonra, geri alamaz). Bunların yanında `PermissionRequest`, `SubagentStop`, `PreCompact`, `Notification` gibi daha özel olaylar vardır. Tam liste Komutlar ve kısayollar ekinde ve belgenin hook referansında durur.

Neye bakarak seçilir? `matcher` alanı, olayın ilgili alanına göre hook'u süzer. Araç olaylarında araç adıdır: `Bash`, `Edit|Write`, `mcp__.*`. `SessionStart` için oturumun nasıl başladığıdır: `startup`, `resume`, `clear`, `compact`. Claude Code, yalnız harf, rakam, alt çizgi, tire, virgül ve `|` içeren değeri tam eşleşme listesi sayar. Başka bir karakter girerse JavaScript düzenli ifadesi olur ve değerin herhangi bir yerinde eşleşir. `Edit.*` hem `Edit` hem `NotebookEdit` ile eşleşir, tam eşleşme için `^Edit$` yaz.

Bir de `if` alanı var. İzin kuralı sözdizimiyle yazılır, örneğin `Bash(git *)`. Hook yalnız çağrı bu kalıba uyarsa çalışır, böylece her komutta betik başlatmazsın. Belge iki sınır koyar. Claude Code `if` alanını yalnız araç olaylarında (`PreToolUse`, `PostToolUse`, `PostToolUseFailure`, `PermissionRequest`, `PermissionDenied`) değerlendirir. Diğer olaylarda `if` olan hook hiç çalışmaz. İkincisi, `if` en iyi çabayla süzer: Claude Code komutun neyi çalıştıracağını çözemezse hook'u yine de çalıştırır. Kesin izin ya da ret için hook'a değil izin sistemine güven.

Çıkış kodu ve JSON karar

Hook'un sonucunu iki yoldan bildirebilirsin. Birini seç, karıştırma.

ÇIKIŞ	ANLAMI
0, çıktı yok	Karar yok. Normal izin akışı sürer. Sessiz kalmak onay vermek değildir
2	Engelleyebilen olayda işlemi durdurur. Stderr gerekçe olur ve Claude'a gider
Başka kod, başlatılamama, bozuk JSON	Engellemeyen hata. İşlem devam eder, yalnız bir bildirim görürsün

Bu tablonun son satırı bölümün en önemli cümlesidir. Betiğin çökerse, yolu yanlışsa ya da bozuk JSON basarsa Claude Code işlemi yine yapar. Güvenlik kapısı olan bir hook'ta bunu istemezsin. İki çözüm var: kapalı hata yönünü betikte kendin yaz (hata yakalayınca `exit 2`) ya da hook'a `"onFailure": "block"` ekle. İkincisi komut ve HTTP hook'ları için geçerli ve belgede v2.1.295 ve sonrasını gerektirir. Yerel sürüm 2.1.284 olduğundan bu makinede alan etkisiz kalır. Eski sürümde yokmuş gibi davran ve çözümü betiğe yaz.

Exit 2 ne yapar, olaya göre değişir. `PreToolUse` araç çağrısını durdurur. `UserPromptSubmit` istemi Claude'a ulaşmadan reddeder. `Stop` Claude'un durmasını engeller ve konuşmayı sürdürür. `PostToolUse` ise geri alamaz, çünkü araç çalışmıştır, yalnız stderr'i Claude'a gösterir. `SessionStart` stderr'i yalnız sana gösterir. Bir olayı engelleyici saymadan önce belgedeki "Exit code 2 behavior per event" tablosuna bak. 8.4

JSON yoluyla daha ince karar verirsin. Exit 0 ile tek bir JSON nesnesi yazdırırsın. `PreToolUse` için karar `hookSpecificOutput` içinde olur: `permissionDecision` değeri `allow`, `deny`, `ask` ya da `defer`, yanında

`permissionDecisionReason`. Birden çok hook farklı karar verirse öncelik `deny`, `defer`, `ask`, `allow` sırasıdır. Stdout'ta JSON dışında tek satır metin (kabuk profilinin bastığı bir karşılama satırı gibi) nesnenin okunmasını bozar.

Hook kararı izin kurallarını delmez. Belgeye göre bir hook `allow` dese bile Claude Code eşleşen deny ve ask kurallarını değerlendirir. Ters yönde exit 2 ile bloklayan hook, izin kuralları değerlendirilmeden çağrıyı durdurur. Bu yüzden "Bash'i tamamen aç, ama şu üç komutu engelle" düzeni çalışır: `Bash` allow'da, engelleyen hook yanında. 7.6

Hook'un gücü: hangi tip kesin?

Beş hook tipi var: `command`, `http`, `mcp_tool`, `prompt`, `agent`. Bölüm 5: Talimat mimarisi: rica ile garanti içinde gördüğün gibi `command` ve `http` kesindir, kod çalışır ve ne yazdıysan onu yapar. `prompt` ve `agent` tipi kararı başka bir modele bırakır. Belge prompt hook'unu, hook girdisini bir Claude modeline verip karar JSON'u alan tek turluk değerlendirme olarak anlatır. Karar bir modelden geldiği için belgenin iyi uygulamalar sayfası bu tipi de kesin saymaz. Garanti gereken yerde `command` kullan. `SessionStart` zaten yalnız `command` ve `mcp_tool` kabul eder.

Aynı olaya eşleşen hook'lar paralel çalışır. Bunun sonucu şu: birinin diğerinin çıktısına dayandığı düzen bazen tutar, bazen tutmaz, bu yüzden sıraya bağlı adımları tek betikte birleştir ve paralel hook'lar aynı dosyaya yazıyorsa onları ayrı dosyalara yazdır. Aynı tanımı birden çok ayar dosyasında yazarsan yalnız bir kez çalışır. 8.7

async: bekletmeyen, durduramayan

`async: true` hook'u arka plana atar. Claude beklemez ama durdurulamaz da: `decision`, `permissionDecision` ve `continue` alanları async hook'ta etkisizdir. `asyncRewake` hook exit 2 verince Claude'u uyarır ama işlemi engellemez, çünkü işlem çoktan bitmiştir. `claude -p` oturumu bitince Claude Code hâlâ çalışan async hook'u iptal eder. Kural basit: engellemesi gereken hook'u hiç async yapma, bildirim, log ve yavaş biçimlendirme gibi işleri ise rahatça async'e ver. 8.6

Hook'u sına, sonra bozarak düştüğünü gör

Hook hiçbir şeye bakmasa da sessizce "çalışır". `/hooks` yapılandırmayı gösterir, doğru çalıştığını göstermez. Sınama için olayın JSON girdisini elle ver ve çıkış kodunu oku. Üç vaka yeter: izinli, yasak, bozuk girdi. Sonra hook'u bilerek boz ve testin düştüğünü gör. 8.4

Hata biçimleri

Gerçek vaka D08: metin kuralı ulaşmadı, hook ulaştı. Bölüm 5: Talimat mimarisi: rica ile garanti içinde anlatılan vakada çalışan yol, sırasız ağır komutu reddeden bir PreToolUse hook'uydu. Hook 16 vakalık testle sınandı ve sıfır yanlış verdi. Hafif komut istisnası bilerek bozulunca iki yanlış verdi, yani test gerçekten bakıyordu. Tek yuvanın fazla dar çıktığı da kaydedildi. Ayrıntısı Gerçek deneyimlerden on ders içinde.

Gerçek vaka D07: hook kuralı uygulatmadı, bağlama girmesini garanti etti. Proje dışı import headless koşumda yüklenmedi (Bölüm 1: Zihinsel model: döngü, bağlam ve harness). Aynı içeriği her oturumda bağlama koyan bir SessionStart hook'u çözümü oldu ve model kuralı kelimesi kelimesine aktardı. Hook burada kuralı zorlamadı, sadece Claude'un gördüğü şeyi garanti etti. Doğrulama: dosya okuma araçları kapalı bir headless oturumda "bu kural ne diyor?" diye sormak. 8.1

Kurgu senaryo: sessizce açık bırakan kapı. Cem'in ekibi `.claude/hooks/kontrol.js` betiğini bir `PreToolUse` hook'una bağlar. Bir refactor sırasında biri betiği başka klasöre taşır. Hook artık başlamaz, bu engellemeyen bir hatadır, yalnız transkriptte bir uyarı çıkar. İki hafta boyunca kontrol çalışmaz ve kimse fark etmez. Ekip `onFailure: "block"` alanını ekler ve sürümü v2.1.295 üstüne çıkarır, ayrıca testi CI'a bağlar: betik yoksa test düşer.

Karar kuralları

- **Her seferinde olması gerekiyorsa `command` ya da `http` hook'u yaz.** Garanti `prompt` ve `agent` tipinde yok.
- **Engellemesi gereken hook senkron olsun.** Async yalnız bildirim ve log için.

- **Hata yönünü seç.** Bilgi veren hook hatada devam edebilir. Güvenlik kapısı hatada engellemeli: betikte `exit 2`, destekleyen sürümde `onFailure: "block"`.
- **`if` ile süzmek hızlandırır, güvenlik sağlamaz.** Kesin ret için deny kuralı.
- **Sıraya güvenme.** Bağımlı adımlar tek betikte.
- **Hook sayısı arttıkça sor:** bu bir kural mı, tercih mi? Kırk hook'a ulaştıysan çoğu tercihtir.

Alıştırma: engelleyen bir PreToolUse hook'u yaz ve üç girdiyle sına

Bunu deneme deposunda yap. Gerçek bir iş deposunda hook'u etkinleştirme.

1. `jq` kurulu olduğundan emin ol (`jq --version`). Deneme deposunda `.claude/hooks/yasak-komut.sh` dosyasını oluştur.

```
#!/bin/sh
# PreToolUse: Bash komutunda yasak kalıp varsa engelle
komut=$(jq -er '.tool_input.command | select(type == "string")') || {
  echo "Geçersiz hook girdisi" >&2
  exit 2
}
if printf '%s' "$komut" | grep -qE 'DROP TABLE|--force'
then
  echo "Politika engelledi: yasak kalıp" >&2
  exit 2
fi
exit 0
```

2. `chmod +x .claude/hooks/yasak-komut.sh` ile çalıştırılabilir yap. 3. Üç girdiyi elle ver ve çıkış kodunu yazdır: - İzinli: `printf '%s' '{"tool_input":{"command":"git status"}}' | .claude/hooks/yasak-komut.sh`, ardından `echo $?`. Beklenen: 0. - Yasak: aynı komut, ama `git push --force`. Beklenen: 2 ve stderr'de gerekçe. - Bozuk: `printf '%s' '{bozuk' | .claude/hooks/yasak-komut.sh`, ardından `echo $?`. Beklenen: 2. 4. Betiği bilerek boz. `exit 2` satırlarından birini `exit 0` yap (önce yasak-kalıp dalını, sonra bozuk-girdi dalını). Üç girdiyi yeniden ver. İlgili satırın artık 0 döndüğünü ve testinin düştüğünü gör. Betiği geri al. 5. Hook'u `.claude/settings.json` içine bağla: `PreToolUse`, `matcher: "Bash"`, tip `command`, yol `${CLAUDE_PROJECT_DIR}/.claude/hooks/yasak-komut.sh`. Yeni oturumda Claude'dan zararsız bir komut ve `echo "--force"` içeren bir komut çalıştırmasını iste. İkincisinin engellendiğini ve Claude'un gerekçeyi okuduğunu gör.

Bitti sayılması için: üç satırlık bir tablo (girdi, beklenen kod, gerçek kod), bozulmuş sürümde düşen satırın kaydı ve gerçek oturumdaki engelin ekran görüntüsü ya da transkript alıntısı. Sonra deneme deposunu sil.

Bağlantılar

Kontrol listesi: 8.1 8.2 8.3 8.4 8.5 8.6 8.7 7.6. Gerçek vakalar: D07, D08, Gerçek deneyimlerden on ders. Önceki bölümler: Bölüm 5: Talimat mimarisi: rica ile garanti, Bölüm 6: İzin ve güvenlik: mod, kural, sandbox, Bölüm 7: Yeniden kullanım: skill, alt ajan, MCP, plugin. Ekler: Komutlar ve kısayollar, Sorun giderme.

Rehber 9 · Paralel işler: bağımsızlığı ölçmek

Uzman · Paralel çalışma yalnız sonuçları ve kaynakları gerçekten bağımsız işlerde kazandırır. Worktree dosya ve index çakışmasını çözer, port, tarayıcı, pano, simülatör ve makine yükü ortak kalır.

Zihinsel model: paralel olmak bağımsız olmak demek değildir

Aynı anda üç iş başlatmak, işi üç kat hızlandırmaz. Hızlanma yalnız işler gerçekten birbirinden bağımsızsa gelir. Bağımsızlık da işin başlığına bakarak değil, sonucuna bakarak ölçülür. Kendine şunu sor: A işinin sonucu, B işinin ne yapacağını değiştirir mi? Değiştirmiyorsa iş paralel olabilir. Değiştiriyorsa ikisini aynı anda başlatmak yalnız çakışma üretir.

Ölçü iki katmanlıdır. Önce **sonuç bağımsızlığı**: iki işin çıktısı aynı dosyaya, aynı karara ya da birbirinin girdisine dokunuyor mu? Sonra **kaynak bağımsızlığı**: iki iş aynı git index'ini, aynı portu, aynı tarayıcı oturumunu, aynı panoyu ya da aynı simülatörü kullanıyor mu? Sonuçları bağımsız iki iş, ortak bir kaynak yüzünden yine de birbirini bozabilir.

Üç paralellik biçimi

Claude Code'da paralel çalışmanın üç yolu var. Farkları bağlam, dosya sistemi ve bedel üzerinden okunur.

BİÇİM	BAĞLAM	DOSYALAR	EN UYGUN İŞ
Alt ajan	Kendi penceresi, sana yalnız özet döner	Varsayılan olarak ana oturumla aynı dizin	Okuma ağırlıklı geniş araştırma
Ayrı oturum	Tamamen ayrı konuşma	Aynı checkout, istersen ayrı worktree	Uzun, birbirinden kopuk işler
Worktree	Oturumdan bağımsız bir dosya kopyası	Kendi dalı, kendi index'i	Aynı depoya yazan paralel işler

Üçü birbirini dışlamaz. Alt ajan ya da ayrı oturum, worktree içinde de çalışabilir. Belgeye göre `isolation: worktree` ayarlı bir alt ajan deponun yalıtılmış bir kopyasında çalışır ve değişiklik yapmazsa worktree otomatik silinir. Yalnız okuyan bir alt ajana worktree gerekmez. Dosya yazan her paralel işe gerekir.

Worktree'nin tam olarak ne yaptığı

`claude --worktree odeme-ekrani` komutu depoyu `.claude/worktrees/` altında yeni bir dala açar. Bu bir **taze checkout**'tur. İki sonucu var ve ikisi de sürpriz çıkarır.

Varsayılan dal. Belgeye göre yeni worktree, mevcut dalından değil deponun varsayılan dalından (genellikle `main`) açılır. Alt ajanların `isolation: worktree` kopyası da aynı kuralı izler. Yerelde commit'lediğin ama push etmediğin işler, ya da bir özellik dalındaki yarım iş worktree'de yoktur. İstersen `worktree.baseRef` ayarını `"head"` yaparsın. Bu ayar dal adı kabul etmez, yalnız `"fresh"` (varsayılan) ya da `"head"` alır.

Ignore edilmiş dosyalar gelmez. Worktree'ye yalnız git'in izlediği dosyalar gelir. `.env` gibi dosyaları otomatik taşımak için proje köküne `.worktreeinclude` yazarsın. Dosya `.gitignore` sözdizimini kullanır ve yalnız hem eşleşen hem ignore edilmiş dosyalar kopyalanır. Bağımlılıkları ise worktree'de yeniden kurman gerekir. Ayrıntı Dosya konumları sayfasında.

Etkileşimli bir worktree oturumundan çıkarken Claude Code temiz worktree'yi siler, iş içerene sorar. `-p` ile koşan oturumlarda çıkış sorusu yoktur, yani temizlik de yoktur. Bu worktree'leri `git worktree remove` ile kendin kaldırırsın.

Paylaşılan kaynaklar

Worktree yalnız dosya ve index çakışmasını çözer. Makinedeki geri kalan her şey hâlâ ortaktır.

- **Git index'i.** Aynı checkout'taki bütün oturumlar tek index'i paylaşır. Başkasının stage'ini senin commit'in alıp götürebilir. Ayrı worktree'nin index'i ayrıdır. 11.2
- **Port.** İki sunucu aynı porta bağlanamaz, ya da ikinci biri birincisini düşürür. Her projeye sabit ve ayrı port ver. 11.3
- **Tarayıcı oturumu.** Paylaşılan sekmeyi başka oturum öne getirir, sen kendi uygulamanı bozuk sanırsın. 11.3
- **Pano.** Makinede tek pano var. İki oturum değer yapıştırırsa biri ötekini ezer. Değeri doğrudan alana yaz, yazdıktan sonra geri oku.
- **Simülatör ve cihaz.** Tek simülatörü iki oturum sürerse ekrandaki durum her ikisine de yabancı olur.

Bu liste kapalı değil. Bir şey makinede tek bir örnek olarak duruyorsa, iki paralel iş için çakışma noktasıdır.

Makine yükü ve ajan sayısı

Her paralel iş makinenin işlemcisini, belleğini ve diskini yer. Ajan sayısı "kaç tane açabilirim" sorusuyla değil, "kaç tane bağımsız iş var" sorusuyla belirlenir. Beş ajan başlatıp ancak iki gerçek bağımsız iş bulmuşsan, kalan üçü yük ve token harcar, hız kazandırmaz. Çok ajanlı iş akışı onlarca ajan çağırabilir ve belge bunun çok token harcadığını söyler. Kullanım farkına `/usage` ile bak. 11.6

Yük ölçümü tek bir eşiğe bağlanmaz. Bir dakikalık yük ortalaması on beş dakikalığın birkaç katıysa yük o pencerede fırlamıştır. CPU, bellek baskısı ve disk birlikte izlenir. 11.4

Birleştirmenin bedeli

Paralel iş, sonunda birleştirmeyi ister ve bu bedel ajan sayısıyla büyür. Her worktree ayrı bir dal olduğu için her biri ayrı incelenir, ayrı test edilir ve ana dala ayrı katılır. İki dal aynı dosyaya dokunduysa çakışmayı birisi çözer. Birleştirme maliyeti işi bölmenin kazancından büyükse, bölme. Bağımsızlığı ölçerken şunu da hesaba kat: işleri bitirdikten sonra birleştirme için harcayacağın süre, paralelliğin sana kazandırdığı süreyi aşıyor mu?

Bulut oturumu

Makineyi yormadan paralel iş çalıştırmanın bir yolu, işi bulut oturumuna göndermektir. Bulut oturumu depoyu taze klonlar ve değişikliklerini `claude/` önekli bir dala yazar. Yerel dosyalarına, yerel simülatörüne ve yerel portuna erişemez. Yerel cihaz isteyen iş yerelde kalır. Bilgisayar uyusa bile ilerler. 11.5

Karar kuralları

- **Önce sonucu sor.** B, A'nın çıktısına bağlıysa ikisini sırala. 11.6
- **Dosya yazan her paralel işe ayrı worktree ver.** Yalnız okuyan iş worktree'siz de olur. 11.1
- **Worktree'ye yerel commit gerekiyorsa** önce push et ya da `worktree.baseRef: "head"` kullan.
- **Port, tarayıcı, simülatör gibi tek örnekli kaynaklara sahip çıkan iş sayısını bire indir** ya da her işe ayrı kaynak ver.
- **Ajan sayısını bağımsız iş sayısına eşitle,** makine yüküne göre aşağı çek. 11.4
- **Birleştirmeyi baştan planla.** Hangi dal hangi sırayla girecek, yazmadan başlama.
- **Makineden bağımsız kalması gereken uzun işi buluta gönder.** 11.5

Hata biçimleri

Gerçek vaka D06: başka oturumun commit'i değişikliklerimi götürdü. Ortak index yüzünden dört dosya başka bir mesajın altına girdi (Bölüm 4: Kanıtla teslim: iddiayı sayıya çevir). Ders: stage ile commit tek komutta olsun, arada derleme gerekiyorsa ayrı worktree kullan. Ayrıntısı Gerçek deneyimlerden on ders sayfasında D06.

Gerçek vaka D08: dört paralel oturum makineyi kilitledi. Dört alt oturum ayrı worktree'lerde aynı anda derleme ve test koştu, başka projelerin derlemeleri de sürüyordu. Yük ortalaması 720'ye çıktı, sakin makinede 4'tü. Worktree dosyaları ayırmıştı, işlemciyi ayırmamıştı. Çözüm kesin katmandaydı: iki yuvalı kilitle bir sarmalayıcı ve bir hook (Bölüm 5: Talimat mimarisi: rica ile garanti, Bölüm 8: Hook ile garanti: olay, çıkış kodu, sınama). Ayrıntısı D08.

Kurgu senaryo: iki özellik, tek ekran. Nilay iki ajanı iki worktree'de çalıştırır. Birine "profil ekranına avatar ekle", ötekine "profil ekranının yerleşimini yenile" der. Dosyalar ayrıdır, ama iki iş aynı ekranı yeniden yazıyor. Birleştirirken biri ötekinin işini siler. Sonuçlar bağımsız değildi, yalnız dosya adları ayrıydı.

Alıştırma: iki bağımsız işi iki worktree'de bitir

Herhangi bir git deposunda yap, ama önce depoda en az bir commit olduğundan emin ol.

1. Birbirine dokunmayan iki küçük iş seç. Örnek: biri README'ye bir bölüm ekler, öteki bir yardımcı fonksiyona test yazar. İkisinin de çıktısının diğerini etkilemediğini bir cümleyle yaz. 2. İki ayrı terminal aç. Birinde `claude --worktree is-bir`, ötekinde `claude --worktree is-iki` çalıştır. `git worktree list` çıktısını kaydet. 3. Her oturumda işi yaptır ve o oturumun worktree'sinde commit'le. Commit'ten önce `git diff --cached` ile kapsamı gör. 4. Ana kopyaya dön. İki dalı sırayla birleştir ve her birleştirmeden sonra testi koş. Çakışma çıkarsa nedenini kaydet. 5. `git worktree remove` ile iki worktree'yi kaldır.

Bitti sayılması için: `git worktree list` iki ayrı satır göstermiş olsun, `git log --stat` her işin kendi commit'inde yalnız kendi dosyalarını göstersin, birleştirmeden sonra test sayısı (sıfır değil) kayıtlı olsun.

Bağlantılar

Kontrol listesi: 11.1 11.2 11.3 11.4 11.5 11.6. Ekler: Gerçek deneyimlerden on ders, Dosya konumları, Komutlar ve kısayollar. Sonraki bölüm: Bölüm 10: Gözetimsiz iş: sen yokken çalışan sistem.

Rehber 10 · Gözetimsiz iş: sen yokken çalışan sistem

Uzman · Gözetimsiz iş, hangi kararın verilebileceğini, neyin işi durduracağını ve bittiğin kanıtını önceden yazmaktır. En pahalı hata işin yapılmaması değil, yapılmadığının fark edilmemesidir.

Zihinsel model: sen yokken karar veren bir sistem kuruyorsun

Gözetimsiz iş, "Claude'u çalıştırıp uzaklaşmak" değildir. Orada olmadığın sürece her kararı Claude verir, her tıkanıklıkta sana soramaz ve her sessizlik senin için bir sonuç gibi görünür. Bu yüzden baştan üç şeyi yazarsın: **hangi kararı Claude verebilir**, **ne olursa durmalı** ve **bittiğini hangi kanıt gösterir**.

Gözetimsiz işin en pahalı hatası iş yapılmaması değil, yapılmadığının fark edilmemesidir. Sabah baktığında log boşsa ve hiçbir hata yoksa, iki ihtimal var: iş sessizce bitti ya da hiç başlamadı. İkisi dışarıdan aynı görünür.

Sen yokken hangi karar verilebilir

"Sormadan ilerle" tek başına yetmez. Hangi kararın serbest, hangisinin yasak olduğunu yazmak gerekir. Pratik ölçü geri alınabilirliktir:

KARAR TÜRÜ	GÖZETİMSİZ KURAL
Geri alınabilir ve kapsam içinde (dosya düzenleme, yerel commit, deneme)	Claude verir, varsayımı raporda yazar
Sonucu değiştiren seçim (hangi kütüphane, hangi veri kaynağı)	En makul varsayımla ilerler, varsayımı işaretler
Geri alınamaz ya da dışarıya etkili (push, deploy, silme, ödeme)	Yapmaz, adımı "engelli" işaretler, kalanı bitirir
İzin gerektiren adım	En sona bırakılır, gerektirmeyen her şey önce biter

Bunu işe başlamadan görev dosyasına yaz. Yetki, kapsam, bütçe ve tıkanma davranışı net olsun. Her projeye sınırsız genel yetki yazma. İş tıkandığında "dur ve sor" yerine "o maddeyi atla, kalanı bitir, sonda yaz" de. Sabah ilk gördüğün şey çalışan bir sonuç olmalı, izin ekranı değil. 13.1

Headless koşum: claude -p

`claude -p "..."` Claude'u etkileşimsiz çalıştırır. Betikten, zamanlayıcıdan ya da başka bir araçtan çağırabilirsin. Çıktıyı `--output-format json` ile makine okuyacak biçime çevirirsin.

Belge çıkış durumunu şöyle tanımlar: başarıda 0, koşum başarısız olursa sıfırdan farklı bir kod. Ama iş içinde oluşan bir hata, örneğin kimlik doğrulama eksikliği, sonuç olarak stdout'a basılır. Yani çıkış koduna tek başına güvenme. Başarıyı üç yerden birlikte oku: çıkış kodu, JSON'daki `is_error` alanı ve sonuç türü (`subtype`). Belgeye göre bütçe ya da tur sınırı gibi sınırlar dolduğunda koşum hata türüyle biter. Bu sınırların dolması başarı değildir, işin yarım kaldığı anlamına gelir. 14.6

Makinenin okuyacağı sonucu serbest metinden ayıklama. `--json-schema` ile şema ver, sonuç `structured_output` alanında gelir. Claude geçerli çıktı üretemezse (örneğin isteği reddettiyse) alan hiç gelmez. Betik önce alanın varlığına baksın. 14.7

```
claude -p "Bu haftanın hata kayıtlarını özetle" --output-format json \
  --json-schema '{"type":"object","properties":{"kritik":{"type":"integer"}},"required":["kritik"]}' \
  > sonuc.json
jq -e '.is_error == false and has("structured_output")' sonuc.json
```

İki sınır işaretleyici vardır:

- **`--max-turns`:** tur sayısını sınırlar, dolunca hatayla çıkar. Belgede var, ama yerel 2.1.284 `--help` çıktısında görünmedi. Kullanmadan önce kendi sürümünde dene.
- **`--max-budget-usd`:** tahmini harcamaya üst sınır koyar, yalnız `-p` ile çalışır. Belge bunun istemci tarafı bir tahmin olduğunu ve gerçek faturadan sapabileceğini, sınırın aşılabileceğini yazar. Sınırı gerçek bütçenin altında tut.

Arka plan işi için belgede bir sınır daha var. Ana konuşmanın başlattığı bir arka plan komutu hâlâ çalışıyorsa koşum onu bekler. Beklenen bir şey yokken 10 dakika kesintisiz boşta beklenirse Claude Code çalışan işi durdurur ve kısmi sonucunu atar (süre bir ortam değişkeniyle değişir). Uzun işi arka plana atıp turu kapatmak bu yüzden güvenilir değildir.

Hedef, döngü ve zamanlama

Dört mekanizma sık karıştırılır. Her biri farklı bir şeyle başlar ve farklı bir şeyle durur.

MEKANİZMA	SONRAKİ TUR NE ZAMAN BAŞLAR	NEREDE YAŞAR	NE ZAMAN BİTER
`/goal`	Önceki tur bitince	Oturum	Bir model koşulu karşılanmış bulunca
`/loop`	Zaman aralığı dolunca	Açık oturum	Sen durdurunca ya da 7 gün sonra
Masaüstü zamanlanmış görev	Zaman gelince	Senin makinen	Sen silince
Bulut rutini (`/schedule`)	Zaman gelince	Anthropic'in bulutu	Sen silince

`/goal` bir tamamlanma koşulu verir. Her turdan sonra ayrı bir model koşulun sağlanıp sağlanmadığına bakar ve sağlanmadıysa Claude'u yeni tura sokar. `-p` ile de çalışır ve koşulu tek koşumda sonuna kadar götürür. Değerlendirici komut çalıştırmaz, dosya okumaz. Yalnız Claude'un konuşmaya getirdiği kanıtı görür. Bu yüzden koşulu "testler geçiyor ve çıkış kodu 0" gibi Claude'un çıktısında görünecek bir şeye bağla.

`/loop` oturum içi bir zamanlayıcıdır. Oturum kapanınca durur. Tekrarlayan görev oluşturulduktan 7 gün sonra son kez çalışır ve kendini siler. Kalıcı zamanlama yerine geçmez. 14.5

Masaüstü zamanlanmış görev makinende çalışır. Belgeye göre görev yalnız masaüstü uygulaması açıkken ve bilgisayar uyanıkken tetiklenir. Uyku boyunca kaçan koşum için uygulama açılınca, son yedi gün içinde en son kaçan zamana tek bir telafi koşumu başlatır. "Bilgisayarı uyanık tut" ayarı boşta uykuyu engeller, kapağı kapatmak yine uyutur.

Bulut rutini (`/schedule`) bilgisayarından bağımsızdır. Belge bunun bir araştırma önizlemesi olduğunu ve davranışın, sınırların değişebileceğini söylüyor. Rutin her koşuda depoyu taze klonlar, yerel dosyalarına erişemez, değişikliklerini `claude/` önekli bir dala yazar. Belgeye göre oturum otomatik modda başlar. En küçük aralık 1 saattir. 14.1

Ön koşullar

Ayrılmadan önce dört şeye bak. Biri eksikse gece boşa gider. 13.2

- **İzin listesi.** Projenin `.claude/settings.json` içindeki allow listesi, gözetimsiz koşumu ayakta tutan şeydir. Başka projedeki liste seni kurtarmaz.
- **Disk.** `df -h .` yeterli boş yer gösteriyor mu?
- **Yük.** `uptime` makinenin sakin olduğunu gösteriyor mu?
- **Uyku ve şarj.** Makine prizde mi, uyku engelleniyor mu? `pmset -g` bunu gösterir (macOS).

Tekrar başlatılabilir adımlar

Uyku ya da çökme işi baştan başlatmamalı. Ara çıktıyı diske yaz, her adımın başında "bu adım bitti mi" diye bak. Kaynağı geçici dizinde tutma, proje dizinine ya da kalıcı bir yola koy. 13.3

Karar kuralları

- **Sen yokken verilebilecek kararları önceden yaz.** Geri alınamaz adım engelli işaretlenir. 13.1

- **Çıkış kodu, `is_error` ve sonuç türünü birlikte oku.** Geçerli JSON başarı kanıtı değildir. 14.6
- **Makineye aktarılacak sonuca şema ver,** `structured_output` varlığını kontrol et. 14.7
- **Uzun işi ön planda tut,** log'u oturumdan bağımsız bir yola yaz. Arka plana düşen işi bitmesini bekleyerek ve çıkış satırını okuyarak uzlaştır. 13.4
- **"Ucuz" bir komutun süresini varsayma.** Yük altında uzar. 11.4
- **Makineden bağımsız olması gereken işi buluta ya da bulut rutinine ver.** Yerel dosya isteyen iş makinede kalır. 14.1
- **Rutinin kendisini izleyen küçük bir nöbet kur.** Hiç koşmayan rutin hata da vermez. 14.3
- **Sessizliği başarı sayma.** Kanıt log'un son satırı, çıkış durumu ve beklenen çıktı dosyasıdır.

Hata biçimleri

Gerçek vaka D01: mod sessizce değişti. Ayarlanan `bypassPermissions` modu zamanlanmış oturumda hiç görünmedi, oturum hata vermeden varsayılan moda düştü ve işi projenin kendi izin listesi ayakta tuttu (Bölüm 6: İzin ve güvenlik: mod, kural, sandbox). Ayrıntısı Gerçek deneyimlerden on ders sayfasında D01.

Gerçek vaka D03: arka plandaki iş tur bitince öldü. Zamanlanmış bir görev uzun bir veri çekimini arka planda başlattı ve turu "bekliyorum" diye kapattı. Oturum kapanınca süreç öldü, çıktı dizinle birlikte silindi. 83 dakika boyunca ağa tek istek gitmedi ve geriye tek satır kanıt kalmadı. Aynı iş ön planda ve kalıcı log'la 12 dakikada bitti. Ayrıntısı D03.

Gerçek vaka D04: "ucuz" kontrol 120 kat uzadı. Görev dosyası bir test komutunu yaklaşık 2,5 saniye diye tarif ediyordu. Bir nöbet koşumunda aynı komut 300 saniyelik ön plan sınırını aştı ve arka plana düştü. Makine yükü o dakikada 68'e çıkmıştı. Ders: arka plana düşen kontrolü bitmesini bekleyerek uzlaştır, yoksa D03 tekrarlanır. Ayrıntısı D04.

Gerçek vaka D05: uyku engelleyici bataryada işe yaramadı. Gece koşumu bir uyku engelleyiciyle sarılmıştı, makine prizde değildi. 19 saatte süreç 1,2 saniye işlemci zamanı kullandı ve çıktı dosyası boş kaldı. Prize takılınca aynı koşum 20 dakikada bitti. Ayrıntısı D05.

Gerçek vaka D09: geçici dizin toptan silindi. Çok günlük bir işin kaynakları geçici dizindeydi ve bir akşam silindi (Bölüm 3: Oturum: masayı doğru yönetmek). Kaynağı kalıcı yola yaz. Ayrıntısı D09.

Kurgu senaryo: sabah boş log. Okan her gece yapılan bir raporu `/loop` ile kurar ve ertesi sabah log'un boş olduğunu görür. Sebep, 7 gün dolduğu için görevin kendini silmesidir. Hata yoktu, çünkü koşum hiç tetiklenmemişti. Kalıcı zamanlamayı bulut rutinine ya da masaüstü göreve taşır ve yanına son başarılı koşumu kontrol eden bir nöbet ekler.

Alıştırma: bir gece koşumu kur, sabah kanıtla oku

Gerçek bir iş deposunda değil, bu iş için açtığın bir deneme deposunda yap.

1. `gorev.md` yaz. İçine yetkili kapsamı, engel davranışını ve istenen çıktı dosyasını yaz. Örnek görev: depodaki her kaynak dosyanın satır sayısını `work/sonuc.json` içine yaz. 2. Ayrılmadan önce `df -h .`, `uptime`, `pmset -g` çıktılarını `work/on-kosul.txt` dosyasına kaydet. `/permissions` ile izin listesini incele. 3. Koşumu şu biçimde kur: JSON çıktı, tur sınırı, kendi log yolu, çıkış kodunun dosyaya yazılması. `` mkdir -p work/runs claude -p "$(cat gorev.md)" --output-format json --max-turns 30 \ > work/runs/sonuc.json 2> work/runs/hata.log printf '%s\n' "$?" > work/runs/exit-code.txt `` 4. Koşumu zamanla ya da elle başlat. Başlamadan önce yarıda öldürüp yeniden başlatmayı bir kez dene, tamamlanan adımlar atlanıyor mu bak. 5. Sabah üç şeye bak: exit-code.txt, sonuc.json içinde is_error ve subtype, work/sonuc.json dosyası.

Bitti sayılması için: `exit-code.txt` içinde 0, `sonuc.json` içinde `is_error: false`, beklenen çıktı dosyası var ve içeriği görev tarifine uyuyor, log'un son satırı işin bittiğini gösteriyor. Bunlardan biri eksikse iş bitmiş sayılmaz, yalnız sessiz kalmıştır.

Bağlantılar

Kontrol listesi: 13.1 13.2 13.3 13.4 13.5 13.6 14.1 14.2 14.3 14.4 14.5 14.6 14.7. Ekler: Gerçek deneyimlerden on ders, Komutlar ve kısayollar, Sorun giderme. Önceki bölüm: Bölüm 9: Paralel işler: bağımsızlığı ölçmek.

Rehber 11 · Ekip ölçeği: ortak ayar, dağıtım ve güven

Uzman · Ekip dosyası bir öneri, yönetilen ayar bir kuraldır. Ayarı, skill'i ve plugin'i kimin yazdığını, kimin incelediğini ve kimin çalıştırdığını bilmek ekip ölçeğinin temel sorusudur.

Zihinsel model: yapılandırma bir bağımlılıktır

Tek başına çalışırken `.claude/settings.json` senin dosyandır. Ekipte çalışırken bu dosya git ile herkesin makinesine gider ve ilk oturumda ne yapacağına sen karar vermezsin, depoyu klonlayan karar verir. Bu yüzden ekip ölçeğinde tek soru şudur: bu dosyayı kim yazdı, kim inceledi, kim çalıştırıyor?

Claude Code ayarı dört sahiple okur. Öncelik yukarıdan aşağıdır: yönetilen ayarlar, komut satırı, `settings.local.json`, `settings.json`, kullanıcı ayarları. Üstteki sahip alttakini ezer.

SAHİP	DOSYA	KİME ULAŞIR	KİM DEĞİŞTİRİR
Kurum	`managed-settings.json` (sistem dizini), MDM profili ya da claude.ai konsolu	Politikanın ulaştığı herkes	Yönetici
Ekip	`.claude/settings.json`	Depoyu klonlayan herkes	Pull request
Kişi, bu proje	`.claude/settings.local.json`	Yalnız sen	Sen
Kişi, her proje	`~/.claude/settings.json`	Yalnız sen	Sen

Belgenin ince bir ayrımı var ve ekip kurarken en çok bu yanıltır: depoda duran ayar her zaman hemen işlemez. `permissions.allow` kuralları, `additionalDirectories` ve `extraKnownMarketplaces` girdileri, her ekip üyesi klasöre güven verdikten sonra geçerli olur. O zamana dek yeni gelen hâlâ izin sorusu görür. Sınırlayan kurallar (`deny`, `ask`) bundan etkilenmez, çünkü yetki vermezler. Kısacası: ekip dosyası bir öneridir, kurum dosyası bir kuraldır. Bunun ayrıntısı bölüm Bölüm 5: Talimat mimarisi: rica ile garanti ve Bölüm 6: İzin ve güvenlik: mod, kural, sandbox içinde.

Karar kuralları

Ortak olanı commit'le, kişisel olanı yerelde tut. Ekibin izin listesi, hook'ları ve ortak plugin'leri `settings.json`'a gider. Kendi modelin, kendi kısayolların `settings.local.json`'a. Claude Code yerel dosyayı oluşturduğunda onu git dışında bırakır. Dosyayı elle oluşturduysan `.gitignore`'a kendin eklemelisin. Bir ekip dosyası senin tercihinle çakışırsa kendi yerel dosyanda ezersin, commit gerekmez. 10.1

CLAUDE.md'ye kod gibi davran. Proje CLAUDE.md'sindeki bir satır bütün ekibin oturumunu değiştirir. Değişikliği pull request'ten geçir, gerekçesini commit mesajına yaz. Aksi hâlde altı ay sonra kimse bir kuralın neden orada durduğunu bilmez ve silmeye cesaret edemez. Kişisel tercihler `CLAUDE.local.md`'ye gider, o da gitignore'dadır. Büyük monorepoda başka ekiplerin CLAUDE.md dosyaları seni de etkiler, `claudeMdExcludes` ile hariç tutabilirsin. 10.2

Tekrar eden işi skill'e çevir, dağıtımı büyüklüğe göre seç. Bir depoya özgü prosedür `.claude/skills/` altında commit'lenir ve klonlayan herkes alır. Birden çok depoda aynı skill lazımsa plugin yap. Plugin, skill'leri, hook'ları ve MCP sunucularını tek birim olarak taşıyan bir dizindir. Bir marketplace de plugin'leri listeleyen bir kataloktur. Depoya ekleyebilirsin, kurum çapında da dağıtabilirsin. 10.3

Kuralı Claude'a değil, makineye uygulat. Kurumun "şu dosya okunmasın" dediği şey `managed-settings.json` içinde `permissions.deny` olarak durur. `allowManagedPermissionRulesOnly: true` ile kullanıcı, proje ve yerel dosyalardaki izin kuralları devre dışı kalır. CLAUDE.md burada yetmez, bölüm Bölüm 5: Talimat mimarisi: rica ile garanti bunun nedenini anlatıyor.

Yönetilen ayarı yayma yöntemini altyapına göre seç. Dört yol var: claude.ai konsolundan sunucu tarafı yönetim, macOS ve Windows için MDM, sistem dizinine bırakılan dosya ve Windows kullanıcı kaydı. Dosya yoluysa macOS'ta `/Library/Application Support/ClaudeCode/`, Linux'ta `/etc/claude-code/` altına koyarsın. Birkaç ekip aynı politikayı paylaşıyorsa her ekip kendi dosyasını `managed-settings.d/` dizinine koyar. Dosyalar alfabetik sırayla birleşir, bu yüzden `10-telemetri.json` ve `20-guvenlik.json` gibi numaralı adlar sıra verir. Listeler birleşir, tekil değerlerde sonraki dosya kazanır. Aynı makineye birden çok kaynak ulaşırsa varsayılan davranış en yüksek öncelikli tek kaynağı kullanmaktır, hepsini birleştirmek için `managedSourcesBehavior: "merge"` gerekir (v2.1.242 ve sonrası). Politikanın gerçekten işlediğini `/status` içindeki `Setting sources` satırından doğrula.

Duyuruyu hatırlatma için kullan, kural için değil. `companyAnnouncements` ayarı başlangıçta kurumun duyurusunu gösterir. Birden çok duyuru varsa her oturumda rastgele biri çıkar. İyi bir duyuru kısa bir hatırlatmadır: "Müşteri verisi yalnız kurum hesabıyla." Zorlama yoktur, yalnız görünürlük sağlar.

Hesap ve veri sınırını baştan çiz. Şirket işini şirketin onayladığı hesapla yap, çünkü veri politikası hesap türüne bağlı. Belgeye göre Team, Enterprise ve API gibi ticari hesaplarda Anthropic, Claude Code'a gönderilen kod ve prompt'larla model eğitmez (müşteri açıkça aksini seçmedikçe). Free, Pro ve Max hesaplarında eğitim izni ayarı geçerlidir. İç alan adı, müşteri verisi ve iç depo adresleri hiçbir kişisel hesaba ve özel depoya gitmesin. Paylaşmadan önce tarama yaptır. 7.2

Yeni depodaki hook ve MCP ayarını çalıştırmadan önce oku. Ortak hook'lar ekip için en güçlü araçtır ve tam bu yüzden en riskli olanıdır: hook senin yetkinle kabukta komut çalıştırır. Ayar dosyalarındaki hook'lar, `env` bloğu ve `apiKeyHelper` gibi yardımcı komutlar güven penceresini göstermeyen `claude -p` ile de çalışır (Bölüm 6: İzin ve güvenlik: mod, kural, sandbox). Tanımadığın bir depoyu otomasyonda çalıştırmadan önce `.claude/settings.json`, hook betikleri ve `.mcp.json` içeriğini oku. 7.7

Onboarding'i üret, elle yazma. `/team-onboarding` son 30 günlük oturum geçmişinden, komutlardan ve MCP kullanımından bir rehber üretir. Yeni ekip üyesi bunu ilk mesaj olarak yapıştırabilir. Team ve Enterprise aboneleri için paylaşılabilir bir bağlantı da döner. Çıktıyı yine de bir insan okusun: geçmişten çıkan alışkanlık ekibin istediği alışkanlık olmayabilir.

Hata biçimleri

Kurgu senaryo: "bende çalışıyordu". Selin'in ekibi `.claude/settings.json` dosyasına bir izin listesi koyar ve herkesin sormadan test çalıştıracağını varsayar. Yeni katılan Volkan depoyu klonlar, güven penceresini kapatıp `claude -p` ile bir betik koşturur. Allow kuralları uygulanmaz, betik izin sorusunda takılır, stderr'e düşen `this workspace has not been trusted` uyarısını da kimse okumaz. Çözüm ekip dosyasını suçlamak değil, yeni gelenin ilk adımına "klasörü interaktif aç, güven penceresindeki kuralları oku" maddesini koymaktır.

Kurgu senaryo: duyuru kural sanıldı. Barış, `companyAnnouncements` içine "`.env` dosyasını okuma" yazar ve işi bitmiş sayar. Duyuru yalnız başlangıçta görünür, hiçbir okumayı engellemez. Doğru yer yönetilen `permissions.deny` kuralıdır.

Gerçek vaka: katman bölünmesi. Gerçek deneyimlerden on ders sayfasındaki D07, bir kuralın "kesin" sandığın katmana hiç ulaşmadığını gösterir. Proje dışı import headless koşumda yüklenmedi (Bölüm 1: Zihinsel model: döngü, bağlam ve harness). Ekip ölçeğinde aynı örüntü şöyle görünür: bir kişide çalışan yapılandırma, başka bir kişinin makinesinde sessizce eksik kalır. Ayırıcı kanıt, kuralı gerçekten çalıştıran oturumda "bu kural ne diyor?" diye sormaktır.

Plugin deposu ayarıyla yetinmek. Marketplace girdisi göreli yolla listelenmiş bir plugin, deponun ayarından kurulur. Plugin dış bir kaynağa işaret ediyorsa yalnız ayarla kurulmaz. Her katılımcı önce `claude plugin install <ad>@<marketplace> --scope project` komutunu çalıştırır, o zamana dek "enabled in project settings but isn't installed" uyarısını görür.

Alıştırma: ekip paketi ve temiz klon

Bunu bir deneme deposunda yap.

1. Depoda `.claude/settings.json` oluştur. İçine bir `permissions.deny` kuralı koy, örneğin `Read(./.env)`, ve `permissions.allow` altına yalnız bir salt okunur komut ekle. 2. Depoya bir skill ekle: `.claude/skills/ozet/SKILL.md`. Skill, açıklaması "değişen dosyaları özetler" olan üç satırlık bir prosedür olsun. 3. `.claude/settings.local.json` oluştur ve kendi kişisel bir tercihini yaz. Dosyanın git dışında kaldığını `git status` ile gör. 4. Her şeyi commit'le, ardından depoyu başka bir dizine klonla: `git clone <depo> ../temiz-klon`. 5. Temiz klonda yeni bir oturum aç. Güven penceresindeki kuralları oku, kabul et, sonra `/ozet` komutunu çalıştır. `.env` okumasını dene ve reddedildiğini gör. 6. Aynı skill'i bir plugin'e çevir: `ekip-paketi/.claude-plugin/plugin.json` ve `ekip-paketi/skills/ozet/SKILL.md`. `claude --plugin-dir ./ekip-paketi` ile yükle. Skill artık `/ekip-paketi:ozet` adıyla görünür.

Bitti sayılması için: temiz klondan alınmış üç kanıt: güven penceresinin listelediği kurallar, `/ozet` çıktısı ve reddedilen `.env` okuması. Altına bir cümle yaz: kişisel tercihin klona geçmediğini nereden gördün.

Bağlantılar

Kontrol listesi: 10.1 10.2 10.3 7.2 7.7. Önceki bölüm: Bölüm 10: Gözetimsiz iş: sen yokken çalışan sistem. Ekler: Dosya konumları, Komutlar ve kısayollar, Gerçek deneyimlerden on ders.

Rehber 12 · Sistemi iyileştirmek: hatayı katmana yazmak

Uzman · Her başarısızlığın adresi bağlam, model kararı ya da harness'tir ve ders doğru katmana yazılır. Üçüncü tekrar kural olur, bakım ritmi ve önce ve sonra karşılaştırması sistemi sağlıklı tutar.

Zihinsel model: her hata bir katmanın adresini taşır

Claude Code yanlış bir şey yaptığında ilk refleks "daha açık yazayım" olur. Çoğu zaman yanlış çözümdür, çünkü hatanın hangi katmandan geldiğini bilmeden düzeltme yazarsın. Bölüm 1: Zihinsel model: döngü, bağlam ve harness üç parçayı anlatmıştı: model karar verir, araçlar iş yapar, harness bunları sarar. Sistemi iyileştirmek bu üç parçaya sorulan tek bir soruyla başlar: bu başarısızlık hangisinden geldi?

BELİRTİ	KATMAN	DÜZELTME NEREDE
Claude kuralı bilmiyordu ya da unuttu	Bağlam	CLAUDE.md, rule, bellek, ya da kuralı kesin katmana taşı
Claude kuralı gördü ama kötü bir seçim yaptı	Model kararı	Daha iyi örnek, daha dar görev, daha güçlü model, doğrulama adımı
Komut çalışmadı, izin istedi, hook engelledi, dosya yüklenmedi	Harness	Ayar, izin listesi, hook, yükleme yolu

Ayrım pratik sonuç verir. Bağlam hatasında kuralı yazarsın, ama o kural tek yerde ve doğru katmanda durur. Model kararı hatasında kural eklemek işe yaramaz, çünkü Claude kuralı zaten biliyordu. Harness hatasında hiçbir metin yardım etmez, ayarı düzeltirsin. Üçünü karıştıran ekip CLAUDE.md'sini şişirir ve sorun yine de geri gelir. Belge de aynı yönde uyarır: uzun dosya hem bağlam harcar hem uyumu düşürür, çelişen iki talimattan Claude birini rastgele seçebilir.

Dersi doğru katmana yaz

Bir ders öğrendiğinde dosyayı seçmeden önce şunu sor: bu cümle nerede doğru olmaktan çıkıyor?

- **Her projede doğruysa** kullanıcı düzeyine yaz: `~/.claude/CLAUDE.md`. Makine ve araç gerçekleri buraya gider: izin motoru, kabuk, zamanlayıcı davranışı.
- **Yalnız bu koda aitse** proje CLAUDE.md'sine yaz. Mimari, kıyas kuralı, sürüm sabitlemesi buradadır.
- **Yalnız bu depo ile bu makinenin kesişimiyse** `CLAUDE.local.md`'ye yaz ve gitignore'a ekle. Yerel yol, sanal ortam konumu gibi şeyler.
- **Tarihli ölçüm ve kanıt ise** ayrı bir kanıt dosyasına yaz. Kuralın kendisi kısa kalır, ölçümün gövdesi orada durur. 18.1 18.4

Yanlış katman sessiz bir kayıptır. Makine gerçeğini proje dosyasına yazarsan diğer projeler onu hiç görmez. Proje gerçeğini kullanıcı dosyasına yazarsan başka depolarda gürültü olur.

Karar kuralları

Üçüncü tekrar kuraldır. Aynı düzeltmeyi ilk kez yapman tesadüf, ikincisi bir işaret, üçüncüsü kuraldır. Düzeltmeleri say. Üçe ulaşınca kuralı yaz, ama önce katmanı seç: istisnası olmayan şey hook ya da deny kuralına gider, tercih CLAUDE.md'ye. Belgenin bölüm Bölüm 5: Talimat mimarisi: rica ile garanti'te geçen tetik listesi aynı mantıkla çalışır: "Claude aynı şeyi iki kez yanlış yaptı" CLAUDE.md'ye girer. 18.3

Kuralı kısa yaz, kanıtı ayrı tut. Bağlamda yer alan her satır her oturumda bedel öder. Kuralın tek cümlesi CLAUDE.md'de, ölçümün gövdesi ayrı bir dosyada olsun. Kuralın sonuna "kanıt: kanit.md, başlık" gibi bir atıf koy. Böylece Claude gerektiğinde açar, her oturumda taşımaz. 18.4

Bekleyeni hemen dosyaya yaz. "Sonra", "unutma", "şimdilik bırak" denen şey oturum içi hafızada kalırsa devirde kaybolur. Proje kökünde bir bekleyenler dosyası tut, CLAUDE.md'de buna dair tek kural olsun. "Neler kaldı?" sorusunun cevabı dosyadan gelmeli. 18.2

Bakımı ritme bağla. Küçük ve düzenli bakım büyük temizlikten ucuzdur.

RİTİM	İŞ	ARAÇ
Haftalık	Kullanımı ve bekleyenleri gözden geçir	`/usage`, bekleyenler dosyası
Aylık	CLAUDE.md'leri buda, izin listesi ve hook testlerini yokla	`/doctor prompt-audit`, `/fewer-permission-prompts`
Çeyreklik	Kendi kullanım analizini çıkar, önceki çeyrekle karşılaştır	`/insights`, oturum kayıtları

Aylık budamada dikkat et: seyrek ama kritik bir kuralı yalnız geçen ay tetiklenmedi diye silme. Dosyanın küçük olması tek başına başarı ölçütü değildir. 19.1 19.3 19.4 19.5

Araçları ne için kullandığını bil. `/usage` kullanım ve limit durumunu gösterir. Pro, Max, Team ve Enterprise planlarında neyin limitini tükettiğinin dökümünü ve öneriler de sunar, önerileri uygulatan `f` kısayolu v2.1.305 ve sonrasını ister. `/insights` makinendeki son oturumları tarar ve neyle uğraştığını, nerede sürtünme yaşadığını gösteren bir HTML rapor yazar. Tek çalıştırmada henüz görmediği en çok 200 oturumu analiz eder. `/doctor` kurulumun, ayarların, eklentilerin ve CLAUDE.md dosyalarının sağlık kontrolünü yapar, düzeltmeleri sen onaylayınca uygular. `/doctor prompt-audit` (v2.1.283 ve sonrası) talimat dosyalarını tarar: eski modeller için yazılmış talimatları, var olmayan dosya ya da komut referanslarını ve birbiriyle çelişen dosyaları arar. Önerileri bir rapor olarak verir ve sen istemeden hiçbir dosyayı değiştirmez. 19.1 19.5

Yeni model çıkınca talimatları yeniden sına. Belgede "model çıkınca şunu yap" diye bir döngü yok, bu bir iş akışı önerisi. Gerekçesi şu: eski modeller için yazdığın vurgu, ek hatırlatma ya da ayrıntılı adım listesi yeni modelde gereksiz ya da zararlı olabilir. Bölüm Bölüm 5: Talimat mimarisi: rica ile garanti'te anlatıldığı gibi vurgu yığmak güncel modellerde kuralı fazla uygulatabilir. Önce `/doctor prompt-audit` ile adayları çıkar, sonra aşağıdaki karşılaştırmayla budamanın bir şeyi bozup bozmadığına bak.

Modeli aynı görevle karşılaştır. Genel izlenim ve tek bir başarılı cevap model seçimini kanıtlamaz. Kendi işinden temsilî iki üç görev seç. Her birini iki ayarla aynı prompt ve aynı başlangıç durumuyla headless koştur. Sonucu, tur sayısını ve maliyeti kaydet, ölçütü önceden belirle. Belgede hazır bir karşılaştırma tarifi yok, bu parçalarla kurarsın. 5.6

```
for m in sonnet opus; do
  claude -p "$(cat gorev.md)" --model $m --output-format json \
    --no-session-persistence > sonuc-$m.json
done
```

Sürüm ve ortam farkını kaydet. Bir karşılaştırmanın değeri, hangi sürümde yapıldığına bağlı. Raporuna `claude --version` çıktısını, platformu ve sağlayıcıyı yaz. Belgede olan özellik eski sürümde ya da başka sağlayıcıda bulunmayabilir. Bu bölümdeki iki örnek: `/doctor prompt-audit` v2.1.283, `/usage` içindeki `f` kısayolu v2.1.305 ister. 19.6

Hata biçimleri

Gerçek vaka: dersin katmanı (D09). Gerçek deneyimlerden on ders sayfasındaki D09'da çok günlük bir işin kaynakları geçici dizindeydi ve silindi (Bölüm 3: Oturum: masayı doğru yönetmek). Vakanın dersi ("birden çok gün sürecek işin kaynağı geçici dizinde tutulmaz") bir makine gerçeğidir, bu yüzden bu rehberin ölçütüne göre kullanıcı düzeyi dosyaya ait. Aynı cümle bir proje CLAUDE.md'sine yazılsaydı başka projeler aynı kayıpla karşılaşırdı. Ölçümün gövdesi (silme tarihi, neyin geri kurulabildiği) kısa kuralın dışında, kanıt sayfasında durur. Sınırı da vaka yazıyor: silmenin nedeni ölçülmedi.

Gerçek vaka: eski rapor canlı sistem sanıldı (D10). Bölüm 1: Zihinsel model: döngü, bağlam ve harness bölümünde bu vakayı bağlam hatası olarak okuduk: eski bir cümle pencereye girmişti. Düzeltmeyi ise kaynağa bakmayan modelin davranışına yazarsın, yani kısa bir kural olur, uzun bir yasak listesi değil: değişebilen bir durumu yazmadan önce kaynağın kendisine bak, kullanıcının yapıştırdığı not da bir rapordur. Ayırıcı kanıt kaynağın o anki değeri ve okuma tarihidir.

Kurgu senaryo: sönmeyen CLAUDE.md. Ferit'in proje dosyası altı ayda 380 satıra çıkar. Her hatadan sonra yeni bir kural eklenir, hiçbiri silinmez. Bir gün iki kural çelişir, Claude birini seçer ve Ferit sebebini anlayamaz. `/doctor prompt-audit` çelişkiyi ve var olmayan iki dosyaya yapılan atıfı bulur. Ferit uzun prosedürleri skill'e taşır, 140 satır kalır.

Alıştırma: önce ve sonra

Bunu kendi proje talimatlarının bir kopyasında yap. Gerçek dosyayı karşılaştırma bitene dek değiştirme.

1. Üç temsilî iş seç: biri küçük bir düzeltme, biri araştırma, biri çok adımlı bir değişiklik. Her birini bir `gorev-N.md` dosyasına yaz. Başarı ölçütünü de oraya yaz. 2. Mevcut talimatlarla her işi headless koştur. Sonucu, tur sayısını ve maliyeti kaydet. 3. `/doctor prompt-audit` çalıştır. Önerilerden en az üçünü uygula: eski vurgular, çelişkiler, var olmayan referanslar. Her silinen satırı bir cümleyle gerekçelendir. 4. Üç işi budanmış talimatlarla aynı koşullarda yeniden koştur. 5. Sonuçları ölçütüne göre karşılaştır.

Bitti sayılması için: üç satırlık bir önce/sonra tablosu (iş, sonuç, tur, maliyet), silinen satırların listesi ve `claude --version` çıktısı. Altına tek cümle yaz: budama hangi katmandaki hatayı çözdü, hangisini çözmedi.

Bağlantılar

Kontrol listesi: 18.1 18.2 18.3 18.4 19.1 19.3 19.4 19.5 19.6 5.6. Önceki bölüm: Bölüm 11: Ekip ölçeği: ortak ayar, dağıtım ve güven. Ekler: Komutlar ve kısayollar, Gerçek deneyimlerden on ders, Sorun giderme.

Rehber 13 · Bitirme projesi: her adımda kanıt

Uzman · Kendi seçtiğin küçük ve gerçek bir işi plan, uygulama, doğrulama ve teslim aşamalarından geçirirsin. Her aşamada önceki bölümlerin becerisini kullanır ve bir kanıt dosyası bırakırsın.

Zihinsel model: uzmanlık, her adımda kanıt bırakmaktır

Önceki on iki bölümde ayrı ayrı beceriler öğrendin: bağlamı yönetmek, işi bitiş çizgisiyle vermek, kuralı doğru katmana koymak, paralel ve gözetimsiz çalışmak, sistemi iyileştirmek. Bu bölümde hepsini tek bir işte birleştireceksin. Ölçüt öncekilerden farklı: yeni bir şey öğrenmeyeceksin, öğrendiğini **her adımda kanıtla** göstereceksin.

Bu ölçüt rastgele seçilmedi. Bölüm 4: Kanıtla teslim: iddiayı sayıya çevir bölümünün ana fikri şuydu: "tamamlandı" bir iddiadır, kanıt değil. Uzmanla acemi arasındaki fark, uzmanın bir sonucu gördüğünde hangi katmandan geldiğini bilmesi ve bir şeyin bittiğini sayıyla, çıktıyla ya da dosyayla göstermesidir. Bitirme projesinde bunu zorunlu kılıyoruz: her aşamanın sonunda masaya konabilecek bir kanıt olacak. Kanıtı olmayan aşama bitmiş sayılmaz.

Bu bölümün yapısı diğerlerinden farklı. Alıştırma yerine bir proje, hata biçimleri yerine bir değerlendirme listesi var. Çünkü burada hata yapmak, öğrenmenin parçası: takıldığın yer, hangi bölümü yeniden okuman gerektiğini gösterir.

Proje

Kendi seçtiğin küçük ama gerçek bir iş yap. "Gerçek" demek, sonucu bir gün gerçekten kullanacağın demek. Oyuncak örnek değil. Birkaç fikir:

- Bir klasördeki dosyaları kurala göre ayıran ve ne yaptığını raporlayan bir betik.
- Bir CSV ya da log dosyasından haftalık özet üreten bir rapor üretici.
- Tek sayfalık, birkaç bölümden oluşan küçük bir web sayfası.

Üç sınırı baştan koy:

1. **Herhangi bir git deposunda yapılabilsin.** Yeni, boş bir depo da olur. Gerçek bir iş deposuna dokunma, bu iş için açtığın bir depoda çalış. 2. **Bir iş günü sürmesin.** Büyük işi küçültüp bitmiş bir şey çıkarmak, büyük işi yarım bırakmaktan değerlidir. Sığmayan kısmı "sonra" listesine yaz. 3. **Dışarıdan veri ve hesap gerektirmesin.** Ücretli servis, gerçek müşteri verisi ya da kurum hesabı olmasın. Gerekirse sahte veri üret.

Seçimini tek cümleyle yaz: "Şunu yapacağım, şu girdiyi alıp şu çıktıyı verecek, şu komutla çalıştırılacak."

Aşamalar

Dört aşama var. Her birinde hangi bölümün becerisini kullandığın yazılı. Aşamayı bitirmeden sonrakine geçme.

1. Plan

Claude'a kod yazdırmadan önce işi tanımla. Girdi, çıktı, "bitti" ölçütü ve dokunulmayacak yerler yazılı olsun. İş küçükse plan modunda bir plan iste ve oku. Beğenmediğin yeri değiştirmeden uygulamaya geçme. Becerin burada Bölüm 2: İş vermek: bitiş çizgisini çizmek (bitiş çizgisini çizmek) ve Bölüm 1: Zihinsel model: döngü, bağlam ve harness (hangi parçanın neyi yapacağı). Projeye bir `CLAUDE.md` aç ve yalnız bu iş için gerçekten gerekli üç dört satır yaz: nasıl çalıştırılır, nasıl test edilir, neye dokunulmaz. Hangi satırın kural, hangisinin tercih olduğunu Bölüm 5: Talimat mimarisi: rica ile garanti içindeki ölçütle ayır. İstisnası olmayan bir şey varsa (örneğin "örnek veri dosyasını silme") bunu CLAUDE.md'de değil izin kuralında tut, Bölüm 6: İzin ve güvenlik: mod, kural, sandbox bu konuda yardım eder.

2. Uygulama

İşi küçük adımlarla ilerlet ve her adımdan sonra çalıştır. Oturum uzarsa bağlamı yönet: yeni bir iş için temiz oturum aç, gerekirse özet bırakarak devam et. Bu Bölüm 3: Oturum: masayı doğru yönetmek bölümünün becerisidir. Tekrar eden bir adımı fark edersen (aynı komutu üçüncü kez yazıyorsan) onu bir skill ya da betiğe çevir, hook gerektiren bir şey varsa Bölüm 8: Hook ile garanti: olay, çıkış kodu, sınama bölümüne dön. Projede birbirinden bağımsız iki parça varsa (örneğin betiğin kendisi ve belgesi) bunları ayrı worktree'lerde paralel yaptırabilirsin. Ama önce bağımsız olduklarını kanıtla, Bölüm 9: Paralel işler: bağımsızlığı ölçmek bölümündeki uyarıyı hatırla: paralel olmak bağımsız olmak demek değildir.

3. Doğrulama

Burada kendi yazdığın kontrolü sına. Claude "çalışıyor" derse sen çalıştır ve çıktıyı gör. En az iki sınama yap:

- **Beklenen yolda:** gerçek bir girdiyle çalıştır, çıktıyı beklenenle karşılaştır.
- **Bozulan yolda:** bir hatayı kasıtlı olarak geri koy (yanlış bir kural, bozuk bir girdi satırı) ve kontrolün bunu yakaladığını gör. Hatayı geri koyduğunda kontrol düşmüyorsa o kontrol hiçbir şeye bakmıyordur. Bölüm 4: Kanıtla teslim: iddiayı sayıya çevir bunun vakalarını anlatıyor.

Test varsa koşulan test sayısını oku: sıfır test koşmak geçmek değildir. Test yoksa yazdır, sonra yine hatayı geri koyarak sına. Doğrulamayı yapan oturum, kodu yazan oturum olmasın. Yeni bir oturumda "bu çıktıyı kırmaya çalış" diye iste.

4. Teslim

Teslim, işi bırakmaktır ama bırakırken bir sonraki kişiye (ya da bir ay sonraki kendine) ne olduğunu anlatman gerekir. Üç şey bırak:

- Çalıştırma komutu ve örnek çıktı.
- Neyin yapılmadığı ve neden. Yarım bıraktığın her madde proje kökündeki bir bekleyenler dosyasına girsin.
- Sürüm ve ortam notu: `claude --version`, işletim sistemi, kullandığın model.

İşi commit'le. Her commit'in mesajı o değişikliğin neden yapıldığını söylesin. Push yapma, deneme deposunda gerek yok. Eğer işi gözetimsiz bir koşuma da bağlamak istersen Bölüm 10: Gözetimsiz iş: sen yokken çalışan sistem bölümündeki kuralları uygula: ara çıktı diske yazılsın, uzun iş arka plana atılıp tur kapatılmasın. Ekibinle paylaşacaksan Bölüm 11: Ekip ölçeği: ortak ayar, dağıtım ve güven bölümündeki ayrımı kullan: ortak olan commit'lenir, kişisel olan yerelde kalır. Son olarak Bölüm 12: Sistemi iyileştirmek: hatayı katmana yazmak bölümündeki soruyu sor: bu projede öğrendiğim ders hangi katmana yazılmalı? Dersi yaz ve nereye yazdığını söyle.

Kanıt listesi

Aşağıdaki tabloyu projenin köküne `KANIT.md` olarak kaydet ve doldur. Her satır için somut bir dosya, çıktı ya da komut göster. "Yaptım" yazmak kanıt sayılmaz.

AŞAMA	BEKLENEN KANIT
Plan	Tek cümlelik proje tanımı, yazılı "bitti" ölçütü, dokunulmayacaklar listesi
Plan	Dört satırı geçmeyen `CLAUDE.md` ve her satırın neden kural ya da tercih olduğu
Uygulama	Küçük adımlara bölünmüş commit geçmişi (`git log --oneline`)
Uygulama	Bağlamın dolduğu ya da yeni oturuma geçtiğin bir noktada bıraktığın özet
Doğrulama	Beklenen yolda gerçek girdi ve çıktı
Doğrulama	Kasıtlı geri konan hata ve kontrolün onu yakaladığını gösteren çıktı
Doğrulama	Koşulan test sayısı (sıfırdan büyük) ya da testin yokluğunu açıklayan not
Teslim	Çalıştırma komutu, örnek çıktı, sürüm ve ortam notu
Teslim	Bekleyenler dosyası: yapılmayanlar ve nedenleri

AŞAMA	BEKLENEN KANIT
Teslim	Öğrendiğin dersin yazıldığı dosya ve katmanın gerekçesi

Değerlendirme

Sayısal puan yok. Projeyi bitirdikten sonra aşağıdaki soruları kendine, dürüstçe sor. "Hayır" ya da "emin değilim" cevabı, geri dönüp yeniden okuman gereken bölümü gösterir.

- Projeyi başlamadan önce tek cümleyle ve ölçülebilir bir "bitti" ölçütüyle tanımladın mı? (Bölüm 2: İş vermek: bitiş çizgisini çizmek)
- Bir sonuç beklenmedik çıktığında önce hangi katmandan geldiğini (model, araç, harness) sordun mu? (Bölüm 1: Zihinsel model: döngü, bağlam ve harness)
- Oturum şiştiğinde yeni oturuma geçtin mi, ve geçerken neyi taşıdığını biliyor musun? (Bölüm 3: Oturum: masayı doğru yönetmek)
- Her "tamamlandı" cümlesinin arkasında senin gördüğün bir çıktı var mı? (Bölüm 4: Kanıtla teslim: iddiayı sayıya çevir)
- Kuralların hangileri Claude'un kararında, hangileri Claude Code'un uyguladığı katmanda? Bunu açıklayabilir misin? (Bölüm 5: Talimat mimarisi: rica ile garanti)
- Kontrolünü hatayı geri koyup düşürerek sınadın mı, yoksa yalnız geçtiğini mi gördün? (Bölüm 4: Kanıtla teslim: iddiayı sayıya çevir)
- Yarım bıraktığın bir şey bir dosyada yazılı mı, yoksa yalnız aklında mı? (Bölüm 12: Sistemi iyileştirmek: hatayı katmana yazmak)
- Paylaşılan bir kaynağı (dal, index, port, dosya) iki yerden kullandıysan çakışmayı nasıl önledin? (Bölüm 9: Paralel işler: bağımsızlığı ölçmek)
- Öğrendiğin dersi doğru katmana yazdın mı ve neden orası? (Bölüm 12: Sistemi iyileştirmek: hatayı katmana yazmak)
- Bu projeyi bir ekip arkadaşına devretsen, `KANIT.md` ve depo yeterli olur muydu? (Bölüm 3: Oturum: masayı doğru yönetmek)
- Bir şey sana en pahalıya geldiyse, o hata bir şeyin kırılmasından mı, yoksa hiçbir şeye bakmadan başarılı görünmesinden mi çıktı? (Gerçek deneyimlerden on ders sayfasındaki vakalar bu soruya dönüyor.)

Kendini dürüstçe sorguladığında üç şeyi de yaz: neyi kaçırmış olabilirsin, hangi iddian en zayıf, hangi kararının gerekçesi yok. Rehberi bitirmiş olmanın işareti, bu üç cümleyi rahatça yazabilmek.

Bağlantılar

Önceki bölümler: Bölüm 1: Zihinsel model: döngü, bağlam ve harness Bölüm 2: İş vermek: bitiş çizgisini çizmek Bölüm 3: Oturum: masayı doğru yönetmek Bölüm 4: Kanıtla teslim: iddiayı sayıya çevir Bölüm 5: Talimat mimarisi: rica ile garanti Bölüm 6: İzin ve güvenlik: mod, kural, sandbox Bölüm 7: Yeniden kullanım: skill, alt ajan, MCP, plugin Bölüm 8: Hook ile garanti: olay, çıkış kodu, sınama Bölüm 9: Paralel işler: bağımsızlığı ölçmek Bölüm 10: Gözetimsiz iş: sen yokken çalışan sistem Bölüm 11: Ekip ölçeği: ortak ayar, dağıtım ve güven Bölüm 12: Sistemi iyileştirmek: hatayı katmana yazmak. Kontrol listesi: 12.1 12.2 18.1 18.2 19.6. Ekler: Gerçek deneyimlerden on ders, Sorun giderme, Terimler sözlüğü.

Ek · Komutlar ve kısayollar

Slash komutları işe göre, en çok işe yarayan CLI bayrakları, klavye kısayolları ve sık karıştırılan çiftler. Belge kontrolü: 29 Ekim 2026.

Claude Code'a üç yoldan komut verilir. Oturumun içinde `/` ile başlayan **slash komutu**, kabukta `claude` ile başlayan **CLI bayrağı** ve **klavye kısayolu**. Aynı iş çoğu zaman üçünden biriyle yapılır: geçmiş oturuma `/resume` ile oturumun içinden, `claude --resume` ile kabuktan dönülür.

Bu sayfa günlük işte fark yaratan komutları işe göre gruplar. Belgedeki komut tablosu 120 satırdan uzun, hepsi burada yok. Belge yerel CLI 2.1.284'ten yeni, bazı komutlar senin sürümünde görünmeyebilir. Emin olmak için oturumda `/help` yaz ya da `claude --version` ile sürümü gör. Kaynaklar: komutlar, CLI referansı, etkileşimli mod.

Slash komutları, işe göre

Bağlam

KOMUT	NE ZAMAN	DİKKAT
`/context`	Bağlamın neyle dolduğunu görmek için	Talimat, araç ve mesaj payını ayrı gösterir. Tahmin yerine buna bak. 2.2
`/compact [odak]`	Uzayan oturumda yer açmak için	Ayrıntı kaybettirir. Neyin korunacağını yaz. 2.6
`/clear`	Konu değişince	Konuşmayı sıfırlar, dosyalara dokunmaz. 2.3
`/btw [soru]`	Ana işi bölmeden kısa bir şey sormak için	Soru ve cevap konuşmaya eklenmez.
`/recap`	Uzun oturumda nerede kaldığını görmek için	Tek satırlık özet. Devir dosyasının yerini tutmaz.

Oturum ve geri dönüş

KOMUT	NE ZAMAN	DİKKAT
`/resume [ad]`	Eski bir oturuma dönmek için	`/rename` ile ad verdiysen adla bulunur. 2.7
`/branch [ad]`	Aynı noktadan başka bir yol denemek için	Seni dala geçirir, orijinal korunur.
`/fork [prompt]`	Konuşmayı arka planda ikinci bir oturumda sürdürmek için	`/branch` ile aynı değil: sen burada devam edersin.
`/rewind`	Kodu, konuşmayı ya da ikisini geri sarmak için	Boş girdide Esc Esc ile de açılır. Yalnız Bash'le değişen dosyayı geri almaz. 15.2
`/rename [ad]`	Oturumu sonra bulabilmek için	Ad vermezsen kendisi üretir.
`/export [dosya]`	Konuşmayı metin olarak saklamak için	Hassas içerik varsa paylaşmadan önce oku.
`/diff`	Çalışma ağacındaki değişikliği görmek için	Claude'un yaptığı düzenlemeler dahil.

Model, çaba ve kullanım

KOMUT	NE ZAMAN	DİKKAT
`/model [model]`	Modeli değiştirmek için	Seçim varsayılan olarak kaydedilir. Yalnız bu oturum için seçicide ayrı seçenek var. 5.2

KOMUT	NE ZAMAN	DİKKAT
`/effort [düzey]`	Düşünme çabasını işe göre ayarlamak için	Düzeyler modele bağlı. 5.3
`/fast`	Hızlı modu açıp kapatmak için	Model kataloğuna ve hesaba bağlı.
`/advisor`	Zor kararda ikinci bir modelin görüşü için	5.5
`/usage`	Oturum maliyeti, plan limiti ve istatistik için	`/cost` ve `/stats` aynı komutun takma adı. 5.4

Talimat, ayar ve izin

KOMUT	NE ZAMAN	DİKKAT
`/init`	Projeye ilk CLAUDE.md taslağı için	Taslağı gerçek komutlarla düzelt. 1.2
`/memory`	Yüklenen talimat dosyalarını ve otomatik belleği görmek için	Import'un yüklenip yüklenmediğine buradan bak. 1.5 1.7
`/status`	Sürüm, model, hesap ve yüklenen ayar dosyaları için	`Setting sources` dosyaları listeler, anahtar bazında kaynak göstermez. 1.8
`/config`	Ayar arayüzü ya da `key=value` ile ayar yazmak için	Takma adı `/settings`. Hangi katmana yazdığına dikkat et.
`/permissions`	allow, ask, deny kurallarını yönetmek için	6.1 6.3
`/fewer-permission-prompts`	Geçmişten salt okunur komutlar için allow listesi önerisi	Öneriyi kabul etmeden önce tek tek oku. 6.6
`/sandbox`	Sandbox'ı açıp kapatmak için	Yalnız desteklenen platformlarda. 7.5
`/hooks`	Etkin hook'ları görmek için	Yalnız gösterir. Hook'un çalıştığını göstermez, onu testle sına. 8.4
`/statusline`	Durum satırını kurmak için	8.3
`/doctor`	Kurulum, ayar ve eklenti sorunlarını taramak için	Claude hiç açılmıyorsa kabukta `claude doctor`.

Genişletme

KOMUT	NE ZAMAN	DİKKAT
`/skills`	Yüklü skill'leri görmek için	Açıklaması zayıf skill listede olsa da çağrılmaz. 9.3
`/reload-skills`	Diske eklenen skill'i yeniden başlatmadan almak için	
`/plugin`	Plugin kurmak ve yönetmek için	9.6
`/mcp`	MCP sunucularının durumunu görmek, yeniden bağlamak için	Bağlı görünmek aracın çalıştığını kanıtlamaz. 9.5
`/agents`	Alt ajan yönetimi	Yeni sürümlerde arayüz açmaz, Claude'dan istemeni ya da `.claude/agents/` dosyasını düzenlemeni söyler.

İnceleme ve uzun iş

KOMUT	NE ZAMAN	DİKKAT
`/plan [görev]`	Belirsiz büyük işte önce plan için	Shift+Tab ile de plan moduna geçilir. 3.3
`/goal [koşul]`	Koşul sağlanana kadar Claude'un turlar boyunca sürmesi için	Koşul doğrulanabilir olmalı. 3.1
`/code-review`	Diff'i ya da PR'ı doğruluk hataları için incelemek için	Takma adı `/review`. 12.6
`/security-review`	Dal değişikliklerini güvenlik için incelemek için	`origin` remote gerekir.
`/loop [aralık] [prompt]`	Oturum açıkken bir işi tekrar tekrar koşmak için	Oturum kapanınca durur. Kalıcı iş için `/schedule`. 14.5
`/schedule`	Bulutta zamanlanmış görev kurmak için	Bilgisayar kapalıyken de koşar. 14.1
`/tasks`	Arka plandaki işleri görmek için	Arka plan işi bitmeden turu kapatma. 13.4

CLI bayrakları

BAYRAK	NE YAPAR	DİKKAT
`-c`, `--continue`	Bu dizindeki son oturumu sürdürür	
`-r`, `--resume [ad]`	Adla ya da seçiciyle oturum sürdürür	
`--fork-session`	Sürdürürken yeni oturum kimliği açar	`--resume` ya da `--continue` ile. 2.7
`-n`, `--name`	Oturuma ad verir	Sonra `--resume <ad>` ile bulunur.
`-p`, `--print`	Headless koşum: cevabı yazar, çıkar	Betik ve rutinlerin temeli.
`--output-format`	`text`, `json`, `stream-json`	Yalnız `-p` ile. 14.6
`--json-schema`	Şemaya uyan doğrulanmış JSON	Sonuç `structured_output` alanında. Alan yoksa üretilememiştir. 14.7
`--max-turns`	Tur sınırı, dolunca hatayla çıkar	Yerel 2.1.284 `--help` çıktısında görünmedi, kullanmadan önce dene. 14.2
`--max-budget-usd`	Tahmini harcama sınırı	Yalnız `-p` ile.
`--no-session-persistence`	Oturumu diske yazmaz	Karşılaştırma koşumlarında listeyi kirletmez.
`--model`	Bu oturumun modeli	Yalnız başlatılan oturuma uygulanır. 5.6
`--effort`	Bu oturumun çaba düzeyi	
`--permission-mode`	Başlangıç izin modu	Gözetimsiz koşumda etkin modu ayrıca doğrula. 6.7
`--allowedTools`, `--disallowedTools`	Sormadan izinli ya da yasak araçlar	Kalıcı kural için proje ayarı daha izlenebilir.
`--settings`	Yalnız bu oturum için ayar dosyası ya da JSON	Öncelikte yönetilen katmanın hemen altında. 1.8
`--add-dir`	Ek çalışma dizini	
`--append-system-prompt`	Sistem prompt'unun sonuna ekleme	`--system-prompt` tamamen değiştirir.

BAYRAK	NE YAPAR	DİKKAT
`--mcp-config`	MCP sunucularını JSON'dan yükler	
`-w`, `--worktree`	Ayrı bir git worktree'de başlatır	11.1
`--bare`	Hook, skill, MCP, bellek ve CLAUDE.md keşfini atlar	Sorunun yapılandırmadan gelip gelmediğini ayırmak için.
`--safe-mode`	Bütün özelleştirmeleri kapatarak başlatır	Sorun giderme için.
`claude doctor`	Oturum açmadan kurulum teşhisi	

Klavye kısayolları

KISAYOL	NE YAPAR
`Esc`	Cevabı keser, yapılan iş korunur. İzin isteminde reddeder.
`Esc` `Esc`	Girdi boşsa rewind menüsünü açar. Girdi doluysa taslağı temizler.
`Shift+Tab`	İzin modları arasında döner. Plan modu da bu döngüde.
`Ctrl+C`	Çalışan işi keser. Boş girdide iki kez basınca çıkar.
`Ctrl+B`	Çalışan Bash komutunu ya da ajanı arka plana alır.
`Ctrl+O`	Transkript görünümü: araç çağrılarının ayrıntısı.
`Ctrl+R`	Girdi geçmişinde geriye doğru arar.
`Ctrl+T`	Claude'un görev listesini gösterir ya da gizler.
`Ctrl+G`	Girdiyi metin editöründe düzenler. Uzun prompt için.
`Ctrl+S`	Girdiyi kenara koyar. Boş girdide geri getirir.
`Ctrl+V`	Panodan görsel yapıştırır. iTerm2'de `Cmd+V`.
`Option+P` / `Alt+P`	Girdiyi silmeden model değiştirir.
`\` sonra `Enter`, ya da `Ctrl+J`	Çok satırlı girdi. Her terminalde çalışır.
`!` ile başlamak	Komutu doğrudan kabukta çalıştırır, çıktı oturuma eklenir.
`@` ile başlamak	Dosya yolu tamamlar, dosyayı bağlama ekler.
Boş girdide `?`	Kısayol yardımını gösterir.

macOS'ta `Option` kısayolları için terminalde Option tuşunu Meta olarak ayarlamak gerekir.

Sık karıştırılanlar

- **`/clear` ile `/compact`:** `/clear` konuşmayı sıfırlar. `/compact` özetleyip sürdürür, özette ayrıntı kaybolabilir.
- **`/branch` ile `/fork`:** `/branch` seni yeni dala geçirir. `/fork` kopyayı arka planda koşturur, sen burada kalırsın.
- **`/rewind` ile Git:** Rewind Claude'un dosya araçlarıyla yaptığı değişikliği geri alır. Yalnız Bash'le değişen dosyayı, alt ajan düzenlemelerinin çoğunu ve oturum dışı değişikliği geri almaz. Asıl geri dönüş Git'tir.
- **`/status` ile `/config`:** `/status` neyin yüklendiğini gösterir. `/config` ayarı değiştirir.
- **`/hooks` ile hook testi:** `/hooks` yapılandırmayı gösterir, hook'un doğru çalıştığını kanıtlamaz.

Ek · Sorun giderme

Belirti, ilk ayırıcı kontrol ve sonraki adım. Talimat, izin, hook, bağlam, genişletme ve koşum sorunları. Belge kontrolü: 29 Ekim 2026.

Bir belirti çoğu zaman birkaç nedenden biridir. Aşağıdaki tablo her belirti için **ilk ayırıcı kontrolü** verir: nedenleri birbirinden ayıran en ucuz bakış. Aynı denemeyi tekrarlamak yerine yeni kanıt topla.

Hangi başlığın uyduğunu bilmiyorsan oturumda `/doctor` koş. Claude hiç açılmıyorsa kabukta `claude doctor`. Sorunun senin yapılandırmandan gelip gelmediğini ayırmak için `claude --safe-mode` ya da `claude --bare` ile başlat. Kaynak: resmî sorun giderme.

Talimat ve ayar

BELİRTİ	İLK AYIRICI KONTROL	SONRAKİ ADIM
CLAUDE.md'deki kural uygulanmıyor	`/memory` listesinde dosya var mı?	Yüklenmiyorsa yol ya da import onayı. Yükleniyorsa kural belirsiz ya da başka kuralla çelişiyor. 1.2 1.5
Import edilen dosya bazen görünüyor, bazen görünmüyor	Etkileşimli mi headless mi, worktree'de mi?	Proje dışı import onayı proje yolu başına tutulur. Her oturumda gereken bilgi için SessionStart hook'u. 1.5 8.1
Ayar değişikliği etki etmiyor	`/status` içinde `Setting sources`	Daha yüksek öncelikli katman aynı anahtarı ezmiş olabilir. Listeler birleşir, ezilmez. 1.8
Model ayarı tutmuyor	`env` içinde `ANTHROPIC_MODEL` var mı?	Kabuktaki değişken dosyadaki `model` anahtarını geçer. 1.8
Otomatik bellek eski bilgi veriyor	Kaydın tarihi ve kaynağın bugünkü hali	Eski kaydı düzelt ya da sil. 1.7

İzin ve hook

BELİRTİ	İLK AYIRICI KONTROL	SONRAKİ ADIM
Aynı komut için sürekli izin soruluyor	Komutta `$(...)`, `${...}`, `source` var mı?	Bu parçaları içeren komut allow kuralıyla eşleşmeyip her seferinde sorabilir. Komutu onlarsız yaz, venv'i aktive etmek yerine yorumlayıcıyı doğrudan çağır. 6.4
Gözetimsiz koşum izin ekranında takıldı	Oturumun gerçek etkin modu	Ayarlanan mod ile açılan mod farklı olabilir. Projenin kendi allow listesine bak. 6.7 13.2
Hook hiç çalışmıyor	`/hooks` listesinde var mı, eşleşme deseni doğru mu?	Betiği aynı JSON girdisiyle elle çalıştır. 8.4
Hook çalışıyor ama engellemiyor	Hook async mi, çıkış kodu 2 mi?	Async hook engelleyemez. Olayın exit 2 davranışını belgede kontrol et. 8.5 8.6
İki hook'un sonucu her seferinde farklı	Aynı olaya eşleşen birden çok hook var mı?	Paralel çalışırlar. Bağımlı adımları tek betikte birleştir. 8.7

Bağlam ve performans

BELİRTİ	İLK AYIRICI KONTROL	SONRAKİ ADIM
Bağlam çabuk doluyor	`/context` dağılımı	Büyük dosyayı parça parça okut, geniş aramayı alt ajana ver. 2.4 2.5

BELİRTİ	İLK AYIRICI KONTROL	SONRAKİ ADIM
Otomatik sıkıştırma art arda tetikleniyor	`/context` içinde `Messages` payı	Odak vererek `/compact`, gerekirse `/clear` ve devir dosyası. 2.6
Oturum dondu	`Ctrl+C` tepki veriyor mu?	Vermiyorsa terminali kapat, aynı dizinde `claude --resume`. Konuşma kaybolmaz.
Bellek ya da işlemci çok yüksek	Büyük derleme dizinleri, eklentiler	`.gitignore`'a ekle, `claude --safe-mode` ile eklenti ve MCP'yi ayır, yeniden başlatıp `--continue`.
`@dosya` ya da arama sonuç vermiyor	`claude doctor` çıktısında Search satırı	Paketli ripgrep çalışmıyor olabilir. Belgedeki ripgrep adımları.

Genişletme

BELİRTİ	İLK AYIRICI KONTROL	SONRAKİ ADIM
Skill hiç devreye girmiyor	`/skills` listesinde var mı?	Varsa açıklama, isteğin diliyle eşleşmiyor. Açıklamaya tetik kelimelerini ve sınırını yaz. 9.3 9.8
Yeni skill görünmüyor	Dosya doğru dizinde mi?	`/reload-skills`.
MCP aracı listede ama veri gelmiyor	`/mcp` durumu ve küçük bir okuma denemesi	Bağlantı, yetki ve sorgu kapsamını ayrı sına. 9.7
Alt ajan raporu "bitti" diyor ama iş yok	Diff ve dosya	Rapor kanıt değildir. Dosyanın kendisine bak. 12.3

Doğrulama, koşum ve yayın

BELİRTİ	İLK AYIRICI KONTROL	SONRAKİ ADIM
Testler geçti ama hata sürüyor	Koşulan test sayısı	Filtre hiçbir şeye eşleşmemiş olabilir. Hatayı geri koyup testin düştüğünü gör. 12.1 12.2
Headless betik başarılı diyor ama sonuç boş	JSON'da `is_error` ve `structured_output`	Yalnız `subtype` alanına güvenme. 14.6 14.7
Gece koşumu sabah bitmemiş, çıktı boş	Süreç yaşıyor mu, işlemci süresi ne, makine şarjda mı?	Arka plan işi tur bitince ölmüş ya da makine uyumuş olabilir. Bkz. deneyimler D03 ve D05. 13.3 13.4
Rutin hiç çalışmamış gibi	Zamanlandı mı, başladı mı, çıktı yazdı mı?	Üç aşamayı ayrı kontrol et. Sessizliği başarı sayma. 14.3
Commit'te başkasının dosyaları var	`git log --stat`	Aynı checkout'ta başka oturum stage etmiş. Ayrı worktree. 11.1 11.2
Önizlemede değişiklik görünmüyor	Sunucu hangi dizinden, hangi porttan çalışıyor?	Başka oturumun sunucusuna bakıyor olabilirsin. 11.3
Rewind sonrası bazı dosyalar eski halinde değil	Dosya Bash'le mi değişti?	Checkpoint onu izlemez. Git ile geri al. 15.2
Belgedeki komut yok	`claude --version`	Belge yerel sürümden yeni olabilir. Güncelle ya da `/help` ile kontrol et. 19.6

Yardım isterken

"Çalışmıyor" yerine dört şey yaz: beklenen, gözlenen, tekrar koşulu ve son değişiklik. Büyük log'u sohbete yapıştırma, dosyaya yaz ve ilgili kısmı oku.

```
Beklenen: PreToolUse hook'u rm -rf içeren komutu reddetsin.
Gözlenen: komut sormadan çalıştı.
Tekrar: yeni oturum, proje kökünde, hook .claude/settings.json içinde.
Son değişiklik: hook'a async: true eklendi.
İstek: nedeni ayır, en küçük düzeltmeyi öner, aynı koşulda sına.
```

Ek · Dosya konumları

Ayar, talimat, bellek, skill, MCP, plugin ve oturum verisi nerede durur, ne işe yarar, git'e girer mi. Belge kontrolü: 29 Ekim 2026.

Claude Code davranışını birkaç düzine dosyadan alır ve yaptığı işi de dosyalara yazar. Bir ayar beklediğin gibi çalışmıyorsa, bir bilgi bir oturumda var ötekinde yoksa ya da disk doluyorsa cevap çoğu zaman bu sayfadaki bir yoldadır.

Yollar macOS ve Linux içindir. Windows'ta `~/.claude`, `%USERPROFILE%\.claude` olur. `CLAUDE_CONFIG_DIR` ortam değişkeni verilirse `~/.claude` altındaki her şey, `~/.claude.json` ve kimlik dosyası da o dizine taşınır. Kaynaklar: ayarlar, .claude dizini, bellek, MCP.

Ayar dosyaları

Öncelik yüksekten düşüğe: yönetilen, komut satırı, proje yerel, paylaşılan proje, kullanıcı. İzin listeleri katmanlar arasında birleşir. Hangi dosyaların yüklendiğini `/status` içindeki `Setting sources` satırı gösterir. 1.8

YOL	KAPSAM	NE İÇİN	GİT
`~/.claude/settings.json`	Kullanıcı, bütün projeler	Kişisel izinler, hook'lar, `env`, model, atıf	Girmez
`.claude/settings.json`	Proje, ekip	Ekibin ortak izinleri, hook'ları, plugin'leri	Girer 10.1
`.claude/settings.local.json`	Proje, yalnız sen	Bu projede kişisel ayar. "Bir daha sorma" dediğin izinler buraya yazılır	Girmez. Claude Code dosyayı yaratırken global git excludes'a ekler
`/Library/Application Support/ClaudeCode/managed-settings.json`	Yönetilen (macOS)	Kurum politikası. Kullanıcı ve proje ayarları bunu ezemez	Girmez. IT dağıtır
`/etc/claude-code/managed-settings.json`	Yönetilen (Linux, WSL)	Aynı	Girmez
`C:\Program Files\ClaudeCode\managed-settings.json`	Yönetilen (Windows)	Aynı. Eski `ProgramData` yolu artık okunmaz	Girmez
`managed-settings.d/*.json`	Yönetilen, parçalı	Politikayı ekipler arasında bölmek. Dosyalar alfabetik sırayla birleşir	Girmez
`managed-mcp.json`	Yönetilen	Kurumun sabit MCP sunucu seti	Girmez

Bozuk bir ayar dosyası bütünüyle atlanır, tek satır bile uygulanmaz. Bir ayar hiç etki etmiyorsa önce `/doctor` ile dosyanın ayrıştırılıp ayrıştırılmadığına bak.

`~/.claude.json`

Claude Code'un kendi durum dosyası. Elle düzenlemen gerekmez. İçinde hesap bilgisi, kullanıcı ve local kapsamlı MCP sunucuları, proje başına güven kararları ve `/config` ekranının yazdığı genel ayarlar durur. Giriş anahtarı bu dosyada değildir: macOS'ta Keychain'de, Linux ve Windows'ta `~/.claude/.credentials.json` dosyasındadır.

Dosya her yeniden yazımda eski kopyasını `~/.claude/backups/` altına bırakır, en yeni beşi tutulur.

Talimat dosyaları

YOL	NE ZAMAN YÜKLENİR	GİT
`~/.claude/CLAUDE.md`	Her oturumda, bütün projelerde	Girmez 1.1
`./CLAUDE.md` ya da `./.claude/CLAUDE.md`	Projede her oturumda	Girer 1.2
`./CLAUDE.local.md`	Projede her oturumda, yalnız bu makinede	Girmez, `.gitignore`'a sen eklersin 1.3
Alt dizindeki `CLAUDE.md`	Claude o dizinde bir dosyaya dokununca. Çalışma dizininin üstündekiler açılışta yüklenir	Girer
`.claude/rules/*.md`	`paths` deseniyle eşleşen dosyaya dokununca	Girer 1.4
`~/.claude/rules/*.md`	Aynı, bütün projelerde	Girmez
`AGENTS.md`	Çalışma dizininde ve üstünde hiç CLAUDE dosyası yoksa	Girer 1.6
Yönetilen `CLAUDE.md`	Her oturumda. Kullanıcı dışlayamaz	Girmez

İki ince nokta var. Birincisi, `CLAUDE.local.md` eklemek de `AGENTS.md`'nin okunmasını durdurur. İkisini birlikte istiyorsan `CLAUDE.md` içine `@AGENTS.md` yaz. İkincisi, `--add-dir` ile eklenen dizinin `CLAUDE.md`'si varsayılan olarak yüklenmez.

Otomatik bellek

Claude'un kendi tuttuğu notlar `~/.claude/projects/<proje>/memory/` altında durur. `MEMORY.md` bir dizindir ve her oturumda ilk 200 satırı ya da ilk 25 KB'ı yüklenir. Konu dosyaları açılışta yüklenmez, Claude gerektiğinde okur. Aynı deponun bütün worktree'leri tek bellek dizinini paylaşır. Bellek makineye yereldir, başka makineye ve buluta gitmez. 1.7

Kapatmak için `/memory` ekranı, ayarda `autoMemoryEnabled: false` ya da `CLAUDE_CODE_DISABLE_AUTO_MEMORY=1`. Yerini `autoMemoryDirectory` değiştirir. Alt ajanların kalıcı belleği ayrı tutulur: `agent-memory/<ad>/`.

Skill, ajan, komut ve stil

YOL	NE	GİT
`~/.claude/skills/<ad>/SKILL.md`	Kişisel skill, bütün projelerde	Girmez
`.claude/skills/<ad>/SKILL.md`	Proje skill'i	Girer 9.2
`~/.claude/agents/*.md`, `.claude/agents/*.md`	Alt ajan tanımları	Proje olan girer
`~/.claude/commands/*.md`, `.claude/commands/*.md`	Eski biçim özel komut. Hâlâ çalışır, aynı adlı skill varsa skill kazanır. Yeni iş için skill tercih et	Proje olan girer
`~/.claude/output-styles/`, `.claude/output-styles/`	Cevap üslubu tanımları	Proje olan girer
`~/.claude/keybindings.json`	Klavye kısayolu değişiklikleri. `/keybindings` açar	Girmez
`~/.claude/statusline.sh`	Durum satırı betiği. Yolu `settings.json` içindeki `statusLine` alanında	Girmez 8.3
`.worktreeinclude`	Git'in yok saydığı dosyalardan hangilerinin yeni worktree'ye kopyalanacağı	Girer 11.1

MCP

KAPSAM	NEREDE DURUR	KİMLE PAYLAŞILIR
local (varsayılan)	`~/.claude.json` içinde bu projenin altında	Kimseyle
project	Proje kökünde `.mcp.json`	Ekiple, git üzerinden
user	`~/.claude.json`	Kimseyle, bütün projelerinde açık
managed	Sistem dizininde `managed-mcp.json`	Kurum

İsim karışıklığına dikkat: MCP'nin "local" kapsamı `~/.claude.json` içindedir, `settings.local.json` ile ilgisi yoktur. `.mcp.json` ilk kullanımda onay ister. Headless koşumda soru sormadan yüklenir. 9.7

Hook ve plugin

Hook'lar ayrı bir dosyada değil, ayar dosyalarının `hooks` alanında tanımlanır: kullanıcı, proje, yerel ve yönetilen katman. Bunlara ek olarak plugin içindeki `hooks/hooks.json`, skill ve alt ajan frontmatter'ı da hook taşıyabilir. Bulut oturumları yerel `~/.claude/settings.json` dosyasını okumaz. 8.1

Plugin'ler `~/.claude/plugins/` altında durur. Burada sürüm başına önbellek (`cache/`), plugin'in güncellemelerde korunan verisi (`data/`) ve marketplace kopyaları (`marketplaces/`) vardır. Bir plugin'in kendi yapısında yalnız `.claude-plugin/plugin.json` manifest dizinindedir. `skills/`, `agents/`, `hooks/` ve `.mcp.json` plugin kökünde durur. 9.6

Oturum verisi

Hepsi `~/.claude/` altındadır. Çoğu `cleanupPeriodDays` ayarına göre silinir, varsayılan 30 gündür.

YOL	İÇERİK	SİLİNİR Mİ
`projects/<proje>/<oturum>.jsonl`	Konuşma kaydı: mesajlar, araç çağrıları ve sonuçları	Süre dolunca. Silinen oturum `/resume` listesinden de kalkar
`projects/<proje>/<oturum>/subagents/`, `tool-results/`	Alt ajan kayıtları, taşan büyük araç çıktıları	Aynı
`file-history/<oturum>/`	Claude'un düzenlediği dosyaların önceki hali. `/rewind` buradan geri yükler	Aynı. Silinmişse rewind dosya geri getiremez
`plans/`	Plan modu dosyaları. Yeri `plansDirectory` ile değişir	Aynı
`debug/`	`--debug` ya da `/debug` açıkken oturum günlükleri	Aynı
`shell-snapshots/`	Açılışta alınan kabuk alias ve fonksiyonları	Temiz çıkışta
`usage-data/report.html`	`/insights` raporu	Aynı süreyle
`history.jsonl`	Yazdığın prompt'lar. Yukarı ok ve `Ctrl+R` buradan okur	Sen silene kadar kalır
`todos/`, `statsig/`, `logs/`	Eski sürümlerin dizinleri. Artık yazılmaz	Temizlik içini boşaltır
`.credentials.json`, `agent-memory/`, `jobs/`, `daemon/`	Saklanması gereken durum	Silme

Oturumun geçici çalışma dizini macOS'ta `/private/tmp/claude-<uid>/<proje>/<oturum>/scratchpad/` altındadır. Kayıtla birlikte silinir, işletim sistemi de silebilir. Birden çok gün sürecek işi oraya koyma. 13.3

Mahremiyet

Konuşma kayıtları ve `history.jsonl` şifrelenmez. Claude'un okuduğu her şey, bir `.env` dosyasının içeriği dahil, düz metin olarak diske yazılır. Azaltmanın yolları: `cleanupPeriodDays` değerini düşürmek, `CLAUDE_CODE_SKIP_PROMPT_HISTORY=1` ile kayıt tutmamak, headless koşumda `--no-session-persistence`, gizli dosyaya okuma yasağı. 7.1

Bir projenin bütün yerel izini `claude purge` siler: kayıtlar, bellek, dosya geçmişi ve `~/.claude.json` içindeki proje girdisi. Önce `--dry-run` ile ne gideceğini gör. 2.1.288'den önceki sürümlerde komutun adı `claude project purge`.

Ek · İpuçları ve püf noktaları

Gözden kaçan ayarlar, kısayollar ve tuzaklar: commit imzası, talimat budama, prompt-audit, kısmi sıkıştırma, hook if alanı, ekip politikası. Belge kontrolü: 29 Ekim 2026.

Bu sayfa belgede yazan ama gözden kaçan ayarları, kısayolları ve tuzakları toplar. Her ipucu resmî belgeyle karşılaştırıldı. Belgede olmayan bir yöntem önerildiğinde bu açıkça yazıldı. Bazı ayarlar yeni sürüm ister. Sürüm şartı belgede yazıyorsa burada da yazıyor. Kendi sürümünü `claude --version` ile gör.

Ayarlar

Commit ve PR imzasını kaldır

Claude Code commit'lere varsayılan olarak `Co-Authored-By` satırı, PR gövdesine "Generated with Claude Code" satırı ekler. İkisini de `~/.claude/settings.json` içinden kapat:

```
{
  "attribution": {
    "commit": "",
    "pr": "",
    "sessionUrl": false
  }
}
```

Boş metin o atfı gizler. `sessionUrl: false` bulut oturumlarındaki oturum bağlantısını da kaldırır. İnternette dolaşan `"includeCoAuthoredBy": false` satırı kullanımdan kalktı. Hâlâ okunuyor ama `attribution` ayarlanınca yok sayılıyor, yani yukarıdaki blokla birlikte yazmanın etkisi yok. Kısa yol `"attribution": false` 2.1.281 ve sonrasını ister. Daha eski sürüm bu değeri reddeder ve ayar dosyasının **tamamını** atlar. CLAUDE.md'ye "kendini imzalama" yazmak da işe yarar, ama kalıcı çözüm ayardır. 15.4

Konuşma kayıtlarının ömrünü ayarla

`cleanupPeriodDays` kayıtların kaç gün tutulacağını belirler. Varsayılan 30. Eski bir oturuma aylar sonra dönmek istiyorsan büyüt, gizlilik için küçült. `0` geçersizdir, en az 1. Bu süre dolunca `/rewind` ve `/resume` de o oturumu bulamaz.

```
{ "cleanupPeriodDays": 90 }
```

`auto` ve `bypassPermissions` yalnız kullanıcı dosyasında çalışır

`permissions.defaultMode` için bu iki değer proje ya da yerel ayara yazılırsa etki etmez. `~/.claude/settings.json` dosyasına yaz. Böylece klonladığın bir depo seni sessizce izinsiz moda sokamaz. 6.7

Ortam değişkenini ayar dosyasına koy

`settings.json` içindeki `env` alanı her oturuma ve alt süreçlere ortam değişkeni verir. Kabuk profilini kirletmeden projeye özel değer tanımlarsın.

```
{ "env": { "BASH_DEFAULT_TIMEOUT_MS": "300000" } }
```

Bash komutlarının varsayılan zaman aşımı 120 saniye, üst sınırı 600 saniyedir. Uzun derleme ya da test koşuyorsan bu iki değeri büyütmek, komutun yarıda kesilmesini önler. 13.4

Kendi git kurallarını kullanıyorsan yerleşiklerini kapat

`includeGitInstructions: false` Claude'un bağlamından yerleşik commit ve PR talimatlarını ve git durum özetini çıkarır. Kendi commit skill'in ya da CLAUDE.md kuralların varsa çatışmayı önler.

İki hesap, iki yapılandırma

`CLAUDE_CONFIG_DIR` bütün ayarları, kayıtları ve plugin'leri başka bir dizine taşır. İş ve kişisel hesabı ayırmak için:

```
alias claude-is='CLAUDE_CONFIG_DIR=~/.claude-is claude'
```

Telemetriyi kapatırken dikkat

`DISABLE_TELEMETRY` değişkeni tanımlı olduğu sürece, değeri `0` ya da `false` olsa bile telemetri kapalı kalır. Yeniden açmak için değişkeni kaldırmak gerekir. Bu değişken `/skill-doctor` gibi bazı özellikleri de kapatır.

Güncellemeleri bir hafta geriden al

`autoUpdatesChannel: "stable"` otomatik güncellemeyi yaklaşık bir hafta geriden ve büyük gerilemesi olan sürümleri atlayarak yapar. Varsayılan `latest`. Gözetimsiz koşumların bir sabah yeni sürümle bozulmasını istemiyorsan kullan. 19.6

```
{ "autoUpdatesChannel": "stable" }
```

Cevap dilini sabitle

`language` ayarına yazdığın değer Claude'a "hep bu dilde cevap ver" diye iletilir. Ses dikteyi ve otomatik oturum başlıklarını da etkiler. Değer doğrulanmaz, `"Türkçe"` yazman yeter.

Talimatlar

Yeni model çıkınca talimatlarını buda

Talimatların bir kısmı eski bir modelin eksiğini kapatmak için yazılmıştır. Yeni model o eksiği taşımıyorsa bu satırlar işe yaramaz, hatta zarar verir. Belge iki şey söylüyor. Büyük harfli vurgu ve "CRITICAL: You MUST" tarzı dil güncel modellerde kuralı aşırı uygulatabilir, düz bir cümle yeterli. Yeni model varsayılan haliyle daha iyi sonuç veriyorsa eski talimatları kaldırmayı değerlendir.

Bundan bir deneme yöntemi çıkar. Belgede bu döngü olarak yazmıyor, bir iş akışı önerisidir: şüphelendiğin talimatları kaldır, temsilî birkaç işi yeni modelle koş, yalnız modelin gerçekten kaçırdığı kuralları geri ekle. 5.6 19.3

`/doctor prompt-audit` eski talimatları işaretler

`/doctor prompt-audit` talimat dosyalarını tarar. Eski modeller için yazılmış talimatları, artık var olmayan dosya ya da komuta işaret eden satırları ve birbiriyle çelişen dosyaları raporlar. Sen onaylamadan hiçbir dosyaya dokunmaz. Varsayılan kapsam CLAUDE.md, CLAUDE.local.md, AGENTS.md ve `.claude/` ile `~/.claude/` altındaki kurallar, skill'ler, komutlar, alt ajanlar ve stillerdir. Tek dosya ya da dizin için yol ver. 2.1.283 ve sonrasında var.

```
/doctor prompt-audit
/doctor prompt-audit .claude/skills/deploy
```

CLAUDE.md'ye bağlam harcamayan not yaz

CLAUDE.md içindeki `<!-- ... -->` blok yorumları bağlama girmeden silinir. Bakımcıya not, kuralın gerekçesini ya da tarihini bırakırsın, modelin dikkati dağılmaz. 18.4

Sıkıştırmadan sonra ne kalır

`/compact` sonrasında proje kökündeki CLAUDE.md diskten yeniden yüklenir. Kaybolan yalnız konuşmada verdiğin talimattır. Oturum boyunca geçerli kalması gereken bir kuralı sohbete değil dosyaya yaz. 2.6

Büyük depoda ilgisiz CLAUDE.md'leri dışla

Bir monorepoda başka ekiplerin CLAUDE.md dosyaları da yüklenir. `claudeMdExcludes` glob ya da mutlak yolla bunları dışarıda bırakır. Değerler katmanlar arasında birleşir. Kurumun yönetilen CLAUDE.md'si dışlanamaz.

```
{ "claudeMdExcludes": ["**/legacy/**/CLAUDE.md"] }
```

Ek dizinin talimatını da yükle

`--add-dir` ya da `/add-dir` ile eklenen dizinin skill'leri yüklenir ama CLAUDE.md'si yüklenmez. O dizinin kuralları da geçerli olsun istiyorsan `CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1` ver. 1.5

Oturum

Oturumlara ad ver

`claude -n odeme-refactor` ya da oturum içinde `/rename odeme-refactor`. Sonra `claude --resume odeme-refactor` ile adıyla dönersin. Uzun kimlik aramak gerekmez. 2.7

Uzun prompt'u editörde yaz

`Ctrl+G` girdiyi varsayılan metin editöründe açar. Çok paragraflı bir brief'i terminal satırında yazmaktan kolaydır. `/config` içinden Claude'un son cevabını da editöre yorum olarak getirebilirsin.

Ana işi bölmeden soru sor

`/btw neden bu kütüphaneyi seçtin?` cevabı gösterir ama konuşmaya eklemez. Bağlam büyümez, ana iş yön değiştirmez.

Kod bloğunu tek tuşla al

`/copy` son cevabı panoya kopyalar. Cevapta birden çok kod bloğu varsa seçici açılır, `w` ile seçtiğini dosyaya yazar. SSH üzerinden pano çalışmıyorsa işe yarar.

Bitiş koşulu olan iş için `/goal`

`/goal bütün testler geçiyor ve lint temiz` Claude'u koşul sağlanana kadar turdan tura çalıştırır. Her turdan sonra ayrı bir model koşulu değerlendirir. Koşul doğrulanabilir olmalı. 3.1

Konuşmanın yalnız bir kısmını sıkıştır

`/rewind` menüsünde geri sarmanın yanında "Summarize from here" seçeneği var. Seçtiğin mesajdan sonrasını özetler, öncesini olduğu gibi bırakır. `/compact` bütün konuşmayı özetler. Baştaki kararlar önemliyse ve sondaki uzun deneme gereksizse bu daha az ayrıntı kaybettirir. 2.6

Yazdığın taslağı kaybetme

Girdide yazı varken Esc Esc yazıyı siler ama geçmişe kaydeder, yukarı okla geri gelir. Yarım bir prompt'u kenara koyup başka bir şey sormak için `Ctrl+S` : girdiyi saklar, boş girdide tekrar basınca geri getirir.

Prompt'u silmeden model değiştir

macOS'ta `Option+P` , diğerlerinde `Alt+P` modeli değiştirir ve yazdığın girdi yerinde kalır. Option kısayolları için terminalde Option tuşunu Meta olarak ayarlaman gerekir. `/model` seçicisinde `s` tuşu seçimi yalnız bu oturum için yapar, varsayılanı değiştirmez. 5.2

Kabuk komutunu Claude'a sormadan çalıştır

Satır başına `!` koyarsan komut doğrudan çalışır ve çıktısı oturuma eklenir. `! git status` Claude'un bir tur harcamasından ucuzdur ve Claude çıktıyı görür. Uzun süren komutu `Ctrl+B` arka plana alır.

Kayıt dosyasından oturum aç

`claude --resume` oturum kimliği yerine bir `.jsonl` kayıt yolunu da kabul eder. 2.1.223 ve sonrasında kimlik araması bütün projeleri tarar, oturumu hangi dizinde açtığını hatırlaman gerekmez.

Hedef oturumla birlikte geri gelir

`/goal` ile koyduğun hedef `--continue` ya da `--resume` ile dönünce geri yüklenir. Kaldırmak için `/goal clear`. Zaman aralığıyla tekrar için `/loop` , ayar dosyasında kalıcı bir koşul için Stop hook'u var. 14.5

Bağlam ve çıktı

Büyük araç çıktısının sınırları

Bash çıktısı varsayılan olarak 30.000 karakterde kesilir, `BASH_MAX_OUTPUT_LENGTH` ile 150.000'e kadar çıkar. MCP aracı çıktısının sınırı 25.000 token, `MAX_MCP_OUTPUT_TOKENS` ile değişir. Sınırı büyütmek bağlamı da büyütür. Büyük çıktıyı dosyaya yazdırıp ilgili kısmını okutmak çoğu zaman daha iyidir. 2.5

Çıktı token'ını büyütmenin bedeli

`CLAUDE_CODE_MAX_OUTPUT_TOKENS` tek cevabın uzunluk sınırını büyütür. Ama bu pay bağlamdan ayrılır, otomatik sıkıştırma daha erken tetiklenir. Uzun dosya yazdırırken işe yarar, her oturumda açık bırakma.

Headless koşuma bütçe koy

`claude -p` ile `--max-budget-usd` tahmini harcama sınıra gelince koşumu durdurur. Döngüye giren bir betiğin sabaha kadar harcamasını önler. Tur sınırı için `--max-turns` var, ama yerel 2.1.284 yardım çıktısında görünmüyor, kullanmadan önce dene. 14.2

Sistem prompt'una ekle, değiştirme

`--append-system-prompt` ve `--append-system-prompt-file` varsayılan sistem prompt'unun sonuna metin ekler. `--system-prompt` ise tamamını değiştirir, araç kullanım talimatları dahil. Çoğu iş için ekleme yeter.

Hata ayıklama

`--safe-mode` ile `--bare` farklı işler

`claude --safe-mode` bütün özelleştirmeleri kapatır: CLAUDE.md, skill, plugin, hook, MCP, bellek. Giriş, model ve izinler normal kalır. Bir sorunun senin yapılandırmandan gelip gelmediğini ayırmanın en hızlı yolu budur. `--bare` ise betikler içindir: keşfi atlar, OAuth'u da okumaz, kimlik `ANTHROPIC_API_KEY` ile gelmelidir.

Ayarın neden etki etmediğini bul

Üç kontrol yeter. `/status` içindeki `Setting sources` yüklenen dosyaları gösterir. `/doctor` ayrıştırılamayan ayar dosyasını bulur. `/context` bağlamda gerçekte nelerin durduğunu gösterir. 1.8

Genişletme

Hook'un `if` alanı her olayda çalışmaz

`if` alanı hook'u araç adı ve argümanıyla süzer: `"Bash(git *)"`, `"Edit(*.ts)"`. Eşleşmeyince süreç hiç başlamaz, başlatma maliyeti de olmaz. Ama `if` yalnız araç olaylarında değerlendirilir: PreToolUse, PostToolUse, PostToolUseFailure, PermissionRequest ve PermissionDenied. Başka bir olaya `if` konmuş hook **hiç çalışmaz**. Tek kural alır, `&&` ya da liste kabul etmez. 8.2 8.4

Yan etkili skill'i otomatik çağrıya kapat

Skill frontmatter'ında `disable-model-invocation: true` olan skill'i Claude kendi başına yüklemez. Yalnız sen `/ad` yazınca çalışır. Yayın, silme ya da mesaj gönderme gibi işler için. `context: fork` skill'i ayrı bir alt ajan bağlamında çalıştırır, ana bağlamı doldurmaz. `allowed-tools` yalnız o turda izinsiz kullanılacak araçları açar. 9.4

Worktree'li alt ajan nereden dallanır

Alt ajan tanımında `isolation: worktree` onu geçici bir worktree'de çalıştırır. Bu worktree senin o anki dalından değil, deponun varsayılan dalından açılır. Commit'lenmemiş değişikliklerin ve dalındaki yeni commit'ler alt ajanda görünmez. Değişiklik yoksa worktree kendiliğinden temizlenir. 11.1

Kullanılmayan skill'leri bul

`/skill-doctor` her skill'in bağlamda ne kadar yer tuttuğunu ve ne sıklıkla kullanıldığını gösterir. Hiç çağrılmayan ama açıklaması her oturumda bağlamda duran skill'i kapatmak için. 2.1.252 ve sonrasında var.

Başka araçtan geç

`/import codex`, `/import gemini` ve `/import cursor` o aracın talimatlarını, MCP sunucularını, komutlarını, ajanlarını ve skill'lerini getirir. Önce `--dry-run` ile ne geleceğini gör.

Skill'i yalnız ilgili dosyada yükle

Skill frontmatter'ındaki `paths` alanı skill'i yalnız eşleşen dosyalarla çalışırken otomatik yükler. `user-invocable: false` skill'i `/` menüsünden gizler, yalnız Claude çağırır. Sadece arka plan bilgisi taşıyan skill için uygundur. 9.3

Alt ajanı CLAUDE.md'siz başlat

Alt ajan tanımında `omitClaudeMd: true` onu kullanıcı, proje ve yerel CLAUDE.md olmadan başlatır. Bağımsız bir inceleme ajanının senin varsayımlarınla yüklenmesini istemiyorsan işe yarar. 2.1.271 ve sonrasında var. 9.1

Gitignore'daki dosyaları worktree'ye taşı

Yeni worktree'de `.env` gibi git'in yok saydığı dosyalar olmaz, uygulama açılmaz. Proje kökündeki `.worktreeinclude` dosyasına yazdığın desenler yeni worktree'ye kopyalanır. 11.1

MCP onaylarını sıfırla

`.mcp.json` sunucularına verdiğin "evet" ya da "hayır" kalıcıdır. Yanlış cevapladıysan `claude mcp reset-project-choices`. Yalnız kendi verdiğin MCP yapılandırmasını kullanmak için `--strict-mcp-config` ile `--mcp-config`. 9.7

Arka plan hook'u Claude'u uyandırabilir

`async: true` hook engelleyemez. `asyncRewake` ise arka planda çalışır ve çıkış kodu 2 ile Claude'u boştayken bile uyandırır, çıktısı sistem notu olarak gelir. Uzun bir testin sonucunu beklemeden çalışmaya devam edip bitince haber almak için. 8.6

Kendi kullanımını gör

`/insights` son oturumlarından bir rapor üretir: hangi projelerde ne yaptığın, nerede takıldığın, hangi özellikleri denemediğin. Rapor `~/.claude/usage-data/report.html` dosyasına yazılır. 19.5

Ekip ve güvenlik

Her komutu denetim için sar

`CLAUDE_CODE_SHELL_PREFIX` Bash komutlarını, hook'ları, durum satırını ve yerel MCP sunucularını verdiğin komutla sarar. Kurum, çalıştırılan her komutu bununla günlüğe yazar.

Politikayı ekipler arasında böl

Yönetilen ayar dizinindeki `managed-settings.d/` altına konan `*.json` dosyaları alfabetik sırayla birleşir. Güvenlik ekibi `10-guvenlik.json`, platform ekibi `20-platform.json` yazar, aynı dosyayı paylaşmaz.

Ekibe açılışta duyuru göster

`companyAnnouncements` bir metin dizisidir. Claude Code açılırken bunlardan birini gösterir. Yeni bir kuralı, iç dokümanın adresini ya da bakım saatini duyurmak için. 10.3

Prompt geçmişi süre dolunca silinmez

Konuşma kayıtları `cleanupPeriodDays` sonunda silinir, ama yazdığın prompt'ların tutulduğu `~/.claude/history.jsonl` silinmez. Bir prompt'a gizli bilgi yapıştırdıysan o dosyada kalır. 7.1

Hook'ları kapatmak durum satırını da kapatır

`disableAllHooks: true` bütün hook'ları kapatır. Durum satırı ve dosya önerisi komutu da aynı mekanizmayla çalıştığı için onlar da kapanır. Bir hook'u geçici olarak devre dışı bırakmak istiyorsan yalnız onu kaldır.

Ek · Gerçek deneyimlerden on ders

Tarihli ölçüm defterinden anonimleştirilmiş vakalar: gözlenen, ders, ayırıcı kanıt ve sınır. Belge kontrolü: 29 Ekim 2026.

Bu sayfadaki on vaka, bu listenin yazarının kendi makinesinde tuttuğu tarihli ölçüm defterinden alındı. Proje adları, yollar ve kişisel veriler çıkarıldı. Her vaka bir gözlemdir: o makinede, o sürümde, o koşulda görüldü. Genel kural gibi okuma, kendi ortamında aynı belirtiyi gördüğünde ilk bakılacak yer olarak kullan.

Vakaların ortak dersi şu: en pahalı hatalar bir şeyin kırılmasından değil, bir şeyin **hiçbir şeye bakmadan başarılı görünmesinden** çıkıyor.

D01 · Gözetimsiz oturumda izin modu sessizce değişti

Gözlenen: Zamanlanmış oturumların izinsiz ilerlemesi için `bypassPermissions` modu ayarlanmıştı. Bir projenin 43 oturum kaydındaki 345 mod değeri tarandı. Bu mod hiç görünmüyordu. Mod ilk kullanımda bir onay penceresi istiyor, zamanlanmış oturum pencereyi gösteremediği için oturum hata vermeden varsayılan moda düşüyordu.

Ders: Ayarladığın mod ile oturumun gerçekte açıldığı mod ayrı şeylerdir. Gözetimsiz koşumu ayakta tutan şey projenin kendi izin listesiydi.

Ayırıcı kanıt: Oturum kaydındaki etkin mod değeri. Ayar dosyasındaki değer kanıt değildir.

Sınır: 29 Ağustos 2026 gözlemi. Nedeni o günkü kayıtlardan çıkarıldı. Belge ayrıca proje ya da yerel ayar dosyasına yazılan `bypassPermissions` değerinin etkisiz olduğunu söyler, mod yalnız kullanıcı ayarından gelir. Davranış sürüme ve yönetilen politikaya göre değişebilir. İlgili maddeler: 6.7, 13.2.

D02 · Hiç test koşmayan filtre yeşil döndü

Gözlenen: Test komutuna süitin okunur adı filtre olarak verildi. Eşleşen test olmadı, çıktı "başarılı" dedi. Aynı gün iki kez rapora "test geçiyor" diye girmek üzereydi. Aynı gün iki benzer yeşil daha görüldü: hiçbir dosya bulamadığı için geçen lint ve dar pencere yüzünden körleşen lint.

Ders: Durum satırını değil sayıyı oku. Sıfır test koşmak geçmek değildir.

Ayırıcı kanıt: Çıktıdaki koşulan test sayısı. Filtreyi tip ya da fonksiyon adıyla verince sayı sıfırdan çıktı.

Sınır: Belirli bir test aracının filtre davranışı. Başka araçlarda ilke aynı, sözdizimi farklı. İlgili maddeler: 12.1, 12.2.

D03 · Arka plana atılan iş, tur bitince iz bırakmadan öldü

Gözlenen: Zamanlanmış bir görev uzun bir veri çekimini arka planda başlattı ve turu "bekliyorum" diye kapattı. Komut başlamadan önce rastgele bekleme yapıyordu. Oturum kapanınca süreç öldü, görev çıktısı dizinle birlikte silindi. 83 dakika boyunca ağa tek istek gitmedi ve geriye tek satır kanıt kalmadı.

Ders: Gözetimsiz oturumda uzun işi arka plana atıp turu kapatma. Log'u oturumdan bağımsız bir yola yaz. Aynı iş ön planda ve kalıcı log'la 12 dakikada bitti.

Ayırıcı kanıt: Normal komutun yarım çıktısı log'da kalır. Önce bekleyen komut ise "hiç başlamadı" ile "öldü" arasında fark bırakmaz.

Sınır: 1 Eylül 2026, tek bir masaüstü kurulumu. İlgili maddeler: 13.3, 13.4.

D04 · "Ucuz" kontrol yük altında 120 kat uzadı

Gözlenen: Görev dosyası bir test komutunu "yaklaşık 2,5 saniye" diye tarif ediyordu. Bir nöbet koşumunda aynı komut 300 saniyelik ön plan sınırını aştı ve arka plana düştü. Testler sonunda geçti. Makine yükü o dakikada 68'e çıkmıştı, eşzamanlı oturumlar yüzünden.

Ders: "Ucuz" bir ölçüm değil varsayımdır. Kontrol arka plana düştüyse turu kapatma, bitmesini bekle ve çıkış satırını oku. Yoksa D03 tekrarlanır ve dışarıdaki nöbetçi makineyi ölü sanıp yanlış alarm verir.

Ayırıcı kanıt: `uptime` yük ortalamaları. Bir dakikalık değer on beş dakikalığın birkaç katıysa yük o pencerede fırlamıştır.

Sınır: 2 Eylül 2026. İlgili maddeler: 11.4, 13.4.

D05 · Uyku engelleyici, bataryada uykuyu engellemedi

Gözlenen: Gece koşumu bir uyku engelleyiciyle sarılmıştı. Makine prizde değildi. 19 saatte süreç 1,2 saniye işlemci zamanı kullandı. Sistem günlüğünde aynı gece 257 uykuya giriş kaydı vardı. Çıktı dosyası 19 saat boyunca boş kaldı. Makine prize takılınca aynı koşum 20 dakikada bitti.

Ders: Batarya ve düşük şarj koşumu öldürmez, dondurur. Donma sessizdir. Gece koşumunun ön koşulu makinenin şarjda olmasıdır.

Ayırıcı kanıt: Geçen süreye değil işlemci süresine bak. Oran sıfıra yakınsa süreç uyuyor demektir.

Sınır: macOS, 5 Eylül 2026. İlgili madde: 13.2.

D06 · Başka oturumun commit'i benim değişikliklerimi götürdü

Gözlenen: Değişiklikler stage edildi, ardından üç dakikalık izole derleme yapıldı. Bu arada aynı repoda çalışan başka bir oturum kendi raporunu commit'ledi. Commit index'in tamamını aldı. Dört dosya başka bir mesajın altına girdi, benim commit'im "değişiklik yok" diye boş döndü.

Ders: Git index'i aynı checkout'taki bütün oturumlar arasında ortaktır. Stage ile commit'i tek komutta yap. Arada derleme gerekiyorsa index'i değil geçici bir kopyayı ya da ayrı worktree'yi kullan.

Ayırıcı kanıt: `git log --stat` ile dosyaların hangi commit'e girdiğine bak.

Sınır: 28 Eylül 2026. İçerik doğruydu, yalnız commit mesajı yanlıştı. Paylaşılan dalda geçmiş yeniden yazılmadı. İlgili maddeler: 11.1, 11.2.

D07 · Proje dışı import headless koşumda yüklenmedi

Gözlenen: Ortak kurallar ayrı bir dosyaya taşındı ve dört projenin CLAUDE.md'sine `@~/...` import'u eklendi. `claude -p` ile koşulan oturum dosyanın içeriğini görmedi, yalnız import satırını gördü. Proje dışı import onayı proje yolu başına tutuluyordu. Her worktree ayrı kayıt sayılıyordu, headless koşum onay penceresini hiç göstermiyordu. Symlink de çözmedi.

Ders: Her oturuma girmesi gereken proje dışı bilgiyi import'a bırakma. SessionStart hook'u aynı içeriği her koşumda bağlama koydu ve model kuralı kelimesi kelimesine aktardı.

Ayırıcı kanıt: Dosya okuma araçları kapalı bir headless oturumda "bu kural ne diyor?" diye sor.

Sınır: 15 Ekim 2026. İlgili maddeler: 1.5, 8.1.

D08 · Dört paralel oturum makineyi kilitledi, metin kuralı yetmedi

Gözlenen: Dört alt oturum ayrı worktree'lerde aynı anda derleme ve test koştu. Başka projelerin derlemeleri de sürüyordu. Yük ortalaması 720'ye çıktı, sakin makinede 4'tü. Oturumlara "ağır komutu kilitle sar" diye metin talimatı verildi. Bir oturum kilitsiz başlattığı derlemeyi sonradan durdurmak zorunda kaldı.

Ders: Metin kuralı yeni oturuma ve başka projeye ulaşmaz. Çalışan yol iki parçalıydı: ağır komutları iki yuvalı kilitle koşturan bir sarmalayıcı ve sırasız ağır komutu reddeden bir PreToolUse hook'u.

Ayırıcı kanıt: Hook 16 vakalık bir testle sınandı, sıfır yanlış verdi. Hafif komut istisnası bozulunca iki yanlış verdi, yani test gerçekten bakıyordu.

Sınır: 26 Ekim 2026. Tek yuva fazla dar çıktı: uzun bir cihaz testi kilidi 30 dakika tuttu, diğerleri boşta bekledi. İlgili maddeler: 8.4, 11.4, 11.6.

D09 · Geçici dizin toptan silindi, çok günlük iş gitti

Gözlenen: Bir yazı serisinin kaynakları, görselleri ve durum dosyası oturumun geçici çalışma dizinindeydi. Dizin ağacı bir akşam toptan silindi. Metinler alt ajan kayıtlarındaki dosya okuma sonuçlarından geri kuruldu. Görsellerin yerel kopyası geri gelmedi.

Ders: Birden çok gün sürecek işin kaynağı geçici dizinde tutulmaz. Proje dizinine ya da kalıcı bir yola yaz. Geçici dizin yalnız o turun ara çıktısı içindir.

Ayırıcı kanıt: Yeniden yaratılan dizinlerin hepsi aynı dakikanın zaman damgasını taşıyordu.

Sınır: 27 Ekim 2026. Silmenin nedeni ölçülmedi. İlgili maddeler: 13.3, 18.2.

D10 · Eski rapor, canlı sistemin yerine okundu

Gözlenen: On gün önceki bir denetim "uygulama yalnız bir ülkede satışta" diyordu. Cümle aynı gün iki kez tekrarlandı, biri kullanıcının yapıştırdığı notta. Sayfaya bakılmadan doğru kabul edildi ve buna göre bir karar önerildi. Yönetim panelinde uygulama 175 ülkede satıştaydı. Aynı oturumda ikinci bir örnek: bir sürüm notundaki "hepsinin görseli var" iddiası yeni sürümde 11 eksik görselle çürüdü.

Ders: Mağaza ayarı, sayı, kapsam gibi değişebilen bir durumu eski rapordan alma. Kullanıcının yapıştırdığı not da bir rapordur. Yazmadan ya da öneri yapmadan önce kaynağın kendisine bak.

Ayırıcı kanıt: Kaynağın şu anki değeri ve okuma tarihi.

Sınır: 28 Ekim 2026. İlgili maddeler: 12.3, 18.4.

Ek · Terimler sözlüğü

Listede geçen terimlerin bir cümlelik karşılıkları, konu başına gruplanmış. Belge kontrolü: 29 Ekim 2026.

Terimler ilk geçtiği maddede kısaca tanımlanır. Bu sayfa hepsini bir arada verir. Her satır terimin bu listede ne anlama geldiğini söyler. Ürün davranışının tamamı için ilgili resmî belgeye bak.

Oturum ve bağlam

TERİM	BU LİSTEDE NE DEMEK
Oturum	Claude ile tek bir konuşma ve onun araç geçmişi. Kapatıp açınca konuşma bağlamı sıfırlanır, dosyalar kalır.
Bağlam (context)	Modelin o anda görebildiği her şey: mesajlar, okunan dosyalar, araç çıktıları, yüklenen talimatlar. Sınırlıdır ve dolar.
Token	Modelin metni işlediği parça. Bağlam ve kullanım token'la ölçülür. Kelimeyle birebir eşleşmez.
Sıkıştırma (`/compact`)	Uzayan konuşmanın özetle değiştirilmesi. Yer açar, ayrıntı kaybettirebilir.
Checkpoint	Claude'un dosya düzenlemelerinden önce aldığı yerel geri dönüş noktası. Git'in yerini tutmaz.
Devir dosyası	Oturum kapanmadan önce durumun yazıldığı dosya. Yeni oturum oradan devam eder.
Otomatik bellek	Claude'un oturumlar arasında kendi tuttuğu notlar. Talimat dosyasından ayrıdır, eskiyebilir.

Talimat ve ayar

TERİM	BU LİSTEDE NE DEMEK
CLAUDE.md	Her oturumun başında yüklenen talimat dosyası. Üç düzeyi var: kullanıcı, proje ve alt dizin.
`CLAUDE.local.md`	Yalnız bu makinede geçerli proje notu. Git'e girmez.
Kural dosyası (`.claude/rules/`)	Yalnız eşleşen dosyalara dokunulunca yüklenen talimat.
`settings.json`	İzin, hook, model ve ortam ayarlarının dosyası. Kullanıcı, proje, yerel ve yönetilen katmanları vardır.
Yönetilen politika	Kurumun yönettiği ve kullanıcının ezemediği ayar katmanı.
Prompt	Claude'a verilen istek metni.
Output style	Claude'un cevap üslubunu değiştiren hazır ya da özel ayar.

İzin ve güvenlik

TERİM	BU LİSTEDE NE DEMEK
İzin modu	Claude'un araç kullanmadan önce ne zaman onay soracağını belirleyen mod. Plan modu bunlardan biridir.
Allow / ask / deny	İzin kuralının üç sonucu: sormadan izin ver, sor, reddet.
Plan modu	Claude'un yalnız okuyup plan yazdığı, dosya değiştirmediği mod.
Sandbox	Komutların dosya sistemi ve ağ erişimini işletim sistemi düzeyinde sınırlayan yalıtım.

TERİM	BU LİSTEDE NE DEMEK
Prompt injection	Okunan sayfa, belge ya da e-postadaki metnin kullanıcı talimatı gibi davranmaya çalışması.

Genişletme

TERİM	BU LİSTEDE NE DEMEK
Hook	Belirli bir olayda (araç öncesi, araç sonrası, oturum başı gibi) otomatik çalışan betik. Modelin kararına bağlı değildir.
Skill	Gerektiğinde yüklenen talimat ve dosya paketi. Açıklaması her zaman bağlamdadır, gövdesi yalnız kullanılınca yüklenir.
Slash komutu	`/` ile başlayan komut. Bir kısmı yerleşik, bir kısmını skill ve plugin'ler ekler.
Alt ajan (subagent)	Kendi bağlamında çalışan ve sonucu ana oturuma özet olarak dönen yardımcı Claude.
MCP	Claude'u dış sistemlere (veritabanı, iş takibi, tarayıcı gibi) araç olarak bağlayan açık protokol.
Plugin	Skill, ajan, hook ve MCP ayarlarını tek pakette dağıtma biçimi.
Statusline	Terminal altında sürekli görünen, kendi betiğinle üretilen durum satırı.

Çalışma biçimi

TERİM	BU LİSTEDE NE DEMEK
Headless (`claude -p`)	Etkileşimsiz koşum. İstek verilir, sonuç metin ya da JSON olarak döner, oturum biter.
Effort	Modelin düşünmeye ayıracağı çaba düzeyi.
Worktree	Aynı Git deposunun ayrı bir klasördeki ikinci çalışma kopyası. Paralel yazarları ayırır.
Rutin	Zamanlanmış ya da tetiklenmiş, gözetimsiz koşan Claude görevi. Bulutta ya da masaüstünde çalışır.
Gözetimsiz koşum	Kullanıcı başında yokken ilerleyen iş. Soru soramaz, izin bekleyemez.
Advisor	Zor kararda ikinci, daha güçlü bir modelin görüşünü alma özelliği.
Harness	Modelin çevresindeki çalışma düzeni: araçlar, izinler, bağlam yönetimi. Claude Code bir harness'tır.

Kendi puan kartın

Her bölüm için kaç maddenin yapıldığını, kaçının sırada olduğunu yaz. Üç ay sonra yeniden doldur.

#	BÖLÜM	MADDE	YAPILDI	SIRADA	GEREK YOK
1	Talimat katmanları	8			
2	Bağlam yönetimi	7			
3	Görev verme	7			
4	Prompt yazımı: önce ve sonra	4			
5	Model, effort ve maliyet	6			
6	İzinler	7			
7	Güvenlik ve gizlilik	7			
8	Hook'lar	7			
9	Skill, komut, ajan, MCP	8			
10	Ekiple kullanım	3			
11	Paralel çalışma	6			
12	Doğrulama	7			
13	Gözetimsiz koşum	6			
14	Rutinler ve zamanlama	7			
15	Git ve yayın	4			
16	Tasarım ve arayüz	4			
17	Mobil ve simülatör	5			
18	Bilgi birikimi	4			
19	Bakım ritmi	6			
	Toplam	**113**			

29 Ekim 2026 · Kaynak kontrolü; yerel CLI 2.1.284. Özellikler hızlı değişiyor; bir komut ya da ayar adı çalışmıyorsa code.claude.com/docs adresindeki güncel belgeye bak.